Sicurezza dei Sistemi

Appunti del Corso

4 marzo 2026

Indice

1 Fondamenti di Sicurezza **10**
1.1 Introduzione alla Sicurezza Informatica . . . 10
1.1.1 La Triade CIA . . . 10
1.1.2 Livelli di Impatto delle Violazioni . . . 10
1.2 Le Sfide della Sicurezza Informatica . . . 11
1.3 Terminologia della Sicurezza . . . 11
1.3.1 Modelli di Controllo degli Accessi . . . 12
1.4 Tipologie di Attacco e di Minacce . . . 12
1.4.1 Classificazione degli Attacchi . . . 12
1.4.2 Le Minacce . . . 12
1.5 Valutazione delle Minacce . . . 13
1.5.1 Superfici di Attacco . . . 13
1.6 Minacce alle Risorse del Sistema . . . 14
1.6.1 Hardware . . . 14
1.6.2 Software . . . 14
1.6.3 Dati . . . 14

2 Crittografia e Funzioni Hash **15**
2.1 Cifratura Simmetrica . . . 15
2.1.1 I Cinque Componenti di uno Schema Simmetrico . . . 15
2.1.2 Requisiti per un Utilizzo Sicuro . . . 15
2.1.3 Metodi di Attacco alla Cifratura Simmetrica . . . 15
2.2 Funzioni Hash e Digest . . . 16
2.2.1 Differenza Fondamentale: Hash vs Crittografia . . . 16
2.3 Analisi degli Algoritmi: DES, Triple DES e AES . . . 17
2.3.1 DES (Data Encryption Standard) . . . 17
2.3.2 Triple DES (3DES) . . . 17
2.3.3 AES (Advanced Encryption Standard) . . . 17
2.4 Modalità Operative e Cifrari . . . 17
2.4.1 Il Problema della Modalità ECB (Electronic Codebook) . . . 17
2.4.2 Cifrari a Blocchi vs Cifrari a Flusso . . . 18
2.5 Autenticazione dei Messaggi . . . 18
2.5.1 Il Limite della Sola Cifratura . . . 18
2.5.2 Autenticazione Tramite Tag (MAC) . . . 18
2.6 Requisiti delle Funzioni Hash One-Way . . . 19
2.7 Crittografia a Chiave Pubblica (Asimmetrica) . . . 19
2.7.1 Miti Comuni sulla Crittografia a Chiave Pubblica . . . 19
2.7.2 I Sei Componenti di uno Schema a Chiave Pubblica . . . 20
2.7.3 Il Processo di Comunicazione Sicura . . . 20
2.7.4 Requisiti per la Crittografia a Chiave Pubblica . . . 20
2.7.5 Panoramica degli Algoritmi Asimmetrici . . . 21
2.8 Firme Digitali . . . 21
2.8.1 Il Processo di Firma e Verifica . . . 21
2.9 Certificati a Chiave Pubblica . . . 22
2.9.1 Struttura e Ruolo della CA . . . 22

2.9.2 Processo di Creazione e Verifica 22
2.10 Numeri Casuali e Dati Memorizzati 22
2.10.1 L'Uso dei Numeri Casuali 22
2.10.2 Cifratura dei Dati Memorizzati (Data at Rest) 22

3 Crittografia 23
3.1 Crittografia Simmetrica 23
3.1.1 Introduzione 23
3.1.2 Requisiti 23
3.1.3 Attacco alla Crittografia Simmetrica 23
3.1.4 Algoritmi a Crittografia Simmetrica 23
3.2 Autenticazione dei Messaggi 24
3.2.1 Autenticazione Senza Cifratura del Messaggio 24
3.2.2 Message Authentication Code 25
3.2.3 Funzione Hash Unidirezionale 25
3.2.4 Proprietà della Funzione Hash 25
3.2.5 Integrità dei Dati 26
3.2.6 Garantire la Riservatezza 26
3.3 Crittografia Asimmetrica 26
3.3.1 Introduzione 26
3.3.2 Componenti di uno Schema a Chiave Pubblica 26
3.3.3 Funzionamento Base 27
3.3.4 Garantire l'Autenticità 27
3.3.5 Busta Digitale 27

4 Autenticazione Digitale degli Utenti 28
4.1 Introduzione 28
4.2 Modello di Autenticazione Digitale 28
4.3 Mezzi di Autenticazione 28
4.4 Autenticazione Multifattore 29
4.5 Rischi nell'Autenticazione degli Utenti 29
4.5.1 Livello di Garanzia 29
4.5.2 Impatto Potenziale 29
4.5.3 Aree di Rischio e Corrispondenza 30
4.6 Autenticazione Basata su Password 30
4.6.1 Vulnerabilità delle Password e Contromisure 30
4.6.2 Persistenza delle Password 31
4.7 Hash delle Password 31
4.7.1 Hashing con Salt 32
4.7.2 Vantaggi del Salt 32
4.8 Cracking delle Password e Contromisure 32
4.8.1 Approcci Tradizionali 32
4.8.2 Approcci Moderni 33
4.8.3 Contromisure 33
4.9 Autenticazione Basata su Token 34
4.9.1 Memory Card 34
4.9.2 Smart Card 35
4.10 Autenticazione Biometrica 35
4.10.1 Tipologie di Caratteristiche Biometriche 35

4.10.2 Accuratezza e Complessità 36

5 Controllo degli Accessi 37
5.1 Introduzione 37
5.1.1 Obiettivi Principali 37
5.1.2 Concetti di Base 37
5.1.3 Contesto del Controllo degli Accessi 37
5.2 Politiche di Controllo degli Accessi 38
5.3 Controllo degli Accessi Discrezionale (DAC) 38
5.3.1 Matrice degli Accessi 38
5.3.2 Liste di Controllo degli Accessi 38
5.3.3 Capability Ticket 38
5.3.4 Tabella di Autorizzazione 39
5.4 Controllo degli Accessi Obbligatorio (MAC) 39
5.5 Controllo degli Accessi Basato sui Ruoli (RBAC) 39
5.5.1 Struttura del Modello 39
5.6 Controllo degli Accessi Basato sugli Attributi (ABAC) 39
5.7 Gestione delle Identità, delle Credenziali e degli Accessi 40
5.7.1 Gestione delle Identità 40
5.7.2 Gestione delle Credenziali 40
5.7.3 Gestione degli Accessi 41
5.7.4 Federazione delle Identità 41
5.8 Trust Framework 41
5.8.1 Scambio di Informazioni 41
5.8.2 Open Identity Trust Framework (OITF) 42
5.8.3 Componenti Principali dell'OITF 42

6 SQL Injection e Sicurezza dei Database 43
6.1 SQL Injection 43
6.1.1 Come Funzionano le SQLi 43
6.1.2 Fonti di Attacco 43
6.2 Tipi di SQL Injection 43
6.2.1 Attacco in Banda 43
6.2.2 Attacco Inferenziale 45
6.2.3 Attacco Fuori Banda 46
6.3 Contromisure per le SQL Injection 46
6.3.1 Programmazione Difensiva 46
6.3.2 Rilevamento 46
6.3.3 Prevenzione a Tempo di Esecuzione 47
6.4 Controllo degli Accessi ai Database 47
6.4.1 Definizione dei Permessi Basati su SQL 47
6.4.2 Autorizzazioni in Cascata 48
6.4.3 Controllo degli Accessi Basato sui Ruoli 48
6.5 Inferenza nei Database 49
6.5.1 Contromisure 49
6.6 Cifratura dei Database 49

7 Malware **50**
7.1 Introduzione . . . 50
7.1.1 Classificazione dei Malware . . . 50
7.1.2 Propagazione dei Malware . . . 50
7.2 Tipi di Malware . . . 50
7.2.1 Advanced Persistent Threat (APT) . . . 50
7.2.2 Adware . . . 50
7.2.3 Exploit Kit . . . 51
7.2.4 Backdoor . . . 51
7.2.5 Exploits . . . 51
7.2.6 Flooders (DoS Client) . . . 51
7.2.7 Keyloggers . . . 51
7.2.8 Logic Bomb . . . 51
7.2.9 Macro Virus . . . 51
7.2.10 Mobile Code . . . 52
7.2.11 Rootkit . . . 52
7.2.12 Spyware . . . 52
7.2.13 Trojan . . . 52
7.2.14 Virus . . . 52
7.2.15 Worm . . . 52
7.2.16 Zombie, Bot . . . 52
7.3 Virus . . . 52
7.3.1 Introduzione . . . 52
7.3.2 Struttura Generale . . . 53
7.3.3 Compressione del File Infetto . . . 53
7.3.4 Fasi di un Virus . . . 53
7.3.5 Classificazione dei Virus . . . 54
7.3.6 Macro Virus e di Scripting . . . 55
7.4 I Worm . . . 55
7.4.1 Meccanismi di Diffusione . . . 55
7.4.2 Ricerca del Target . . . 56
7.4.3 Propagazione del Worm . . . 56
7.5 Vulnerabilità Lato Client . . . 57
7.5.1 Drive-by-Download . . . 57
7.5.2 Watering Hole . . . 57
7.5.3 Malvertising . . . 57
7.5.4 Clickjacking . . . 57
7.6 Trojan . . . 57
7.7 Payload di un Malware . . . 58
7.7.1 Distruzione dei Dati . . . 58
7.7.2 Ransomware . . . 58
7.7.3 Bombe Logiche . . . 59
7.8 Agenti di Attacco . . . 59
7.8.1 Bot . . . 59
7.8.2 Botnet . . . 59
7.8.3 Uso dei Bot . . . 59
7.8.4 Controllo Remoto . . . 60
7.8.5 Contromisure . . . 61

7.9 Furto di Credenziali . . . 61
7.9.1 Keylogger . . . 61
7.9.2 Spyware . . . 61
7.9.3 Phishing . . . 62
7.10 Stealthing . . . 63
7.10.1 Backdoor . . . 63
7.10.2 Rootkit . . . 63

8 Attacchi Denial-of-Service 65
8.1 Introduzione . . . 65
8.1.1 Gli Attacchi DDoS . . . 65
8.2 TCP SYN Flood . . . 65
8.2.1 Funzionamento del TCP SYN Flood . . . 65
8.2.2 Come si Verifica un TCP SYN Flood . . . 66
8.2.3 Tipologie di Attacco SYN Flood . . . 66
8.2.4 Mitigazione degli Attacchi SYN Flood . . . 66
8.3 ICMP Flood . . . 67
8.3.1 Funzionamento dell'ICMP Flood . . . 67
8.3.2 Provocare un ICMP Flood . . . 67
8.3.3 Mitigare l'ICMP Flood . . . 67
8.4 UDP Flood . . . 67
8.4.1 Funzionamento dell'UDP Flood . . . 67
8.4.2 Parallelismo dell'UDP Flood . . . 68
8.4.3 Mitigare l'UDP Flood . . . 68
8.5 HTTP Flood . . . 68
8.5.1 Funzionamento dell'HTTP Flood . . . 68
8.5.2 Tipologie di Attacco HTTP Flood . . . 68
8.5.3 Mitigare un HTTP Flood . . . 69
8.6 DNS Amplification Attack . . . 69
8.6.1 Reflection Attack . . . 69
8.6.2 Amplification Attack . . . 69
8.6.3 DNS Amplification Attack . . . 69
8.6.4 Mitigare il DNS Amplification Attack . . . 70

9 Intrusion Detection System 71
9.1 Introduzione . . . 71
9.2 Struttura di un IDS . . . 71
9.3 Configurazioni di IDS . . . 71
9.4 Principi di Base di un IDS . . . 71
9.5 Requisiti di un IDS . . . 72
9.6 Analisi di un IDS . . . 72
9.6.1 Approcci di Analisi . . . 72
9.7 Host-Based Intrusion Detection System . . . 73
9.7.1 Architetture Centralizzate vs Decentralizzate . . . 74
9.7.2 Architettura di un HIDS Distribuito . . . 74
9.7.3 Processo di Rilevazione degli Eventi Sospetti . . . 74
9.7.4 Monitoraggio della LAN . . . 75
9.8 Network-Based Intrusion Detection System . . . 75
9.9 IDS Distribuito . . . 75

9.9.1 Problemi Chiave per i Sistemi IDS 75
9.9.2 Attacchi e Metodi di Intrusione Moderni 75
9.9.3 Sistemi Cooperativi e Rilevamento Distribuito 76
9.9.4 Vantaggi del Rilevamento Distribuito 76
9.9.5 Implementazione del Sistema IDS Ibrido 76
9.9.6 Comportamento del Sistema Centrale e Sensori Distribuiti . . 76
9.10 Honeypot 77
9.10.1 Caratteristiche degli Honeypot 77
9.10.2 Obiettivi degli Honeypot 77
9.10.3 Classificazione degli Honeypot 77
9.10.4 Evoluzione degli Honeypot 78
9.10.5 Posizionamento degli Honeypot nella Rete 79

10 Firewall **80**
10.1 Introduzione 80
10.2 Requisiti Minimi 80
10.3 Politiche d'Accesso 80
10.4 Caratteristiche di un Firewall 80
10.5 Filtri del Firewall 80
10.5.1 Firewall a Filtraggio di Pacchetto 81
10.5.2 Tipi di Attacchi al Firewall a Filtraggio di Pacchetto 81
10.5.3 Firewall a Filtraggio di Pacchetto con Stato 81
10.6 Tipi di Firewall 82
10.6.1 Gateway a Livello Applicativo 82
10.6.2 Proxy a Livello di Circuito 82
10.7 Classificazione per Locazione 83
10.7.1 Bastion Host 83
10.7.2 Host-Based Firewall 84
10.7.3 Personal Firewall 84
10.8 Architetture di Sicurezza di Rete 84
10.8.1 La Rete DMZ 84
10.8.2 La Rete VPN 85
10.8.3 Firewall Distribuiti 85
10.8.4 Topologia delle Reti con Firewall 86
10.9 Intrusion Prevention System 86
10.9.1 Introduzione 86
10.9.2 Rilevamento degli Attacchi di un IPS 86
10.9.3 Host-Based IPS (HIPS) 87
10.9.4 Network-Based IPS (NIPS) 87
10.9.5 Sistema Immunitario Digitale 88

11 Buffer Overflow **89**
11.1 Principi del Buffer Overflow 89
11.1.1 Cause e Conseguenze 89
11.1.2 Requisiti per Sfruttare un Buffer Overflow 89
11.2 Shellcode 90
11.2.1 Caratteristiche dello Shellcode 90
11.2.2 Sviluppo dello Shellcode 90
11.2.3 Restrizioni 90

11.3 Difese Contro i Buffer Overflow 91
11.3.1 Difese a Tempo di Compilazione 91
11.3.2 Difese a Tempo di Esecuzione 92
11.4 Altre Forme di Attacchi Overflow 92
11.4.1 Sostituzione dello Stack Frame 92
11.4.2 Ritorno alla Chiamata di Sistema 93
11.4.3 Heap Overflow 93
11.4.4 Overflow nell'Area Dati Globale 93

12 Sicurezza del Software 94
12.1 Problemi di Sicurezza del Software 94
12.2 Sicurezza vs. Qualità del Software 94
12.3 Programmazione Difensiva 94
12.4 Gestione Sicura dell'Input 94
12.4.1 Dimensione dell'Input e Buffer Overflow 95
12.4.2 Interpretazione dell'Input e Attacchi di Injection 95
12.4.3 Tecniche di Validazione dell'Input 95
12.5 Scrittura di Codice Sicuro e Gestione della Memoria 96
12.5.1 Corretta Implementazione dell'Algoritmo 96
12.5.2 Tipizzazione, Memory Leak e Race Condition 96
12.6 Interazione con il Sistema Operativo e Privilegi 96
12.6.1 Il Principio del Minimo Privilegio (*Least Privilege*) 97
12.7 Gestione Sicura dell'Output e Cross-Site Scripting (XSS) 97
12.8 Analisi delle Vulnerabilità e Sicurezza delle Dipendenze 97

13 Sicurezza nei Sistemi Operativi e Virtualizzazione 98
13.1 Introduzione alla Sicurezza dei Sistemi Operativi 98
13.2 Il Processo di Messa in Sicurezza (NIST SP 800-123) 98
13.3 Hardening del Sistema Operativo 99
13.3.1 Manutenzione, Logging e Backup 99
13.4 Sicurezza in Ambiente Linux/Unix 99
13.5 Sicurezza in Ambiente Windows 100
13.6 Sicurezza della Virtualizzazione 100
13.6.1 Tipologie di Hypervisor 100
13.6.2 Minacce e Isolamento 100
13.7 Metasploit Framework: Penetration Testing e Assessment 101
13.7.1 Architettura e Moduli 101
13.7.2 Considerazioni Etiche e Legali 101

14 Cloud Computing e Internet of Things (IoT) 102
14.1 Cloud Computing: Definizioni e Architettura 102
14.1.1 Definizione e Caratteristiche Essenziali 102
14.1.2 Modelli di Servizio Cloud 102
14.1.3 Modelli di Deployment 103
14.2 Sicurezza nel Cloud Computing 104
14.2.1 Multi-Tenancy e Isolamento 104
14.2.2 Sicurezza dei Dati (*Data at Rest* e *Data in Transit*) 104
14.2.3 Security as a Service (SecaaS) 104
14.2.4 Il Modulo di Sicurezza OpenStack Keystone 104

14.3 Internet of Things (IoT) e Livelli di Rete 104
14.3.1 Architettura di Rete IoT 105
14.4 Sicurezza nell'Ambiente IoT 105
14.4.1 Requisiti e Framework Operativi 105
14.4.2 Sicurezza Open-Source: Il modulo MiniSec 105

15 Sicurezza IT e Prevenzione dei Rischi 107
15.1 Definizione e Ciclo di Gestione 107
15.1.1 Il Modello Plan-Do-Check-Act (PDCA) 107
15.2 Contesto e Policy di Sicurezza 107
15.3 Il Processo di Risk Assessment 108
15.3.1 Approcci alla Valutazione del Rischio 108
15.4 Analisi Dettagliata del Rischio 108
15.4.1 1. Contesto e Identificazione degli Asset 108
15.4.2 2. Identificazione di Minacce e Vulnerabilità 108
15.4.3 3. Analisi e Calcolo del Rischio 109
15.5 Valutazione e Trattamento del Rischio 109

16 Controlli di Sicurezza 110
16.1 Classificazione dei Controlli di Sicurezza 110
16.2 Selezione e Analisi Costi-Benefici 110
16.3 Monitoraggio e Gestione dei Cambiamenti 110

17 Sicurezza Fisica e delle Infrastrutture 111
17.1 Panoramica sulla Sicurezza Fisica 111
17.2 Minacce alla Sicurezza Fisica 111
17.2.1 Catastrofi Naturali e Minacce Ambientali 111
17.2.2 Minacce Tecniche 112
17.2.3 Minacce Causate dall'Uomo 113
17.3 Misure di Prevenzione e Mitigazione della Sicurezza Fisica 113
17.3.1 Minacce Ambientali 113
17.3.2 Minacce Tecniche 114
17.3.3 Minacce Causate dall'Uomo 115
17.4 Ripristino da Violazioni della Sicurezza Fisica 115
17.4.1 Ripristino dei Dati e dei Servizi 115
17.4.2 Ripristino dell'Attrezzatura e del Sito 115

18 Algoritmi di Crittografia 116
18.1 Principi degli Algoritmi Crittografici 116
18.2 Data Encryption Standard (DES) 116
18.3 Triple DES (3DES) 117
18.4 Advanced Encryption Standard (AES) 117
18.4.1 Panoramica dell'Algoritmo 117
18.4.2 Dettagli delle Trasformazioni AES 118
18.5 Cifrari a Flusso (Stream Ciphers) e RC4 118
18.5.1 Struttura e Funzionamento 118
18.5.2 L'Algoritmo RC4 118
18.6 Modalità Operative dei Cifrari a Blocchi 119
18.7 Distribuzione delle Chiavi 119

19 Crittografia a Chiave Pubblica e Autenticazione del Messaggio 120
19.1 Funzioni di Hash Sicure . . . 120
19.1.1 Funzioni di Hash Semplici . . . 120
19.1.2 La Funzione di Hash Sicura SHA . . . 121
19.1.3 Elaborazione di SHA-512 . . . 121
19.2 HMAC . . . 121
19.2.1 Obiettivi di Progettazione . . . 122
19.2.2 Algoritmo HMAC . . . 122
19.2.3 Sicurezza di HMAC . . . 122
19.3 Cifratura Autenticata . . . 122
19.3.1 Funzionamento di OCB . . . 123
19.4 L'Algoritmo di Cifratura a Chiave Pubblica RSA . . . 123
19.4.1 Descrizione dell'Algoritmo . . . 123
19.4.2 Generazione delle Chiavi . . . 123
19.4.3 Esempio di Funzionamento . . . 124
19.4.4 La Sicurezza di RSA . . . 124
19.5 Diffie-Hellman e Altri Algoritmi Asimmetrici . . . 124
19.5.1 L'Algoritmo Diffie-Hellman . . . 124

20 Sicurezza su Internet: Protocolli e Standard 125
20.1 Secure E-mail e S/MIME . . . 125
20.1.1 MIME . . . 125
20.1.2 S/MIME e Funzionalità . . . 125
20.1.3 Funzionamento Generale . . . 126
20.2 DomainKeys Identified Mail (DKIM) . . . 126
20.2.1 Architettura della Posta su Internet . . . 126
20.2.2 Strategia di DKIM . . . 127
20.3 Secure Sockets Layer (SSL) e Transport Layer Security (TLS) . . . 127
20.3.1 Architettura TLS . . . 127
20.3.2 Protocolli TLS . . . 127
20.3.3 Attacchi a SSL/TLS: Heartbleed . . . 128
20.4 HTTPS . . . 128
20.5 Sicurezza di IPv4 e IPv6 (IPsec) . . . 128
20.5.1 Associazioni di Sicurezza (SA) . . . 128
20.5.2 Encapsulating Security Payload (ESP) . . . 128

21 Autenticazione su Internet 129
21.1 Kerberos . . . 129
21.1.1 Il Protocollo Kerberos . . . 129
21.1.2 Realm Kerberos e Kerberi Multipli . . . 130
21.1.3 Versioni di Kerberos . . . 130
21.2 X.509 . . . 130
21.2.1 Formato del Certificato . . . 130
21.2.2 Revoca dei Certificati . . . 131
21.3 Public-Key Infrastructure (PKI) . . . 131
21.3.1 Modello di Trust e Problemi . . . 131
21.3.2 Public Key Infrastructure X.509 (PKIX) . . . 131

1 - Fondamenti di Sicurezza

1.1 Introduzione alla Sicurezza Informatica

Per sicurezza informatica si intendono una serie di misure e controlli che assicurano la confidenzialità, l'integrità e la disponibilità delle risorse del sistema informativo, compresi l'hardware, il software, il firmware e le informazioni elaborate, memorizzate e trasmesse.

1.1.1 La Triade CIA

I tre obiettivi chiave al centro della sicurezza informatica, noti anche come triade CIA, sono:

Confidentiality (Confidenzialità) Ossia preservare le autorizzazioni riguardo l'accesso e la divulgazione delle informazioni. Comprende:

- **Confidenzialità dei dati:** assicura che le informazioni confidenziali non siano rese disponibili a individui non autorizzati.
- **Privacy:** assicura che gli individui controllino quali informazioni che li riguardano possano essere raccolte e conservate e da chi e a chi tali informazioni possano essere divulgate.

Integrity (Integrità) Ossia la protezione contro la modifica impropria delle informazioni. Racchiude:

- **Integrità dei dati:** assicura che le informazioni e i programmi siano modificati solo in un modo specificato e autorizzato.
- **Integrità del sistema:** assicura che un sistema esegua il suo compito prestabilito in modo ineccepibile.

Availability (Disponibilità) Assicura che il servizio non sia negato agli utenti autorizzati.

1.1.2 Livelli di Impatto delle Violazioni

La sicurezza informatica definisce tre livelli di impatto per valutare le conseguenze potenziali di una violazione:

Basso La perdita potrebbe avere un effetto limitato (es. efficacia ridotta).

Moderato La perdita potrebbe avere un serio effetto negativo (es. operatività svolta con grandi difficoltà).

Alto La perdita potrebbe avere un effetto negativo grave o catastrofico (es. impossibilità di svolgere funzioni primarie).

1.2 Le Sfide della Sicurezza Informatica

La sicurezza informatica presenta complessità intrinseche che la rendono una disciplina difficile da gestire:

1. **Mentalità avversaria**: Bisogna sempre anticipare i potenziali attacchi contro ogni meccanismo di difesa.
2. **Procedure controintuitive**: Spesso i meccanismi non sono ovvi e la loro necessità sfugge ai non addetti ai lavori.
3. **Gestione delle chiavi**: I sistemi si basano su segreti, sollevando problemi di creazione, distribuzione e protezione.
4. **Lotta continua**: È uno scontro perenne tra l'attaccante che cerca falle e il difensore che cerca di chiuderle.
5. **Sicurezza vs Usabilità**: Spesso la sicurezza è percepita come un ostacolo all'efficienza.
6. **Progettazione built-in**: Necessità di integrare la sicurezza fin dalle fasi iniziali della progettazione (anziché aggiungerla a posteriori come *bolt-on*).

1.3 Terminologia della Sicurezza

Le vulnerabilità rappresentano delle minacce per il sistema e un potenziale danno per quest'ultimo. Una minaccia che si concretizza rappresenta un attacco. Di seguito un riepilogo rapido e le definizioni dettagliate:

Termine	**Definizione**
Asset	La risorsa che si vuole difendere, che sia essa un sistema o dei dati.
Vulnerabilità	Qualsiasi evento con il potenziale di avere un impatto negativo sulle attività organizzative.
Minaccia	Evento con il potenziale di causare un danno.
Attacco	Qualsiasi tipo di attività malevola che tenta di raccogliere, interrompere o distruggere le risorse del sistema informativo o le informazioni stesse.
Contromisura	Azione o tecnica che riduce minacce o vulnerabilità.
Rischio	Probabilità che una minaccia sfrutti una vulnerabilità $\times$ Impatto.

Tabella 1.1: Terminologia di base della sicurezza.

1.3.1 Modelli di Controllo degli Accessi

Il controllo degli accessi definisce chi può accedere a cosa.

- **RBAC (Role-Based Access Control)**: Il controllo è basato sui *ruoli* (es. admin, user) a cui gli utenti vengono assegnati. Semplifica enormemente la gestione in grandi organizzazioni.
- **ABAC (Attribute-Based Access Control)**: Modello più granulare e dinamico, basato su *attributi* del soggetto, dell'oggetto e dell'ambiente (es. orario, posizione geografica).

1.4 Tipologie di Attacco e di Minacce

1.4.1 Classificazione degli Attacchi

Gli attacchi possono essere classificati secondo diverse dimensioni:

Attacco passivo Tentativo di apprendere informazioni del sistema senza alterarne il funzionamento (es. intercettazione). L'obiettivo principale è la **prevenzione** (tramite cifratura).

Attacco attivo Tentativo di alterare le risorse del sistema o influenzarne il funzionamento. L'obiettivo principale è la **rilevazione** e la risposta.

Attacco interno Effettuato da un *insider*, utente autorizzato ad accedere alle risorse ma utilizzate in modo non autorizzato.

Attacco esterno Effettuato da un *outsider*, ossia un utente non autorizzato del sistema.

1.4.2 Le Minacce

Divulgazione Non Autorizzata

La divulgazione non autorizzata (unauthorized disclosure) è una minaccia alla confidenzialità. I seguenti tipi di attacchi possono portare a questa conseguenza:

Esposizione (exposure) Può essere intenzionale o risultato di un errore umano/software, che si concretizza nella conoscenza non autorizzata di dati sensibili.

Intercettazione (interception) Attacco comune nelle comunicazioni: ricezione di una copia di informazioni destinate a un altro dispositivo.

Inferenza (inference) Nota come analisi del traffico; un avversario ottiene informazioni dall'osservazione del pattern del traffico su una rete o dalla struttura di un database.

Intrusione (intrusion) Ottenimento di un accesso non autorizzato a dati sensibili superando le protezioni del sistema.

Inganno

L'inganno (deception) è una minaccia all'integrità del sistema o dei dati:

Mascheramento (masquerade) Tentativo di un utente non autorizzato di accedere fingendosi un utente autorizzato.

Falsificazione (falsification) Alterazione o sostituzione di dati validi con dati falsi.

Ripudio (repudiation) Un utente rinnega falsamente di aver inviato, ricevuto o posseduto determinati dati.

Interruzione

L'interruzione (disruption) è una minaccia alla disponibilità o all'integrità del sistema:

Interdizione (incapacitation) Attacco alla disponibilità (es. danneggiamento hardware o disattivazione servizi tramite malware).

Corruzione (corruption) Attacco all'integrità in cui le risorse o i servizi funzionano in modo non voluto.

Ostruzione (obstruction) Interferisce con le comunicazioni disabilitando collegamenti o alterando le informazioni di controllo.

Usurpazione

L'usurpazione (usurpation) è una minaccia all'integrità del sistema:

Appropriazione indebita (misappropriation) Sottrazione di servizio, come l'uso di un host come piattaforma per lanciare attacchi DDoS (Distributed Denial of Service) verso altri bersagli.

Uso improprio (misuse) Disabilitazione o ostacolo delle funzioni di sicurezza tramite programmi malevoli o abusi di privilegi.

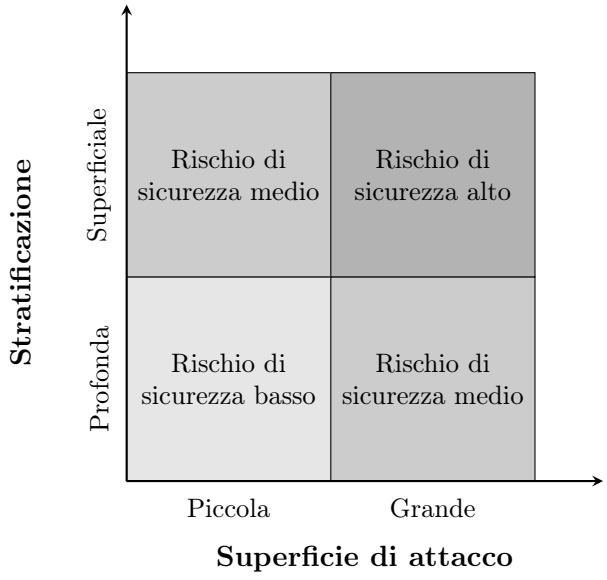
1.5 Valutazione delle Minacce

1.5.1 Superfici di Attacco

Una superficie di attacco è l'insieme di tutte le vulnerabilità raggiungibili e sfruttabili all'interno di un sistema. Si dividono in:

- **Superficie di Rete**: Porte aperte, protocolli di rete, interfacce web.
- **Superficie Software**: Vulnerabilità nel codice di applicazioni o sistemi operativi.
- **Superficie Umana**: Vulnerabilità legate alle persone (ingegneria sociale, errori operativi).

L'**analisi della superficie di attacco** è una tecnica cruciale per valutare le minacce. L'obiettivo è ridurla il più possibile per rendere più difficile il compito dell'avversario.



1.6 Minacce alle Risorse del Sistema

Le minacce possono colpire diverse categorie di risorse:

1.6.1 Hardware

Disponibilità L'attrezzatura viene rubata o disabilitata, impedendo così l'erogazione del servizio.

Confidenzialità Viene rubata una chiavetta USB non cifrata.

Integrità Dispositivi hardware vengono modificati o danneggiati.

1.6.2 Software

Disponibilità Vengono cancellati dei programmi, negando l'accesso agli utenti.

Confidenzialità Viene fatta una copia non autorizzata del software.

Integrità Un programma funzionante viene modificato per farlo fallire durante l'esecuzione o fargli eseguire operazioni non desiderate.

1.6.3 Dati

Disponibilità Vengono cancellati i file, negando l'accesso agli utenti.

Confidenzialità Lettura non autorizzata di dati o analisi statistica che rivela informazioni sottese.

Integrità I file esistenti vengono modificati o vengono creati nuovi file illeciti.

2 - Crittografia e Funzioni Hash

2.1 Cifratura Simmetrica

La **cifratura simmetrica**, nota anche come **cifratura convenzionale** o **a chiave singola**, è stata l'unica tipologia di crittografia utilizzata fino all'introduzione della crittografia a chiave pubblica alla fine degli anni '70.

2.1.1 I Cinque Componenti di uno Schema Simmetrico

Un sistema di cifratura simmetrica si basa su cinque elementi fondamentali che interagiscono per proteggere l'informazione:

Testo in Chiaro (Plaintext): Il messaggio o i dati originali che vengono forniti come input all'algoritmo.

Algoritmo di Cifratura: L'algoritmo che esegue le sostituzioni e le trasformazioni sul testo in chiaro per renderlo incomprensibile.

Chiave Segreta (Secret Key): Un valore segreto, fornito anch'esso come input all'algoritmo. Le trasformazioni esatte eseguite dipendono da questa chiave.

Testo Cifrato (Ciphertext): Il messaggio "mescolato" prodotto come output. Il suo valore dipende sia dal testo in chiaro sia dalla chiave segreta.

Algoritmo di Decifratura: È essenzialmente l'algoritmo di cifratura eseguito in senso inverso. Prende in input il testo cifrato e la stessa chiave segreta per riprodurre il testo in chiaro originale.

2.1.2 Requisiti per un Utilizzo Sicuro

Per garantire che il sistema sia effettivamente impenetrabile, sono necessari due requisiti fondamentali:

1. **Un algoritmo di cifratura robusto:** L'algoritmo deve essere tale che un avversario, anche conoscendo l'algoritmo e avendo accesso a testi cifrati (e persino a coppie plaintext-ciphertext), non possa decifrare i messaggi o risalire alla chiave.

2. **Gestione sicura della chiave:** Mittente e destinatario devono ottenere la chiave in modo sicuro e mantenerla segreta. Se la chiave è compromessa, tutte le comunicazioni sono a rischio.

2.1.3 Metodi di Attacco alla Cifratura Simmetrica

- **Criptoanalisi (Cryptanalysis):** Sfrutta la natura dell'algoritmo e le caratteristiche del testo per dedurre il messaggio o la chiave. Il successo nel dedurre la chiave è considerato catastrofico per l'intero sistema.

- **Attacco a Forza Bruta (Brute-force Attack):** Consiste nel provare sistematicamente ogni possibile chiave. Richiede un metodo per riconoscere il testo in chiaro corretto una volta decifrato, operazione non banale se il testo è compresso o non è in linguaggio naturale.

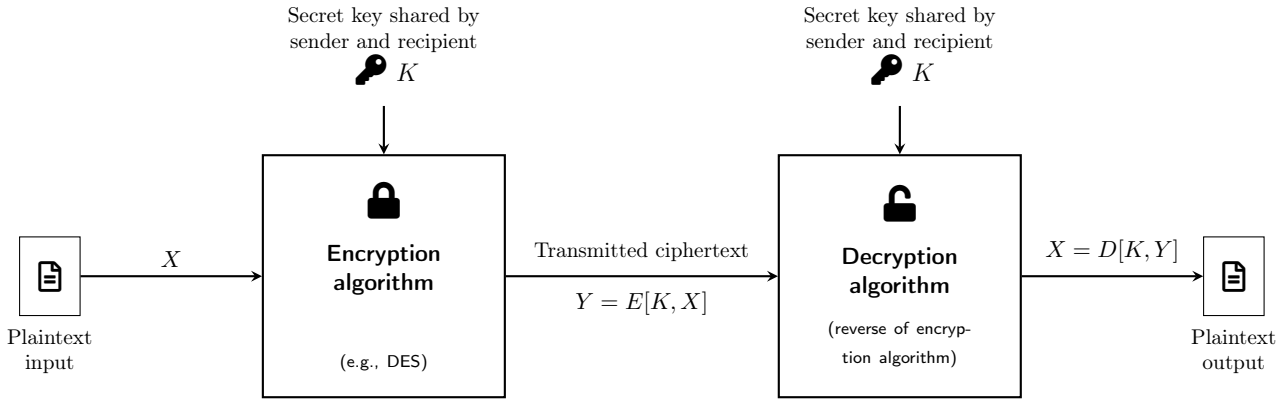
2.2 Funzioni Hash e Digest

Una **funzione hash** è un algoritmo matematico che prende un input di dimensione arbitraria e restituisce una stringa di bit di lunghezza fissa, chiamata **digest** o valore di hash.

2.2.1 Differenza Fondamentale: Hash vs Crittografia

La distinzione cruciale tra i due concetti risiede principalmente nella **reversibilità** e nello scopo finale:

- **Crittografia:** È un processo a due vie, quindi **reversibile**. Il suo scopo è la **confidenzialità**. Un messaggio cifrato è progettato per essere decifrato e riportato al suo stato originale da chi possiede la chiave corretta.
- **Hashing:** È un processo a una via, quindi **irreversibile**. È computazionalmente impossibile ricostruire l'input originale a partire dal suo digest. Il suo scopo primario è garantire l'**integrità** dei dati. Se si calcola l'hash di un file prima e dopo una trasmissione e i due digest corrispondono, si ha la certezza che il file non è stato alterato.



2.3 Analisi degli Algoritmi: DES, Triple DES e AES

La cifratura simmetrica moderna si è evoluta attraverso tre standard principali adottati dal NIST.

2.3.1 DES (Data Encryption Standard)

Adottato nel 1977, è stato per decenni il pilastro della sicurezza federale USA.

- **Funzionamento:** È un **cifrario a blocchi** che opera su unità di **64 bit**. Utilizza la **Rete di Feistel**, applicando sostituzioni e permutazioni per 16 round.
- **Chiave:** 56 bit effettivi (8 bit usati per la parità).
- **Sicurezza attuale:** La dimensione ridotta della chiave lo rende oggi vulnerabile ad attacchi a forza bruta eseguiti con hardware moderno.

2.3.2 Triple DES (3DES)

Evoluzione necessaria per contrastare l'aumento della potenza di calcolo.

- **Meccanismo:** Applica l'algoritmo DES tre volte in sequenza (**EDE - Encrypt-Decrypt-Encrypt**) con due o tre chiavi distinte.
- **Vantaggi e Svantaggi:** Porta la chiave a 168 bit rendendo la forza bruta impraticabile, ma è molto **lento** e pesante computazionalmente.

2.3.3 AES (Advanced Encryption Standard)

Selezionato nel 2001, è l'attuale standard globale.

- **Struttura:** Non usa Feistel ma una **rete a sostituzione e permutazione (SPN)**. Opera su blocchi di **128 bit**.
- **Chiavi:** Supporta lunghezze di **128, 192 e 256 bit**.
- **Efficienza:** Estremamente sicuro e performante sia in hardware che in software.

2.4 Modalità Operative e Cifrari

2.4.1 Il Problema della Modalità ECB (Electronic Codebook)

Nella cifratura di grandi quantità di dati, la modalità ECB è l'approccio più semplice: ogni blocco è cifrato indipendentemente. Tuttavia, questa semplicità causa una grave debolezza: blocchi di plaintext identici producono blocchi di ciphertext identici, permettendo di **sfruttare le regolarità** (es. i contorni di un'immagine rimangono visibili). Per ovviare a ciò si usano modi operativi che concatenano i blocchi.

2.4.2 Cifrari a Blocchi vs Cifrari a Flusso

Caratteristica	Cifrario a Blocchi	Cifrario a Flusso
Elaborazione	Un blocco alla volta (es. 128 bit).	Continuo (tipicamente un byte alla volta).
Meccanismo	Trasforma blocchi interi.	XOR tra plaintext e *keystream* pseudocasuale.
Velocità	Standard.	Quasi sempre più veloci.
Esempi	AES, DES.	RC4 (storico), ChaCha20.

2.5 Autenticazione dei Messaggi

Mentre la crittografia protegge dagli attacchi passivi, l'**autenticazione** protegge dagli **attacchi attivi** (falsificazione). Un messaggio è autentico se è genuino, proviene dalla fonte dichiarata e non è stato alterato.

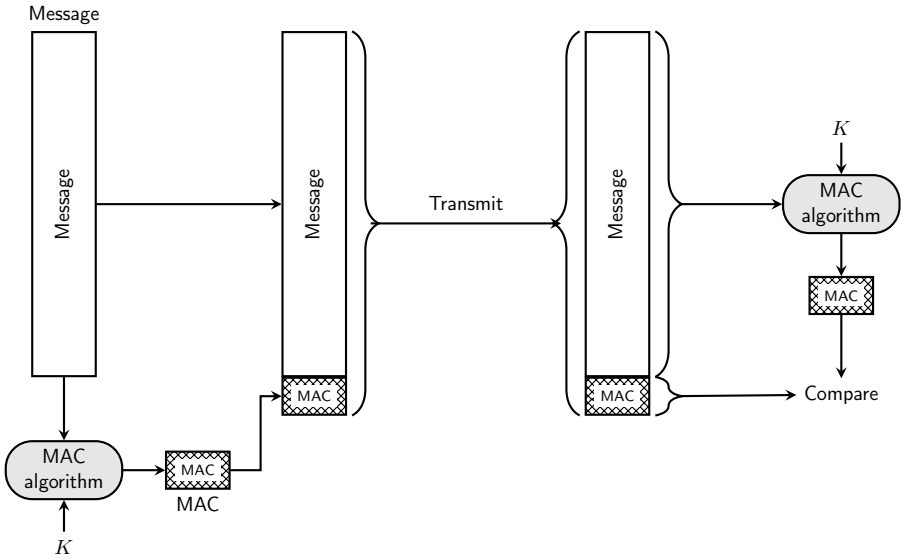
2.5.1 Il Limite della Sola Cifratura

Cifrare con chiave simmetrica non garantisce automaticamente l'integrità. Un attaccante potrebbe riordinare i blocchi cifrati (attacco alla modalità ECB). Il destinatario decifrerebbe i blocchi ma otterrebbe un messaggio dal significato alterato senza accorgersene.

2.5.2 Autenticazione Tramite Tag (MAC)

Si separa l'autenticazione dalla cifratura aggiungendo un **Message Authentication Code (MAC)**:

- Il mittente calcola il tag usando il messaggio e una chiave segreta.
- Il destinatario ricalcola il tag e lo confronta.
- Garantisce autenticità senza necessità di confidenzialità (messaggio in chiaro).



2.6 Requisiti delle Funzioni Hash One-Way

Per essere crittograficamente sicura, una funzione hash H deve soddisfare:

1. **Input di Dimensione Variabile:** Applicabile a dati di qualsiasi dimensione.
2. **Output di Lunghezza Fissa:** Digest sempre della stessa lunghezza.
3. **Facilità di Calcolo:** Calcolo veloce di $H(x)$.
4. **Resistenza alla Preimmagine:** Dato h, impossibile trovare x tale che $H(x) = h$.
5. **Resistenza alla Seconda Preimmagine:** Dato x, impossibile trovare $y \neq x$ con lo stesso hash.
6. **Resistenza alle Collisioni:** Impossibile trovare *qualsiasi* coppia (x, y) con $H(x) = H(y)$.

2.7 Crittografia a Chiave Pubblica (Asimmetrica)

La **crittografia a chiave pubblica**, proposta pubblicamente per la prima volta da Diffie e Hellman nel 1976, rappresenta un'avanzata rivoluzionaria nel campo della crittografia. A differenza degli algoritmi simmetrici che operano su bit, quelli a chiave pubblica si basano su **funzioni matematiche complesse**.

La caratteristica fondamentale è la sua natura **asimmetrica**: utilizza una **coppia di chiavi** separate (una pubblica e una privata), a differenza della cifratura simmetrica che ne usa solo una. L'uso di due chiavi ha profonde conseguenze in termini di confidenzialità, distribuzione delle chiavi e autenticazione.

2.7.1 Miti Comuni sulla Crittografia a Chiave Pubblica

Esistono alcune idee **errate** comuni riguardo a questa tecnologia che è bene chiarire:

1. **"È più sicura della crittografia simmetrica":** Falso. La sicurezza di qualsiasi schema crittografico dipende dalla **lunghezza della chiave** e dal **lavoro computazionale** richiesto per romperla. In linea di principio, nessuno dei due approcci è intrinsecamente superiore all'altro in termini di resistenza alla criptoanalisi.
2. **"Ha reso obsoleta la crittografia simmetrica":** Falso. A causa dell'elevato **carico computazionale** (overhead) degli algoritmi a chiave pubblica, questi sono significativamente più lenti. Non c'è alcuna previsione che la cifratura simmetrica venga abbandonata; anzi, i due sistemi vengono spesso usati insieme per combinare efficienza e sicurezza.
3. **"La distribuzione delle chiavi è banale":** Falso. Anche la distribuzione delle chiavi pubbliche richiede protocolli specifici, spesso con un agente centrale (come una Certificate Authority), e le procedure non sono né più semplici né più efficienti di quelle usate per la crittografia simmetrica.

2.7.2 I Sei Componenti di uno Schema a Chiave Pubblica

Uno schema di crittografia a chiave pubblica si articola in sei elementi fondamentali:

- **Testo in Chiaro (Plaintext):** Il messaggio leggibile fornito come input.
- **Algoritmo di Cifratura:** L'algoritmo che esegue le trasformazioni sul testo in chiaro.
- **Chiave Pubblica e Privata:** Una coppia di chiavi correlate matematicamente. Se una viene usata per cifrare, l'altra deve essere usata per decifrare. La **chiave pubblica** è resa nota a tutti, mentre la **chiave privata** è conosciuta solo dal proprietario.
- **Testo Cifrato (Ciphertext):** Il messaggio cifrato prodotto come output, il cui valore dipende dal plaintext e dalla chiave usata.
- **Algoritmo di Decifratura:** L'algoritmo che, ricevendo il testo cifrato e la chiave corrispondente, produce il testo in chiaro originale.

2.7.3 Il Processo di Comunicazione Sicura

Il vantaggio principale di questo approccio è che **le chiavi private non devono mai essere distribuite**. Il processo si articola in quattro passaggi:

1. **Generazione:** Ogni utente genera la propria coppia di chiavi.
2. **Pubblicazione:** Ogni utente inserisce la propria chiave pubblica in un registro accessibile a tutti (directory pubblica), mantenendo la chiave privata assolutamente segreta.
3. **Cifratura:** Se Bob vuole inviare un messaggio ad Alice, cerca la **chiave pubblica di Alice** e la usa per cifrare il messaggio.
4. **Decifratura:** Quando Alice riceve il messaggio, usa la sua **chiave privata** per decifrarlo. Solo lei può farlo poiché solo lei possiede la chiave privata corrispondente.

2.7.4 Requisiti per la Crittografia a Chiave Pubblica

1. **Facilità di Generazione:** Deve essere computazionalmente semplice generare la coppia di chiavi (PU_b, PR_b).
2. **Facilità di Cifratura:** Deve essere semplice per un mittente, conoscendo PU_b e il messaggio M, generare il testo cifrato $C = E(PU_b, M)$.
3. **Facilità di Decifratura:** Deve essere semplice per il destinatario decifrare C usando la propria chiave privata PR_b: $M = D(PR_b, C)$.
4. **Sicurezza della Chiave Privata:** Deve essere **computazionalmente impossibile** risalire a PR_b pur conoscendo la chiave pubblica PU_b.
5. **Sicurezza del Messaggio:** Deve essere **computazionalmente impossibile** per un avversario recuperare il messaggio originale M conoscendo solo PU_b e il testo cifrato C.

2.7.5 Panoramica degli Algoritmi Asimmetrici

- **RSA (Rivest-Shamir-Adleman):** Sviluppato nel 1977, è l'algoritmo più diffuso. Opera su numeri interi e la sua sicurezza risiede nella difficoltà di fattorizzare numeri molto grandi. Si usa per **cifratura** e **firme digitali**.
- **Diffie-Hellman Key Agreement:** Il primo pubblicato. Serve esclusivamente a stabilire in modo sicuro una **chiave segreta condivisa** su un canale insicuro, per poi usarla in crittografia simmetrica. Non cifra messaggi direttamente.
- **DSS (Digital Signature Standard):** Basato sull'algoritmo DSA, è progettato **esclusivamente per la firma digitale**, senza la cifratura di dati.
- **ECC (Elliptic Curve Cryptography):** Concorrente moderno di RSA. Offre sicurezza equivalente con **chiavi molto più piccole**, riducendo il carico computazionale (ottimo per dispositivi mobili e IoT).

2.8 Firme Digitali

La firma digitale è una trasformazione crittografica che fornisce:

- **Autenticità dell'Origine:** Verifica l'identità del firmatario.
- **Integrità dei Dati:** Assicura che i dati non siano stati alterati dopo la firma.
- **Non Ripudio:** Impedisce al firmatario di negare la paternità dell'atto.

2.8.1 Il Processo di Firma e Verifica

Supponendo che Bob firmi un messaggio per Alice:

Fase 1: Firma del messaggio (Bob)

1. **Creazione del Digest:** Bob calcola l'hash (*digest*) del messaggio M usando una funzione sicura (es. SHA-512).
2. **Cifratura del Digest:** Bob cifra il digest con la propria **chiave privata**. Il risultato è la **firma digitale** S.
3. **Invio:** Bob invia il messaggio in chiaro M e la firma S.

Fase 2: Verifica della firma (Alice)

1. **Calcolo Locale:** Alice ricalcola l'hash del messaggio M ricevuto.
2. **Decifratura della Firma:** Alice usa la **chiave pubblica di Bob** per decifrare S e ottenere il digest originale.
3. **Confronto:** Se i due digest coincidono, la firma è valida: il messaggio è autentico e integro.

Nota bene: La firma digitale **non fornisce confidenzialità**, poiché il messaggio viaggia in chiaro.

2.9 Certificati a Chiave Pubblica

Per risolvere il problema della possibile falsificazione delle chiavi pubbliche (attaccante che si finge qualcun altro), si ricorre ai **certificati**.

2.9.1 Struttura e Ruolo della CA

Un certificato lega un'identità a una chiave pubblica. È composto dalla chiave pubblica dell'utente, l'ID utente (nome, email) e il periodo di validità, il tutto **firmato digitalmente** da una **Certificate Authority (CA)** fidata.

2.9.2 Processo di Creazione e Verifica

1. **Richiesta:** L'utente presenta un certificato non firmato alla CA per la verifica dell'identità.
2. **Emissione:** La CA calcola l'hash del certificato e lo cifra con la **propria chiave privata**, creando la firma del certificato.
3. **Verifica:** Chiunque riceva il certificato usa la **chiave pubblica della CA** per decifrare la firma e verificare che l'hash corrisponda. Se il controllo passa, si ha la certezza che la chiave pubblica contenuta appartenga realmente all'utente dichiarato.

Lo standard universale per questi documenti è **X.509**, utilizzato in TLS, SSH e IPsec.

2.10 Numeri Casuali e Dati Memorizzati

2.10.1 L'Uso dei Numeri Casuali

Fondamentali per la generazione di chiavi RSA, chiavi di sessione e protocolli di handshaking. I requisiti sono:

- **Casualità (Randomness):** Distribuzione uniforme e indipendenza statistica.
- **Imprevedibilità:** Impossibilità di prevedere valori futuri conoscendo i precedenti. In crittografia si usano i **CSPRNG** (Generatori Pseudocasuali Crittograficamente Sicuri).

2.10.2 Cifratura dei Dati Memorizzati (Data at Rest)

La protezione dei dati su disco è spesso trascurata rispetto ai dati in transito. I rischi includono l'aggiramento dei controlli del sistema operativo e il recupero di dati "cancellati" ma ancora fisicamente presenti sui settori del disco.

Le strategie di protezione includono: software commerciali per singoli file o cartelle, appliance hardware di back-end tra server e storage, cifratura in background per l'intero disco (**BitLocker**, **FileVault**).

3 - Crittografia

3.1 Crittografia Simmetrica

3.1.1 Introduzione

La crittografia simmetrica (o crittografia a chiave singola), era l'unico tipo di crittografia in uso prima dell'introduzione della crittografia a chiave pubblica.

La crittografia simmetrica è composta da 5 elementi:

1. **Testo in chiaro:** i dati originali che vengono immessi nell'algoritmo come input.
2. **Algoritmo di crittografia:** esegue varie trasformazioni sul testo in chiaro.
3. **Chiave segreta:** le sostituzioni e le trasformazioni esatte eseguite dall'algoritmo dipendono dalla chiave.
4. **Testo cifrato:** l'output dell'algoritmo di crittografia. Per un dato messaggio, due chiavi diverse produrranno due testi cifrati diversi.
5. **Algoritmo di decifrazione:** l'algoritmo di crittografia eseguito al contrario. Prende il testo cifrato e la chiave segreta e produce il testo in chiaro originale.

3.1.2 Requisiti

Abbiamo bisogno di un algoritmo di crittografia forte, in cui l'avversario non dovrebbe essere in grado di decifrare il testo cifrato o scoprire la chiave anche se è in possesso dei testi cifrati e i relativi testi in chiaro. Inoltre, il mittente e il destinatario devono aver ottenuto copie della chiave segreta in modo sicuro e devono conservare la chiave in modo sicuro. Se qualcuno riesce a scoprire la chiave e conosce l'algoritmo, tutte le comunicazioni che utilizzano questa chiave sono leggibili.

3.1.3 Attacco alla Crittografia Simmetrica

Esistono due approcci generali per attaccare uno schema di crittografia simmetrica:

Crittoanalisi Gli attacchi crittoanalitici si basano sulla natura dell'algoritmo, tentando di dedurre un testo in chiaro specifico o la chiave utilizzata.

Attacco brute-force Consiste nel provare ogni possibile chiave fino a ottenere una traduzione intelligibile in testo in chiaro. In media, è necessario provare metà di tutte le possibili chiavi per ottenere il successo.

3.1.4 Algoritmi a Crittografia Simmetrica

Gli algoritmi di crittografia simmetrica più comunemente utilizzati sono i cifrari a blocchi, che elaborano il testo in chiaro in blocchi di dimensioni fisse e producono un blocco di testo cifrato di dimensioni uguali per ogni blocco di testo in chiaro. L'algoritmo elabora quantità di testo in chiaro più lunghe come una serie di blocchi

di dimensioni fisse. Gli algoritmi simmetrici più importanti, tutti cifrari a blocchi, sono il Data Encryption Standard (DES), il triplo DES e l'Advanced Encryption Standard (AES).

DES (Data Encryption Standard)

Il DES (Data Encryption Standard) è un algoritmo di crittografia simmetrica che utilizza una chiave di 56 bit e opera su blocchi di dati di 64 bit, applicando 16 cicli di permutazioni, sostituzioni e trasposizioni per cifrare i dati. Con il tempo, la sua sicurezza è stata messa in discussione a causa della crescita della potenza di calcolo: la chiave di 56 bit è ormai vulnerabile a brute force, rendendo DES obsoleto per applicazioni moderne.

3DES (Triple DES)

Il 3DES (Triple Data Encryption Standard) è una variante del DES sviluppata per rafforzare la sicurezza senza progettare un algoritmo completamente nuovo. Consiste nell'applicare il DES tre volte su ciascun blocco di dati: il primo passaggio cifra i dati con una chiave, il secondo li decifra con una seconda chiave, e il terzo li ricifra con una chiave finale (anche se spesso si usano due chiavi identiche). Questo approccio porta la lunghezza effettiva della chiave a 112 o 168 bit, rendendolo significativamente più sicuro rispetto a DES. Tuttavia, 3DES è più lento rispetto ad algoritmi moderni e, nel tempo, è stato dichiarato deprecato a causa di vulnerabilità teoriche e della crescente efficienza di metodi crittanalitici.

AES (Advanced Encryption Standard)

L'AES (Advanced Encryption Standard) è l'attuale standard di crittografia simmetrica adottato per sostituire DES e 3DES. Supporta chiavi di 128, 192 e 256 bit, offrendo un livello di sicurezza molto più alto rispetto ai suoi predecessori. Opera su blocchi di 128 bit utilizzando un numero variabile di cicli di trasformazioni (10, 12 o 14 a seconda della lunghezza della chiave) per crittografare i dati. Grazie alla sua efficienza e sicurezza, AES è largamente utilizzato in applicazioni moderne.

3.2 Autenticazione dei Messaggi

La crittografia protegge dagli attacchi passivi, come le intercettazioni. Un requisito diverso è la protezione dagli attacchi attivi, quali falsificazione di dati. La protezione contro tali attacchi è nota come autenticazione dei messaggi o dei dati.

Un messaggio si dice autentico quando è genuino e proviene dalla sua presunta fonte. Gli aspetti importanti sono verificare che il contenuto del messaggio non sia stato alterato e che la fonte sia autentica.

3.2.1 Autenticazione Senza Cifratura del Messaggio

Esaminiamo diversi approcci all'autenticazione dei messaggi. In tutti questi approcci, un tag di autenticazione viene generato e aggiunto a ciascun messaggio per la trasmissione. Il messaggio stesso non è crittografato e può essere letto da chiunque, pertanto la riservatezza del messaggio non viene fornita.

3.2.2 Message Authentication Code

Una tecnica di autenticazione prevede l'uso di una chiave segreta per generare un piccolo blocco di dati, noto come codice di autenticazione del messaggio, che viene aggiunto al messaggio. Questa tecnica presuppone che due parti comunicanti, ad esempio A e B, condividano una chiave segreta comune K_{AB}.

Il processo funziona come segue:

1. Quando A ha un messaggio da inviare a B, calcola il codice di autenticazione del messaggio.
2. Il messaggio più il codice vengono trasmessi al destinatario previsto.
3. Il destinatario esegue lo stesso calcolo sul messaggio ricevuto, utilizzando la stessa chiave segreta, per generare un nuovo codice di autenticazione del messaggio.
4. Il codice ricevuto viene confrontato con il codice calcolato.

3.2.3 Funzione Hash Unidirezionale

Un'alternativa al codice di autenticazione del messaggio è la funzione hash unidirezionale. Come per il codice di autenticazione del messaggio, una funzione hash accetta un messaggio di dimensione variabile M come input e produce un digest del messaggio di dimensione fissa $H(M)$ come output. In genere, il messaggio viene riempito fino a un multiplo intero di una lunghezza fissa (ad esempio, 1024 bit) e il riempimento include il valore della lunghezza del messaggio originale in bit. Il campo lunghezza è una misura di sicurezza per aumentare la difficoltà per un aggressore di produrre un messaggio alternativo con lo stesso valore hash.

3.2.4 Proprietà della Funzione Hash

Lo scopo di una funzione hash è di produrre un'impronta digitale di un file, messaggio o altro blocco di dati. Per essere utile per l'autenticazione dei messaggi, una funzione hash H deve avere le seguenti proprietà:

1. H può essere applicata a un blocco di dati di qualsiasi dimensione.
2. H produce un output di lunghezza fissa.
3. $H(x)$ è relativamente facile da calcolare per qualsiasi x dato, rendendo pratiche sia le implementazioni hardware che software.
4. Deve essere resistente alle collisioni deboli: si riferisce alla difficoltà di trovare un input che produce un valore di hash specifico, dato un valore di hash già noto.
5. Deve essere resistente alle collisioni forti: si riferisce alla difficoltà di trovare due input distinti x e y che producono lo stesso valore di hash.

3.2.5 Integrità dei Dati

Oltre a fornire l'autenticazione, un digest del messaggio fornisce anche l'integrità dei dati. Svolge la stessa funzione di una sequenza di controllo dei frame: se alcuni bit nel messaggio vengono accidentalmente alterati durante il transito, il digest del messaggio sarà in errore.

3.2.6 Garantire la Riservatezza

È possibile combinare autenticazione e riservatezza in un singolo algoritmo crittografando un messaggio più il suo tag di autenticazione. L'autenticazione dei messaggi viene fornita come funzione separata dalla crittografia dei messaggi.

3.3 Crittografia Asimmetrica

3.3.1 Introduzione

La crittografia a chiave pubblica, proposta per la prima volta pubblicamente da Diffie e Hellman nel 1976, è il primo progresso veramente rivoluzionario nella crittografia in letteralmente migliaia di anni. La crittografia a chiave pubblica è asimmetrica, comportando l'uso di due chiavi separate, a differenza della crittografia simmetrica, che utilizza solo una chiave.

3.3.2 Componenti di uno Schema a Chiave Pubblica

Uno schema di crittografia a chiave pubblica ha sei componenti:

1. **Testo in chiaro:** il messaggio leggibile che viene immesso nell'algoritmo come input.

2. **Algoritmo di crittografia:** l'algoritmo di crittografia esegue varie trasformazioni sul testo in chiaro.

3. **Chiave pubblica e privata:** si tratta di una coppia di chiavi selezionate in modo che se una viene utilizzata per la cifratura, l'altra viene utilizzata per la decifratura. Le trasformazioni esatte eseguite dall'algoritmo di crittografia dipendono dalla chiave pubblica o privata fornita come input.

4. **Testo cifrato:** si tratta del messaggio criptato prodotto come output. Dipende dal testo in chiaro e dalla chiave. Per un dato messaggio, due chiavi diverse produrranno due testi cifrati diversi.

5. **Algoritmo di decifratura:** questo algoritmo accetta il testo cifrato e la chiave corrispondente e produce il testo in chiaro originale.

Come suggeriscono i nomi, la chiave pubblica della coppia viene resa pubblica affinché altri possano utilizzarla, mentre la chiave privata è nota solo al suo proprietario. Un algoritmo crittografico a chiave pubblica si basa su una chiave per la cifratura e su una chiave diversa ma correlata per la decifratura.

3.3.3 Funzionamento Base

Se Bob desidera inviare un messaggio privato ad Alice, Bob crittografa il messaggio utilizzando la chiave pubblica di Alice. Quando Alice riceve il messaggio, lo decifra utilizzando la sua chiave privata.

Nessun altro destinatario può decifrare il messaggio perché solo Alice conosce la sua chiave privata. In questo schema andiamo a garantire la riservatezza ma non l'autenticità del messaggio: solo il destinatario previsto dovrebbe essere in grado di decifrare il testo cifrato perché solo il destinatario previsto è in possesso della chiave privata richiesta.

3.3.4 Garantire l'Autenticità

In questo schema, un utente crittografa i dati utilizzando la propria chiave privata, garantendo l'autenticità ma non la riservatezza. In questo modo, chiunque disponga della chiave pubblica di Bob potrà decifrare il messaggio e leggerne il contenuto.

3.3.5 Busta Digitale

Un'altra applicazione in cui la crittografia a chiave pubblica viene utilizzata per proteggere una chiave simmetrica è la busta digitale, che può essere utilizzata per proteggere un messaggio senza dover prima predisporre che mittente e destinatario abbiano la stessa chiave segreta. La tecnica è definita busta digitale, che è l'equivalente di una busta sigillata contenente una lettera non firmata.

Supponiamo che Bob desideri inviare un messaggio riservato ad Alice, ma non condividono una chiave segreta simmetrica. Bob fa quanto segue:

1. Prepara un messaggio.
2. Genera una chiave simmetrica casuale che verrà utilizzata solo questa volta.
3. Crittografa quel messaggio usando la crittografia simmetrica della chiave monouso.
4. Crittografa la chiave monouso usando la crittografia a chiave pubblica con la chiave pubblica di Alice.
5. Allega la chiave monouso crittografata al messaggio crittografato e inviala ad Alice.

Solo Alice è in grado di decrittografare la chiave monouso e quindi di recuperare il messaggio originale. Se Bob ha ottenuto la chiave pubblica di Alice tramite il certificato di chiave pubblica di Alice, allora Bob è sicuro che si tratti di una chiave valida.

4 - Autenticazione Digitale degli Utenti

4.1 Introduzione

L'autenticazione digitale rappresenta il processo con cui si stabilisce la fiducia nelle identità presentate elettronicamente a un sistema informatico. Questa fiducia è fondamentale per consentire a un sistema di determinare se l'individuo autenticato è autorizzato a compiere specifiche operazioni, come accedere a dati sensibili o a risorse del sistema.

4.2 Modello di Autenticazione Digitale

La prima fase richiede la registrazione dell'utente, durante la quale un richiedente si rivolge a una Registration Authority (RA) per diventare sottoscrittore presso un Credential Service Provider (CSP). La RA, come entità fidata, garantisce l'identità del richiedente al CSP, il quale emette una credenziale elettronica collegata a un token, come una chiave crittografica o una password cifrata, che identifica univocamente il sottoscrittore.

L'autenticazione vera e propria avviene tra il sottoscrittore e un sistema verificatore, dove il richiedente dimostra il possesso e il controllo del token. Il verificatore, dopo aver accertato l'identità, trasmette un'asserzione alla relying party (RP), che utilizza tali informazioni per prendere decisioni di autorizzazione. Questo modello illustra le principali entità e funzioni essenziali per un sistema di autenticazione sicuro, sebbene nella pratica possano esistere varianti o complessità aggiuntive.

4.3 Mezzi di Autenticazione

L'identità di un utente può essere autenticata tramite quattro modalità principali:

1. **Qualcosa che l'utente conosce:** come password, PIN o risposte a domande specifiche.

2. **Qualcosa che l'utente possiede:** come una keycard elettronica, una smart card o una chiave fisica. Questi strumenti sono noti come token.

3. **Qualcosa che l'utente è:** metodi di biometria statica, come il riconoscimento delle impronte digitali, della retina o del volto.

4. **Qualcosa che l'utente fa:** metodi di biometria dinamica, come il riconoscimento della voce, delle caratteristiche della scrittura o del ritmo di battitura.

Ciascun metodo, se correttamente implementato, può fornire una protezione sicura, ma ognuno presenta anche vulnerabilità, come il rischio di furto o falsificazione per password e token, e la possibilità di falsi positivi o negativi per la biometria. Inoltre, la gestione delle password e dei token può risultare onerosa in termini di sovraccarico amministrativo.

4.4 Autenticazione Multifattore

L'autenticazione multifattore (MFA) utilizza più di uno dei metodi precedentemente descritti. La robustezza del sistema aumenta con l'integrazione di più fattori: un sistema a due fattori è più sicuro rispetto a uno a fattore singolo, mentre l'uso di tre fattori incrementa ulteriormente la sicurezza. L'MFA è considerata una soluzione efficace per migliorare la protezione complessiva delle identità digitali, riducendo le possibilità di accessi non autorizzati.

4.5 Rischi nell'Autenticazione degli Utenti

La valutazione dei rischi di sicurezza per l'autenticazione degli utenti considera tre elementi fondamentali:

- il livello di garanzia
- l'impatto potenziale
- le aree di rischio

Questi aspetti sono valutati per adattare le misure di sicurezza in funzione delle minacce specifiche.

4.5.1 Livello di Garanzia

Il livello di garanzia rappresenta il grado di affidabilità che un'organizzazione può avere nel fatto che un utente sia realmente colui al quale è stata rilasciata una determinata credenziale. È un parametro che misura la fiducia nel processo di verifica e nell'identità di chi utilizza la credenziale.

I livelli di garanzia sono:

Livello 1: Bassa fiducia, appropriato per situazioni con requisiti di sicurezza minimi, come la registrazione in un forum online.

Livello 2: Fiducia discreta, adatto a interazioni con il pubblico che richiedono una verifica preliminare dell'identità.

Livello 3: Alta fiducia, utilizzato per l'accesso a informazioni riservate di valore elevato, richiede autenticazione multifattoriale.

Livello 4: Massima fiducia, destinato all'accesso a risorse di valore molto elevato o ad alto rischio, tipicamente con autenticazione multifattoriale e verifica di presenza.

4.5.2 Impatto Potenziale

Il livello di impatto potenziale valuta le conseguenze di una compromissione dell'autenticazione. Viene suddiviso in tre livelli:

Basso: Implica effetti negativi limitati, con riduzione della funzionalità operativa e danni minori.

Moderato: Può causare una significativa riduzione dell'efficacia operativa e danni economici e personali rilevanti, senza tuttavia mettere in pericolo la vita.

Alto: Può comportare gravi danni, incluse perdite finanziarie significative, interruzione delle operazioni principali e lesioni potenzialmente letali per gli individui.

4.5.3 Aree di Rischio e Corrispondenza

L'analisi dei rischi richiede di determinare il livello di impatto in caso di errore nell'autenticazione per ciascuna categoria di rischio rilevante. Questa valutazione definisce il livello di garanzia necessario per mitigare adeguatamente il rischio associato.

Ad esempio, un errore di autenticazione che comporti accesso non autorizzato a un database può avere un impatto finanziario di:

Basso: Perdita finanziaria minima, con effetti limitati per l'organizzazione.

Moderato: Grave perdita finanziaria, con significative responsabilità per l'organizzazione.

Alto: Perdita finanziaria catastrofica, con gravi conseguenze per tutte le parti coinvolte.

Se l'impatto potenziale è basso, un livello di garanzia pari a 1 è sufficiente; con un impatto moderato, è necessario un livello di garanzia di 2 o 3; e per un impatto alto, è richiesto il livello 4. Un'analisi approfondita deve considerare l'impatto in ciascuna area di rischio, assicurando che il livello di garanzia scelto sia in linea con i requisiti di sicurezza e con le potenziali conseguenze di una violazione dell'autenticazione.

4.6 Autenticazione Basata su Password

L'autenticazione tramite password è una delle difese principali contro gli accessi non autorizzati.

4.6.1 Vulnerabilità delle Password e Contromisure

I sistemi basati su password sono soggetti a vari attacchi che richiedono specifiche contromisure per mitigare i rischi:

1. **Attacco offline basato su dizionario:** L'attaccante ottiene il file delle password e confronta gli hash memorizzati con quelli di password comuni. Le contromisure includono limitazioni sull'accesso al file delle password e l'uso di misure di rilevamento intrusioni.

2. **Attacco a un account specifico:** Si tenta di scoprire la password di un account specifico tramite tentativi ripetuti. Un meccanismo di blocco dopo un numero limitato di tentativi falliti è la principale contromisura.

3. **Attacco con password comuni:** L'attaccante usa password comuni per una vasta gamma di ID utente. Politiche restrittive sulle password e il monitoraggio degli IP aiutano a contrastare questo tipo di attacco.

4. **Indovinare la password di un singolo utente:** Conoscendo dettagli personali, l'attaccante prova a indovinare la password. Politiche di gestione della password che impongono complessità e segretezza rappresentano la contromisura.

5. **Appropriazione di una Workstation:** L'attaccante sfrutta una sessione lasciata incustodita. Il logout automatico in caso di inattività riduce questa vulnerabilità.

6. **Sfruttare errori dell'utente:** Tattiche di ingegneria sociale o condivisione non autorizzata di password sono comuni. Le contromisure includono educazione degli utenti e rilevamento delle intrusioni.

7. **Utilizzo di password multiple identiche:** La condivisione di password tra diversi sistemi facilita gli attacchi. Una politica che proibisca password identiche su dispositivi diversi è la migliore contromisura.

8. **Monitoraggio elettronico:** Le password inviate su rete possono essere intercettate. L'uso di crittografia per il canale di comunicazione e la gestione del riutilizzo delle password sono essenziali.

4.6.2 Persistenza delle Password

Nonostante le vulnerabilità, le password continuano a essere ampiamente utilizzate per ragioni di costo, convenienza e compatibilità:

- Le soluzioni hardware, come scanner biometrici, richiedono l'implementazione di software appropriato su client e server, con una reciproca accettazione non ancora diffusa.

- I token fisici, come le smart card, possono risultare scomodi e costosi da utilizzare.

- Gli schemi di single sign-on introducono un singolo punto di rischio per la sicurezza.

- I gestori automatici di password non sono ancora abbastanza avanzati per supportare il roaming tra piattaforme multiple, riducendo la loro praticità.

Data l'importanza e la persistenza delle password, è essenziale comprenderne l'uso e studiarne le tecniche di gestione per garantire la sicurezza degli utenti.

4.7 Hash delle Password

L'uso delle password hashate con l'aggiunta di un salt (sale) rappresenta una tecnica efficace per aumentare la sicurezza delle password memorizzate nei sistemi informatici.

4.7.1 Hashing con Salt

Questo metodo è basato sull'applicazione di una funzione hash che genera un codice a partire dalla password dell'utente combinata con un valore di salt univoco. Il salt è un valore casuale o pseudocasuale che viene memorizzato insieme all'hash della password nel file delle password del sistema. Questo approccio aggiunge un livello di complessità che rende difficile per un attaccante indovinare la password.

Quando un utente registra una nuova password:

1. **Scelta e combinazione della password con salt:** La password scelta viene combinata con un valore di salt, generato con numeri pseudocasuali o casuali per garantire maggiore sicurezza.

2. **Generazione dell'hash:** La password e il salt diventano input di una funzione di hashing. Questa funzione produce un hash di lunghezza fissa, che viene poi memorizzato insieme al salt.

3. **Verifica durante il login:** Quando l'utente inserisce il proprio ID e password, il sistema recupera il salt e l'hash memorizzati. La password fornita viene combinata con il salt e trasformata tramite la stessa funzione di hashing. Se l'hash risultante corrisponde a quello memorizzato, la password è considerata corretta e l'utente viene autenticato.

4.7.2 Vantaggi del Salt

L'uso del salt aggiunge robustezza al sistema di hashing per tre motivi principali:

1. **Prevenzione di duplicati:** Se due utenti scelgono la stessa password, l'uso di valori di salt diversi per ciascun account garantisce che gli hash generati siano diversi.

2. **Aumento della complessità negli attacchi offline:** L'uso di un salt di b bit incrementa il numero di possibili combinazioni di password di un fattore 2^b. Questo significa che un attaccante deve testare ogni ipotesi di password in combinazione con ogni possibile valore di salt, moltiplicando il numero di tentativi necessari.

3. **Difesa contro riutilizzo di password su più sistemi:** Anche se un utente usa la stessa password su più sistemi, l'uso di salti casuali rende difficile per un attaccante determinare tale coincidenza tra diversi file delle password.

4.8 Cracking delle Password e Contromisure

4.8.1 Approcci Tradizionali

Metodi Basati su Dizionari

Consiste nel costruire un grande dizionario di possibili password e confrontarle con gli hash memorizzati. Ogni possibile password viene sottoposta alla funzione di hashing, utilizzando tutti i valori di salt disponibili, e poi confrontata con gli hash presenti

nei file delle password. Se non viene trovata alcuna corrispondenza, il software di cracking applica variazioni su ogni parola del dizionario, come inversione della parola, aggiunta di numeri, caratteri speciali o modifiche alle maiuscole e minuscole.

Rainbow Table

Un approccio alternativo consiste nel pre-calcolare gli hash per un'ampia gamma di password e valori di salt, memorizzando i risultati in una cosiddetta rainbow table. Questo consente di ridurre drasticamente il tempo di verifica. Tuttavia, l'uso di salti e lunghezze di hash sufficientemente grandi può rendere inefficaci le rainbow table.

Variazioni delle Password e Predicibilità

Un problema significativo deriva dalla scelta di password prevedibili da parte degli utenti. Spesso, le persone scelgono parole comuni, nomi personali o sequenze facilmente indovinabili, rendendo il cracking più semplice. Gli attaccanti possono sfruttare questa debolezza testando le password con liste di parole comuni, nomi propri e permutazioni basate su informazioni personali.

4.8.2 Approcci Moderni

Aumento della Potenza di Calcolo

Con l'evoluzione della tecnologia, la potenza di calcolo è aumentata in modo significativo. Le GPU consentono ai programmi di cracking di eseguire milioni di combinazioni al secondo, aumentando esponenzialmente la velocità rispetto ai sistemi basati esclusivamente su CPU.

Algoritmi di Generazione Sofisticati

Anche gli algoritmi per generare password si sono evoluti. Tecniche basate su modelli probabilistici e approcci di apprendimento automatico, come quelli derivati dall'elaborazione del linguaggio naturale, hanno permesso di ridurre il numero di tentativi necessari per violare una password. Utilizzando dati reali provenienti da fughe di password, i ricercatori hanno potuto creare modelli statistici delle scelte comuni degli utenti.

4.8.3 Contromisure

Controllo degli Accessi al File delle Password

Per contrastare un attacco alle password è necessario impedire l'accesso al file delle password da parte degli avversari. Se la sezione del file che contiene l'hash delle password è accessibile solo da utenti privilegiati, l'avversario non potrà leggerla senza prima conoscere la password di un utente privilegiato. In molti sistemi, gli hash delle password sono memorizzati in un file separato dagli ID utente, noto come file shadow delle password, che è rigorosamente protetto per evitare accessi non autorizzati.

Strategie di Selezione della Password

Quando gli utenti sono liberi di scegliere le proprie password, spesso tendono a sceglierne di troppo corte o facilmente intuibili. Se invece si assegnano password casuali e complesse, diventa difficile per gli utenti ricordarle. L'obiettivo ideale è consentire la scelta di password memorizzabili, ma non prevedibili. Per raggiungere tale obiettivo si usano diverse tecniche:

- **Educazione degli utenti:** Insegnare agli utenti a scegliere password sicure può essere utile, ma spesso non sufficiente, specialmente in ambienti con molti utenti.
- **Password generate dal computer:** Le password generate automaticamente possono risultare difficili da ricordare, riducendo l'accettazione da parte degli utenti.
- **Controllo reattivo delle password:** Il sistema può eseguire periodicamente un proprio cracker per individuare le password deboli. Sebbene efficace, questa strategia può essere onerosa in termini di risorse e non risolve il problema delle password deboli finché non vengono rilevate.
- **Politica delle password complesse:** Consente agli utenti di scegliere le proprie password, ma il sistema verifica se rispettano determinati criteri di complessità.

4.9 Autenticazione Basata su Token

I token rappresentano un metodo di autenticazione in cui l'utente possiede un oggetto fisico che serve per verificare la propria identità. Tra i tipi più comuni di token troviamo le memory card e le smart card.

4.9.1 Memory Card

Le memory card sono in grado di immagazzinare dati, ma non di elaborarli. L'esempio più noto è costituito dalle carte bancarie dotate di una banda magnetica, che memorizza un semplice codice di sicurezza leggibile e modificabile tramite un lettore di carte. Queste card possono essere utilizzate per accesso fisico e per autenticazione combinando la carta con un PIN o una password. Questo sistema offre maggiore sicurezza rispetto alla sola password, poiché un avversario dovrebbe entrare in possesso fisico della carta e conoscere il PIN.

Tuttavia, ci sono alcuni svantaggi:

- **Richiede un lettore speciale:** l'uso di un token comporta costi aggiuntivi per l'acquisto e la manutenzione del lettore.
- **Rischio di perdita del token:** se il token viene smarrito, il proprietario è temporaneamente impossibilitato ad accedere al sistema e deve sostenere il costo per la sua sostituzione.
- **Insoddisfazione degli utenti:** alcuni utenti potrebbero trovare scomodo l'uso di memory card per accedere a sistemi informatici.

4.9.2 Smart Card

Le smart card sono token che includono un microprocessore e possono assumere varie forme, dalle carte bancarie a dispositivi simili a chiavi o calcolatrici. Le smart card hanno diverse caratteristiche:

- **Caratteristiche fisiche:** includono un microprocessore, che rende possibile l'elaborazione di dati e operazioni complesse, come la crittografia.
- **Interfaccia utente ed elettronica:** la comunicazione può avvenire tramite contatti fisici o a distanza, utilizzando frequenze radio (contactless).
- **Protocollo di autenticazione:** le smart card possono implementare protocolli statici (autenticazione simile alle memory card), generare password dinamiche o usare metodi di challenge-response per rispondere a richieste del sistema.

Le smart card possono essere utilizzate come carte d'identità elettroniche (eID), offrendo funzioni come:

- **ePass:** per l'uso governativo, simile a un passaporto elettronico.
- **eID:** per l'accesso a servizi governativi e commerciali.
- **eSign:** per generare firme digitali.

4.10 Autenticazione Biometrica

L'autenticazione biometrica rappresenta una tecnologia che si basa sull'uso di caratteristiche fisiche uniche degli individui per garantire la sicurezza e l'accesso ai sistemi informatici. Le caratteristiche biometriche sono intrinseche a ciascun individuo e risultano difficili da replicare, offrendo così un livello di sicurezza superiore.

4.10.1 Tipologie di Caratteristiche Biometriche

Esistono numerosi tipi di caratteristiche fisiche e comportamentali che possono essere utilizzate per l'autenticazione biometrica. Tra queste, alcune delle più comuni e significative includono:

Caratteristiche Facciali

Il riconoscimento facciale è uno dei metodi più popolari poiché simula il modo in cui gli esseri umani si identificano visivamente. Le tecnologie di riconoscimento facciale identificano i tratti del viso, come la posizione e la forma degli occhi, del naso, della bocca e del mento. Un approccio alternativo utilizza termocamere a infrarossi per rilevare un termogramma del viso, che si basa sul sistema vascolare sottostante e offre un'ulteriore protezione contro le maschere o le immagini contraffatte.

Impronte Digitali

Le impronte digitali presentano un modello unico di rilievi e solchi che varia da persona a persona. I sistemi di riconoscimento digitale analizzano questi modelli ed estraggono caratteristiche specifiche per creare un "template" numerico.

Geometria della Mano

I sistemi basati sulla geometria della mano analizzano diverse caratteristiche, come la forma, la lunghezza e la larghezza delle dita. Seppur meno preciso, questo metodo è ampiamente utilizzato per applicazioni aziendali grazie alla sua rapidità e semplicità.

Schema della Retina

Sfrutta il pattern unico delle vene presenti nella parte posteriore dell'occhio. Questa tecnologia è molto precisa ma richiede apparecchiature specifiche. Gli utenti devono fissare un punto preciso per consentire la scansione.

Scansione dell'Iride

Il processo è meno invasivo rispetto alla scansione retinica e fornisce un elevato livello di accuratezza. Le scansioni dell'iride possono essere eseguite anche da una distanza moderata, rendendo la tecnologia più pratica per l'uso quotidiano.

Firma

Ogni persona possiede uno stile unico di scrittura, specialmente per quanto riguarda la firma. Sebbene ci siano naturali variazioni tra i campioni della stessa persona, il riconoscimento biometrico delle firme analizza la velocità, la pressione e il movimento utilizzati per creare la firma. Questo metodo può essere utile per verificare documenti e transazioni.

Voce

L'identificazione vocale sfrutta le caratteristiche uniche delle corde vocali e delle cavità risonanti di ciascun individuo. Tuttavia, i modelli vocali possono variare nel tempo a causa di fattori come malattie, invecchiamento e stress, rendendo complesso il riconoscimento vocale. Ciononostante, è spesso utilizzato per applicazioni come l'assistenza clienti e l'accesso remoto.

4.10.2 Accuratezza e Complessità

L'accuratezza dei sistemi biometrici può variare. A differenza delle password, che corrispondono esattamente o meno a ciò che è memorizzato, le misure biometriche devono essere valutate in termini di quanto strettamente un attributo corrisponde a quello salvato. Pertanto, i sistemi biometrici utilizzano criteri di tolleranza e soglie di accettazione per determinare una corrispondenza, affrontando problemi di falsi positivi (riconoscimento errato di un intruso come utente legittimo) e falsi negativi (rifiuto dell'accesso a un utente legittimo).

5 - Controllo degli Accessi

5.1 Introduzione

L'obiettivo principale del controllo degli accessi è quello di regolamentare chi può accedere alle risorse, in che modo e in quali circostanze.

5.1.1 Obiettivi Principali

Gli obiettivi principali della sicurezza informatica si concentrano su tre aspetti:

1. Impedire l'accesso non autorizzato, proteggendo così i dati sensibili e prevenendo potenziali danni.
2. Prevenire accessi non autorizzati da parte di utenti legittimi, per evitare che accedano in modi che non sono permessi o che compromettano la sicurezza del sistema.
3. Consentire l'accesso autorizzato, in modo che gli utenti legittimi possano accedere alle risorse in modo sicuro e conforme alle politiche di sicurezza.

5.1.2 Concetti di Base

Il controllo degli accessi è essenziale per mantenere la sicurezza e l'integrità di un sistema, regolando non solo chi ha accesso, ma anche come e quando può avvenire l'accesso.

5.1.3 Contesto del Controllo degli Accessi

Il controllo degli accessi opera all'interno di un contesto più ampio, che include le seguenti funzioni e componenti:

Autenticazione Processo di verifica delle credenziali di un utente o di un'entità per confermarne la validità.

Autorizzazione Concessione di diritti e permessi a un'entità per l'accesso a risorse di sistema, determinando chi è affidabile per un certo utilizzo.

Controllo Verifica indipendente e revisione delle attività e dei registri di sistema, per assicurare la conformità alla politica di sicurezza e individuare possibili violazioni.

Il controllo degli accessi può apparire come un unico modulo logico, ma spesso è suddiviso tra più componenti, ciascuno dei quali svolge una parte del processo. I sistemi operativi includono generalmente componenti di controllo degli accessi, che, in molti casi, sono robusti. Pacchetti di sicurezza aggiuntivi, applicazioni come i sistemi di gestione di database e dispositivi esterni, come i firewall, integrano queste funzionalità con ulteriori livelli di protezione.

5.2 Politiche di Controllo degli Accessi

Le politiche di controllo degli accessi definiscono i tipi di accesso consentiti, le condizioni di accesso e le entità autorizzate. Sono generalmente suddivise in quattro categorie principali. Il DAC rappresenta l'approccio più tradizionale per la gestione del controllo degli accessi, mentre il MAC risponde a esigenze di sicurezza militari e di fiducia elevata. Le politiche RBAC e ABAC sono sempre più diffuse e forniscono una maggiore flessibilità nella gestione dei privilegi. Queste quattro politiche non sono esclusive; possono essere combinate per garantire la sicurezza di diverse risorse all'interno di un sistema.

5.3 Controllo degli Accessi Discrezionale (DAC)

Il controllo degli accessi discrezionale (Discretionary Access Control) è basato sull'identità dell'entità richiedente e sulle autorizzazioni specifiche che stabiliscono quali azioni siano permesse o proibite. È definito "discrezionale" perché consente a un'entità di trasferire i propri privilegi a un'altra entità, autorizzandola ad accedere a determinate risorse. Questo approccio può essere adottato da un sistema operativo o un sistema di gestione di database attraverso una matrice degli accessi.

5.3.1 Matrice degli Accessi

La matrice degli accessi è una struttura dati bidimensionale in cui una dimensione rappresenta i soggetti (utenti o gruppi di utenti) e l'altra gli oggetti (risorse del sistema, come file o campi di dati). Ogni cella della matrice specifica i permessi di accesso di un soggetto su un oggetto. In pratica, la matrice è spesso implementata in modo sparso, tramite due principali modalità di decomposizione: le liste di controllo degli accessi (ACL) e i capability ticket.

5.3.2 Liste di Controllo degli Accessi

Una ACL elenca, per ogni oggetto, gli utenti autorizzati e i rispettivi permessi. Può includere anche una voce di default che definisce un insieme predefinito di permessi per utenti non elencati. Tale voce di default dovrebbe seguire la regola del minimo privilegio o concedere solo accesso in lettura. Le ACL sono vantaggiose per determinare i permessi su una specifica risorsa, ma non sono ideali per visualizzare i permessi associati a un singolo utente.

5.3.3 Capability Ticket

I capability ticket sono liste di permessi legati a un utente, specificando quali operazioni è autorizzato a eseguire su determinati oggetti. Ogni utente può possedere diversi ticket e, in alcuni casi, può trasferirli ad altri utenti.

5.3.4 Tabella di Autorizzazione

Per superare i limiti delle ACL e dei capability ticket, è possibile utilizzare una tabella di autorizzazione. In questa struttura, ogni riga contiene un permesso di accesso associato a un soggetto su una risorsa specifica.

5.4 Controllo degli Accessi Obbligatorio (MAC)

Il controllo degli accessi obbligatorio (Mandatory Access Control) è regolato dalle etichette di sicurezza delle risorse, che indicano il livello di sensibilità o criticità, e dalle autorizzazioni di sicurezza delle entità, che stabiliscono i livelli di accesso consentiti. È "obbligatorio" poiché l'entità autorizzata non può, a propria discrezione, estendere i suoi privilegi di accesso a un'altra entità.

5.5 Controllo degli Accessi Basato sui Ruoli (RBAC)

Il controllo degli accessi basato sui ruoli (Role-Based Access Control) si fonda sui ruoli che gli utenti ricoprono nel sistema, piuttosto che sulla loro identità individuale. Un ruolo rappresenta una funzione lavorativa all'interno dell'organizzazione, con specifici permessi di accesso assegnati a ciascun ruolo. Gli utenti vengono associati ai ruoli staticamente o dinamicamente, a seconda delle loro responsabilità operative.

5.5.1 Struttura del Modello

Le relazioni tra utenti, ruoli e risorse formano una struttura molti-a-molti. Gli utenti possono essere assegnati a più ruoli, e ogni ruolo può essere associato a più risorse. Il sistema RBAC si presta a una rappresentazione tramite una matrice di accesso. Questa matrice associa gli utenti ai ruoli e i ruoli alle risorse. In genere, gli utenti sono numerosi rispetto ai ruoli, quindi ogni utente può occupare più ruoli, e più utenti possono condividere lo stesso ruolo.

Questo sistema facilita l'applicazione del principio del minimo privilegio: ogni ruolo detiene solo i privilegi necessari per le funzioni assegnate. In questo modo, gli utenti assegnati allo stesso ruolo usufruiscono dello stesso insieme minimo di permessi.

5.6 Controllo degli Accessi Basato sugli Attributi (ABAC)

Il controllo degli accessi basato sugli attributi (Attribute-Based Access Control) è un modello che consente di definire autorizzazioni basate su condizioni riferite sia alle proprietà del soggetto che della risorsa. Permette, ad esempio, di configurare una regola unica per concedere il diritto di proprietà a chiunque sia il creatore di una risorsa, rendendo l'accesso flessibile e potente.

5.7 Gestione delle Identità, delle Credenziali e degli Accessi

La gestione delle identità, delle credenziali e degli accessi (Identity, Credential, and Access Management – ICAM) rappresenta un approccio completo per la gestione delle identità digitali, delle credenziali e del controllo degli accessi.

Questo sistema permette di:

- Creare identità digitali affidabili per persone e per Non-Person Entities come processi e dispositivi automatizzati che necessitano di accesso alle risorse.
- Associare queste identità a credenziali utilizzabili per l'accesso da parte di individui o NPE.
- Utilizzare le credenziali per autorizzare l'accesso a risorse aziendali.

5.7.1 Gestione delle Identità

La gestione dell'identità si concentra sulla creazione e gestione delle identità digitali di individui o NPE, indipendentemente da applicazioni o contesti specifici.

Creazione di un'Identità Digitale

La creazione di un'identità digitale inizia con la raccolta di dati, seguita dalla definizione di attributi che identificano univocamente l'individuo. La gestione dell'identità prevede un ciclo di vita completo, con procedure di protezione, accesso e condivisione dei dati di identità, nonché la possibilità di revoca.

5.7.2 Gestione delle Credenziali

Una credenziale è un oggetto che lega autorevolmente un'identità a un token di accesso, come smart card, chiavi crittografiche o certificati digitali. La gestione delle credenziali comprende diverse fasi:

1. **Certificazione della necessità di una credenziale:** un responsabile approva la richiesta di credenziali per un individuo o entità.
2. **Iscrizione:** il richiedente si registra, dimostrando la propria identità e fornendo dati biografici e biometrici.
3. **Generazione:** la credenziale viene creata tramite crittografia, firme digitali o altri processi.
4. **Distribuzione:** la credenziale viene assegnata al richiedente.
5. **Gestione del ciclo di vita:** include processi di revoca, sostituzione, riemissione e manutenzione generale della credenziale.

5.7.3 Gestione degli Accessi

La gestione degli accessi definisce le modalità con cui le entità possono ottenere l'accesso alle risorse aziendali, sia fisiche che logiche. La funzione di controllo degli accessi verifica l'identità del richiedente e utilizza le credenziali per autorizzare l'accesso. Per implementare il controllo degli accessi aziendale, sono necessarie tre componenti:

Gestione delle risorse Definisce le regole per ogni risorsa, includendo requisiti di credenziali e attributi necessari per accedere.

Gestione dei privilegi Stabilisce i permessi di accesso associati a ciascun individuo, formando il profilo di accesso basato sugli attributi della loro identità.

Gestione delle politiche Specifica le azioni consentite per un dato utente su una risorsa, in base agli attributi dell'utente, dell'oggetto e alle condizioni ambientali.

5.7.4 Federazione delle Identità

La federazione delle identità riguarda la fiducia reciproca tra organizzazioni nel riconoscere identità digitali e credenziali emesse da terze parti. Consente, infatti, di verificare identità esterne e di garantire l'identità dei propri utenti in contesti di collaborazione esterna. La federazione si basa su tecnologie, standard e politiche condivise che permettono alle organizzazioni di riconoscere e accettare identità e attributi provenienti da altre entità, facilitando così la cooperazione interorganizzativa.

5.8 Trust Framework

Nell'ambito dell'e-commerce e delle transazioni online, i concetti di fiducia, identità e attributi sono diventati fondamentali per aziende e fornitori di servizi. Per garantire efficienza, privacy e sicurezza legale, le transazioni seguono il principio del need-to-know: è importante conoscere solo gli attributi essenziali del cliente, come il numero di licenza, l'ID, la nazionalità, o altre informazioni specifiche che variano in base al contesto.

5.8.1 Scambio di Informazioni

Approccio Classico

Le transazioni tra organizzazioni diverse o con utenti singoli richiedono la condivisione di informazioni sull'identità, che includono attributi associati. Entrambe le parti devono avere fiducia nella sicurezza e privacy delle informazioni condivise. Il metodo classico prevede che un fornitore di servizi di identità autentichi l'utente e garantisca la validità degli attributi, mentre la parte che si fida accetta le credenziali fornite.

5.8.2 Open Identity Trust Framework (OITF)

Per superare i limiti della condivisione tradizionale, è stato sviluppato l'Open Identity Trust Framework (OITF), un sistema aperto e standardizzato per lo scambio di identità e attributi. Di seguito sono illustrati i principali termini e componenti di questo framework:

OpenID Uno standard che permette agli utenti di autenticarsi su più siti utilizzando un unico servizio di terze parti, semplificando la gestione delle identità digitali.

OIDF (OpenID Foundation) Un'organizzazione che supporta e promuove l'adozione di OpenID.

ICF (Information Card Foundation) Un ente che facilita l'uso delle Information Card, identità digitali organizzabili come carte di credito.

OIX (Open Identity Exchange) Fornisce certificazioni di conformità al modello OITF.

AXN (Attribute Exchange Network) Una rete che permette uno scambio efficiente di attributi di identità verificati e autorizzati.

5.8.3 Componenti Principali dell'OITF

L'OITF prevede i seguenti ruoli e componenti:

Relying Parties (RP) Fornitori di servizi che devono fidarsi delle identità e attributi degli utenti.

Soggetti Utenti finali come clienti, dipendenti o partner.

Fornitori di attributi (AP) Entità che verificano attributi specifici e generano credenziali conformi alle regole dell'AXN.

Fornitori di identità (IDP) Autenticano gli utenti e forniscono identità digitali utilizzate per gestire gli attributi degli utenti.

Periti Valutano e certificano la conformità dei fornitori di servizi e degli RP al modello OITF.

Revisori Verificano che le pratiche delle parti siano conformi agli accordi dell'OITF.

Risolutori di dispute Forniscono servizi di arbitrato e risoluzione dei conflitti.

Fornitori di infrastruttura di fiducia Implementano l'infrastruttura di fiducia richiesta per soddisfare le specifiche OITF.

In un sistema di identità digitale, il trust framework opera come un programma di certificazione che permette a una parte di fidarsi della sicurezza e delle politiche di privacy di un'altra parte. Più formalmente, il framework è composto da un insieme di accordi verificabili che includono controlli e misure correttive per garantire l'adempimento degli impegni presi. Ogni trust framework definisce i diritti e le responsabilità dei partecipanti e specifica le procedure per garantire la conformità.

6 - SQL Injection e Sicurezza dei Database

6.1 SQL Injection

Le SQL injection (o SQLi) sono un tipo di vulnerabilità in cui l'attaccante utilizza parte di un codice SQL per manipolare un database e accedere a informazioni potenzialmente preziose.

6.1.1 Come Funzionano le SQLi

SQLi sfrutta vulnerabilità presenti a livello database di un'applicazione (quali le query). Mediante un attacco di tipo SQL injection, l'attaccante può estrarre o manipolare i dati dell'applicazione Web.

6.1.2 Fonti di Attacco

È possibile caratterizzare gli attacchi SQLi in base al fronte di attacco e al tipo di attacco. Le principali fonti sono:

Input utente In cui l'attaccante inietta i comandi SQL fornendo un input opportunamente costruito.

Cookie Utilizzati per recuperare le informazioni sullo stato dell'utente, in cui l'attaccante potrebbe modificarli opportunamente per far in modo che, quando il server effettuerà una query basata sul contenuto dei cookie, la struttura e la funzionalità della query siano modificate.

Input fisico dell'utente Tramite il quale è possibile iniettare comandi SQL anche al di fuori del contesto di richieste Web. L'input può consistere di codici a barre, moduli cartacei che vengono digitalizzati attraverso riconoscimento dei caratteri e inviati al database management system.

6.2 Tipi di SQL Injection

I tipi di attacco possono essere raggruppati in tre categorie principali:

- Attacchi in banda
- Attacchi inferenziali
- Attacchi fuori banda

6.2.1 Attacco in Banda

Un attacco in banda sfrutta lo stesso canale di comunicazione per iniettare codice e per ottenere i risultati. I dati ottenuti sono direttamente visualizzati nella pagina web. Gli attacchi in banda si classificano in:

Tautologia

Questa forma di attacco inietta il codice in una o più istruzioni condizionali per far in modo che vengano valutate sempre come vere.

Esempio Supponiamo che abbiamo una query SQL in un'applicazione web che estrae i dati di un utente dal database in base al nome utente fornito tramite un form.

```
SELECT * FROM users WHERE username = '$username';
```

Se l'input non viene gestito correttamente, l'attaccante potrebbe inserire questo valore nel campo username:

```
' OR '1'='1
```

ottenendo:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

1. La condizione `username = ''` fallisce (non ci sono utenti con il nome vuoto).
2. Ma `OR '1'='1'` è sempre vero, quindi la query restituirà tutti gli utenti presenti nella tabella `users`, perché la condizione è sempre vera per tutti i record.

Commenti di Fine Riga

Dopo aver iniettato codice in un determinato campo, codici legittimi che seguono sono annullati dall'uso del commento di fine riga. In questo modo, il testo seguente non venga processato come codice ma come commento.

Esempio Supponiamo di avere una query che verifica le credenziali di un utente:

```
SELECT * FROM users WHERE username = '$username' AND password
    = '$pwd';
```

Se l'input non viene gestito correttamente, l'attaccante potrebbe inserire questo valore nel campo username:

```
' OR '1'='1' --
```

ottenendo:

```
SELECT * FROM users WHERE username = '' OR '1'='1' -- ' AND
    password = '';
```

1. La condizione `username = ''` fallisce (non ci sono utenti con il nome vuoto).
2. Ma `OR '1'='1'` è sempre vero, quindi la query restituirà tutti gli utenti presenti nella tabella `users`, perché la condizione è sempre vera per tutti i record.
3. `-` è un commento in SQL, quindi tutto ciò che viene dopo (in questo caso la verifica della password) viene ignorato.

Query Piggybacked

L'attaccante aggiunge ulteriori query rispetto a quella prevista, veicolando l'attacco a dorso di una richiesta legittima. Questa tecnica sfrutta configurazioni del server che permettono di eseguire diverse query all'interno di una singola riga di codice.

Esempio Supponiamo di avere una query SQL che estrae informazioni su un utente in base al suo nome utente:

```
SELECT * FROM users WHERE username = '$username';
```

Se l'input non viene gestito correttamente, l'attaccante potrebbe inserire questo valore nel campo username:

`'; DROP TABLE users; --`

ottenendo:

```
SELECT * FROM users WHERE username = ''; DROP TABLE users; --
    ';
```

1. La prima parte, `SELECT * FROM users WHERE username = '';`, non restituirà alcun risultato (il nome utente è vuoto).
2. La seconda parte, `DROP TABLE users;`, viene eseguita come una seconda istruzione SQL indipendente e cancella la tabella `users`.
3. Il commento `-` viene usato per ignorare il resto della query.

Questo attacco è devastante perché elimina completamente la tabella `users`, cancellando tutti i dati contenuti.

6.2.2 Attacco Inferenziale

In un attacco inferenziale non c'è un effettivo trasferimento di dati, ma l'attaccante è in grado di ricostruire alcune informazioni inviando particolari richieste e analizzando le risposte del server. Sono attacchi inferenziali:

Error-based SQL Injection

Un attacco basato su errori sfrutta le risposte di errore del database per dedurre informazioni sensibili. Manipolando una query in modo da far generare errori al database, l'attaccante può ricavare dettagli sulla struttura delle tabelle, i campi, o i dati stessi.

Blind SQL Injection

Permette ad un attaccante di risalire ai dati contenuti in un database anche quando questo è sufficientemente sicuro da non restituire informazioni sensibili all'attaccante. Quest'ultimo pone al server domande di tipo booleano. Se la query sottoposta viene valutata come vera, l'applicazione web continua a funzionare correttamente. Se la query sottoposta viene valutata falsa, anche se il server non restituisce un chiaro messaggio di errore, la pagina differisce notevolmente dalla pagina ottenuta nel normale funzionamento.

6.2.3 Attacco Fuori Banda

In un attacco fuori banda i dati vengono estratti sfruttando un canale differente (per esempio, un'e-mail con i risultati della query viene generata e inviata al richiedente). Questo può essere usato quando esistono vincoli sull'estrazione delle informazioni, ma il server del database implementa pochi controlli sulle comunicazioni in uscita attraverso altri canali.

6.3 Contromisure per le SQL Injection

Le contromisure possono essere classificate in tre categorie:

- programmazione difensiva
- rilevazione
- prevenzione a tempo di esecuzione

6.3.1 Programmazione Difensiva

Molti attacchi SQLi vanno a buon fine perché gli sviluppatori, spesso, hanno utilizzato tecniche di programmazione non sicure. Con la programmazione difensiva andiamo a ridurre i danni provocati da attacchi SQLi.

Pratiche Manuali di Programmazione Difensiva

Una tipica vulnerabilità sfruttata è la scarsa validazione dell'input. La soluzione più semplice consiste nel controllare che se il tipo di dato atteso è numerico, non contenga caratteri che non siano di questo tipo.

Inserimento Parametrizzato della Query

Questo approccio cerca di bloccare attacchi SQLi dando la possibilità allo sviluppatore di definire dettagliatamente la struttura di una query SQL, inserendo i valori dei parametri uno per volta, così che, qualsiasi input inserito dell'utente, non possa modificare la struttura della query.

SQL DOM

SQL DOM è un insieme di classi che abilitano la validazione automatica dei tipi di dati e dei caratteri di escape. Questo approccio sfrutta l'incapsulamento delle query per garantire un accesso sicuro e affidabile al database. Viene modificato il processo di costruzione delle query, passando da un approccio basato sulla concatenazione di stringhe a uno strutturato che utilizza una API che implementa controlli sui tipi.

6.3.2 Rilevamento

Numerosi metodi di rilevazione sono stati proposti e sviluppati, quali i seguenti:

Basati sulle Firme (signature-based)

Cercano di individuare specifiche sequenze di attacco. Tali approcci richiedono un aggiornamento costante e potrebbero non rilevare attacchi basati su codice automodificante.

Basati sulle Anomalie (anomaly-based)

Questo approccio tenta di individuare attività che si discostano dal modello standard del sistema monitorato. Si utilizzano numerosi approcci. Generalmente, vi è una fase di addestramento, in cui il sistema apprende il modello del comportamento tipico del sistema, seguita da una fase di vera e propria individuazione.

Analisi del Codice

Le tecniche di analisi del codice prevedono l'uso di suite di test per rilevare vulnerabilità SQLi. Questi test sono progettati per generare diversi tipi di attacchi SQLi e valutare la risposta del sistema.

6.3.3 Prevenzione a Tempo di Esecuzione

Sono state sviluppate diverse tecniche di prevenzione a tempo di esecuzione come strumento di difesa contro attacchi SQLi. Queste tecniche controllano le query a tempo di esecuzione per valutare se sono coerenti con il modello di query previsto.

6.4 Controllo degli Accessi ai Database

I DBMS forniscono controllo degli accessi che può essere discrezionale o basato sui ruoli. Tipicamente, i DBMS supportano varie politiche di gestione, fra cui:

- **Amministrazione centralizzata**, in cui un numero limitato di utenti può concedere o revocare permessi.
- **Amministrazione basata sulla proprietà**, dove il proprietario di una tabella (l'utente che la crea) può concedere e revocare i permessi.
- **Amministrazione decentralizzata** nella quale il proprietario di una tabella può autorizzare altri utenti a concedere o revocare permessi per quella tabella.

6.4.1 Definizione dei Permessi Basati su SQL

SQL fornisce due comandi per la gestione dei permessi, `GRANT` e `REVOKE`.

Sintassi del comando GRANT

```
GRANT {privileges | role}
[ON table]
TO {user | role | PUBLIC}
[IDENTIFIED BY password]
[WITH GRANT OPTION]
```

Sintassi del comando REVOKE

```
REVOKE {privileges | role}
[ON table]
FROM {user | role | PUBLIC}
```

6.4.2 Autorizzazioni in Cascata

L'istruzione `GRANT OPTION` permette la concessione di permessi in cascata tra più utenti, consentendo a chi riceve un permesso di trasmetterlo ad altri. Tuttavia, anche la revoca segue una logica a cascata: se l'utente che ha originariamente concesso il permesso lo revoca, tutti i permessi successivi trasmessi in base a quello originale vengono automaticamente revocati.

Ciò può generare complicazioni quando un utente riceve lo stesso permesso da più fonti. La convenzione generale adottata prevede che la revoca si applichi a tutti i permessi derivati, salvo che un permesso esista indipendentemente dalla concessione originale.

6.4.3 Controllo degli Accessi Basato sui Ruoli

Un sistema per il controllo degli accessi basato sui ruoli (Role-based Access Control, RBAC) rappresenta una valida soluzione per il controllo degli accessi in un database.

È possibile classificare gli utenti che accedono al database in tre categorie principali:

1. **Proprietario dell'applicazione**, il proprietario degli oggetti del database (come tabelle, colonne e righe).
2. **Utente finale diverso dal proprietario**, l'utente finale che utilizza alcuni oggetti del database attraverso un'applicazione, ma non è il proprietario di nessuno degli oggetti utilizzati.
3. **Amministratore**, l'utente che ha la responsabilità di amministrare, interamente o parzialmente, il database.

Ruoli Predefiniti

- I ruoli fissi del server sono definiti a livello del server, sono indipendenti da qualsiasi database utente e sono concepiti per semplificare l'amministrazione del server.
- I ruoli fissi del database hanno validità a livello del singolo database.
- Ai ruoli definiti dagli utenti è possibile assegnare permessi a porzioni del database. Esistono due tipi di ruoli definiti dall'utente: standard e applicazione.
    1. Per un ruolo di tipo standard, un utente può assegnare, ad altri utenti, lo stesso ruolo.
    2. Un ruolo di tipo applicazione è associato a un'applicazione piuttosto che a un gruppo di utenti, e richiede una password.

6.5 Inferenza nei Database

L'inferenza, nel contesto dei database, consiste nell'effettuare interrogazioni legittime e dedurre informazioni riservate attraverso le risposte ricevute.

Il problema dell'inferenza si presenta quando la combinazione di più dati rappresenta un'informazione più sensibile rispetto ai singoli dati. In generale, è possibile utilizzare due tecniche di inferenza per ottenere ulteriori informazioni:

- Analizzare le dipendenze funzionali tra gli attributi all'interno di una o più tabelle
- Unire viste che usano gli stessi vincoli

6.5.1 Contromisure

In generale, esistono due approcci per contrastare l'inferenza tramite l'uso di algoritmi per la rilevazione:

Rilevamento in Fase di Progettazione del Database

Questo approccio elimina un canale di inferenza modificando la struttura del database o intervenendo sul sistema per il controllo degli accessi. Alcuni esempi comprendono la rimozione delle dipendenze fra i dati dividendo una tabella in più tabelle oppure l'aumento del livello di dettaglio nell'assegnazione dei ruoli in un sistema RBAC.

Rilevamento in Fase di Esecuzione delle Query

Questo approccio mira ad eliminare un canale di inferenza durante l'esecuzione di una o più query. Se viene individuato un canale di inferenza, la query viene bloccata o modificata.

6.6 Cifratura dei Database

La cifratura rappresenta l'ultimo meccanismo di difesa nella sicurezza dei database. La cifratura dei database implica due svantaggi:

Gestione delle chiavi Gli utenti autorizzati devono conoscere la chiave di decifratura per poter accedere ai dati. Fornire le chiavi relative alle singole parti di un database, diventa un problema complesso.

Rigidità Quando il database viene cifrato, interamente o parzialmente, eseguire la ricerca sui record diventa più complesso.

La cifratura può essere applicata all'intero database, a livello di record (cifrare determinati record), a livello di attributi (cifrare determinate colonne) o a livello del singolo campo.

7 - Malware

7.1 Introduzione

Il malware è un programma inserito in un sistema, di solito in modo nascosto, con l'intento di compromettere la confidenzialità, l'integrità o la disponibilità dei dati della vittima.

7.1.1 Classificazione dei Malware

Il malware può essere classificato in due grandi categorie: in base ai meccanismi di propagazione e alle azioni eseguite (o payload) una volta raggiunto il bersaglio.

7.1.2 Propagazione dei Malware

Il malware può propagarsi aggiungendo a programmi presenti nel sistema, porzioni di codice malevolo. La prima categoria di propagazione del malware riguarda frammenti di software parassita che si attaccano a contenuti eseguibili esistenti. Il frammento può essere codice macchina che infetta un'applicazione, un programma di utilità o persino il codice utilizzato per l'avvio del sistema. Più recentemente, il frammento è stato rappresentato da codice di scripting, utilizzato in file come documenti Word, fogli di calcolo Excel o documenti PDF.

7.2 Tipi di Malware

Il malware comprende una varietà di software dannosi progettati per compromettere la sicurezza, la privacy o l'integrità dei sistemi informatici. Di seguito sono presentati i principali tipi di malware con una descrizione dettagliata delle loro caratteristiche e modalità di azione.

7.2.1 Advanced Persistent Threat (APT)

Un APT è una minaccia avanzata e persistente che utilizza diverse forme di malware per attaccare un obiettivo specifico in modo continuo e discreto per un lungo periodo. Gli attacchi APT sono spesso sofisticati, orchestrati da gruppi di hacker organizzati, e mirano a ottenere l'accesso prolungato a una rete per rubare dati sensibili o sabotare le operazioni del bersaglio.

7.2.2 Adware

L'adware è un tipo di software che visualizza o scarica pubblicità indesiderata, spesso sotto forma di pop-up o reindirizzamenti a siti web commerciali. Sebbene alcune forme di adware possano essere relativamente innocue, altre possono compromettere l'esperienza dell'utente, raccogliere informazioni sulla navigazione e risultare difficili da rimuovere.

7.2.3 Exploit Kit

Un exploit kit è un insieme di strumenti che automatizza la creazione e la distribuzione di nuovo malware sfruttando vulnerabilità software note. Questi kit offrono meccanismi di propagazione sofisticati e moduli payload che anche utenti inesperti possono combinare e distribuire. Un esempio di exploit kit è il famoso Zeus, noto per il furto di credenziali bancarie. Altri kit, come Blackhole, Sakura e Phoenix, sono stati ampiamente utilizzati per generare attacchi complessi.

7.2.4 Backdoor

Le backdoor sono meccanismi che consentono di aggirare i normali controlli di sicurezza, garantendo un accesso non autorizzato a un sistema compromesso o a funzionalità di un programma. Le backdoor possono essere installate da attaccanti per ottenere un punto di accesso permanente, permettendo loro di eludere le protezioni di sicurezza.

7.2.5 Exploits

Gli exploit sono pezzi di codice progettati per sfruttare specifiche vulnerabilità nei software. Gli attacchi che utilizzano exploit possono danneggiare i sistemi, eseguire codice non autorizzato o accedere a dati sensibili.

7.2.6 Flooders (DoS Client)

Questi strumenti vengono utilizzati per generare un grande volume di traffico o dati al fine di inondare e sovraccaricare un sistema informatico o una rete. Il risultato è un attacco denial-of-service (DoS), che rende inaccessibili le risorse del sistema.

7.2.7 Keyloggers

I keylogger monitorano e registrano ogni tasto premuto su un sistema compromesso, fornendo agli attaccanti dati sensibili come password, numeri di carta di credito e altre informazioni riservate.

7.2.8 Logic Bomb

Le logic bomb sono malware programmati per rimanere inattivi fino al verificarsi di una specifica condizione predefinita, come una data o un evento. Una volta attivate, possono eseguire azioni dannose, come cancellare dati o disattivare sistemi.

7.2.9 Macro Virus

I macro virus utilizzano codice o script incorporati in documenti, che si attivano quando il documento viene aperto o modificato. Sono particolarmente diffusi in documenti creati con software come Word o Excel.

7.2.10 Mobile Code

Il mobile code consiste in script o software che possono essere trasferiti ed eseguiti su diverse piattaforme senza modifiche. Esempi comuni includono JavaScript e ActiveX, spesso usati nelle pagine web.

7.2.11 Rootkit

I rootkit sono insiemi di strumenti progettati per ottenere e mantenere il controllo root (amministratore) su un sistema senza essere rilevati. Possono nascondere la presenza di altri malware e modificare il funzionamento del sistema.

7.2.12 Spyware

Lo spyware raccoglie informazioni dal computer dell'utente senza il suo consenso e le trasmette a una terza parte. Questo software può monitorare la digitazione, registrare dati sullo schermo o analizzare i file del sistema.

7.2.13 Trojan

I trojan appaiono come programmi utili ma nascondono una funzione dannosa. Eludono i meccanismi di sicurezza sfruttando le autorizzazioni legittime del sistema e possono portare all'installazione di altri malware.

7.2.14 Virus

Un virus è un tipo di malware che, una volta eseguito, tenta di replicarsi inserendosi in altri file o programmi eseguibili. Quando questi file vengono avviati, il virus si attiva e può causare danni o ulteriori infezioni.

7.2.15 Worm

Un worm è un programma che si replica autonomamente e si diffonde ad altri host in rete sfruttando vulnerabilità software. I worm possono propagarsi rapidamente, causando disagi su vasta scala.

7.2.16 Zombie, Bot

Un bot o zombie è un programma che, una volta installato su un sistema compromesso, può essere utilizzato per lanciare attacchi contro altri sistemi, spesso coordinato da un server di comando e controllo in una botnet.

7.3 Virus

7.3.1 Introduzione

Un virus informatico è un software che infetta altri programmi, modificandoli. La modifica include l'iniezione di codice che consente al virus di replicarsi e infettare altri contenuti.

Come i virus biologici, un virus informatico contiene il "codice istruttivo" per replicarsi. Ogni volta che il computer infetto entra in contatto con un codice non infetto, il virus si trasferisce in questo nuovo luogo, diffondendosi da computer a computer. La diffusione può avvenire tramite supporti fisici come dischi o chiavette USB, oppure tramite reti, dove documenti, applicazioni e servizi di sistema condivisi facilitano la propagazione.

7.3.2 Struttura Generale

Un virus è costituito da tre parti:

Meccanismo di infezione Il mezzo con cui un virus si propaga, permettendogli di replicarsi. Il meccanismo viene anche chiamato vettore di infezione.

Trigger L'evento che determina quando il payload viene attivato o consegnato, talvolta noto come bomba logica.

Payload Ciò che il virus fa, oltre a diffondersi. Il payload può implicare un danno oppure un'attività benigna ma percettibile.

Una volta avviato, il virus cerca di infettare altri file eseguibili non infetti. Se vengono soddisfatte determinate condizioni, il virus esegue il suo payload e poi trasferisce il controllo al programma originale.

7.3.3 Compressione del File Infetto

Un virus può anche comprimere il file eseguibile infetto per evitare che la sua dimensione più grande rispetto al file originale non infetto possa farne scoprire la modifica. Questo processo di compressione e decompressione consente al virus di rimanere nascosto durante l'esecuzione del programma.

7.3.4 Fasi di un Virus

Un virus tipicamente attraversa le seguenti quattro fasi:

1. **Fase dormiente:** il virus è inattivo. Al termine di questa fase il virus verrà attivato da qualche evento. Non tutti i virus prevedono questa fase.

2. **Fase di propagazione:** il virus inserisce una copia di se stesso in altri programmi o in determinate aree di sistema del disco. La copia potrebbe non essere identica alla versione che si sta propagando in quanto i virus spesso mutano per sfuggire al rilevamento. Ogni programma infetto conterrà a questo punto un clone del virus, che a sua volta entrerà in una fase di propagazione.

3. **Fase di attivazione:** il virus viene attivato per svolgere la funzione cui è destinato. Come per la fase dormiente, la fase di attivazione può essere scatenata da numerosi eventi di sistema, compreso il conteggio del numero di volte che questa copia del virus ha creato copie di se stessa.

4. **Fase di esecuzione:** viene espletata la funzione. La funzione può essere innocua, come un messaggio sullo schermo, o dannosa, come la distruzione di programmi e di file di dati.

7.3.5 Classificazione dei Virus

Classifichiamo i virus in base al tipo di target che il virus cerca di infettare e il metodo di cui fa uso il virus per nascondersi dal rilevamento da parte degli utenti e dagli antivirus.

In Base al Target da Attaccare

Boot sector infector Infetta un master boot record o un boot record e si diffonde quando un sistema viene avviato dal disco contenente il virus.

File infector Infetta i file che il sistema operativo o la shell considerano come eseguibili.

Macro virus Infetta i file con macro o codice di scripting che viene interpretato da un'applicazione.

Virus multipartito Infetta i file in diversi modi. In genere, il virus multipartito è in grado di infettare più tipi di file, quindi l'eliminazione totale del virus prevede l'intervento in tutti i potenziali luoghi di infezione.

In Base alla Strategia di Camuffamento

Una classificazione dei virus per strategia di camuffamento comprende le seguenti categorie:

Virus criptato Che utilizza la crittografia per occultare il proprio contenuto. Una porzione di virus crea una chiave crittografica casuale e cifra il resto del virus. La chiave viene memorizzata assieme al virus. Quando viene invocato un programma infetto, il virus utilizza la chiave casuale memorizzata per decifrare il virus. Quando il virus si replica, viene selezionata una chiave casuale diversa. Dal momento che buona parte del virus è cifrata con una chiave diversa per ogni istanza, non è possibile osservare un pattern di bit costante.

Virus furtivo Dove l'intero virus, e non soltanto un payload, viene nascosto. Può utilizzare mutazione del codice, compressione o tecniche di rootkit per raggiungere tale scopo.

Virus polimorfo Che nel corso della replicazione crea copie funzionalmente equivalenti, ma con pattern di bit nettamente diversi per eludere gli antivirus. In questo caso, la signature del virus sarà diversa per ogni copia. Per ottenere tale variazione, il virus può inserire in modo casuale istruzioni inutili oppure scambiare l'ordine di istruzioni indipendenti. Un approccio più efficace consiste nell'utilizzare la crittografia, come per i virus criptati. La porzione di virus responsabile della generazione delle chiavi e dell'esecuzione della cifratura/decifratura viene detta mutation engine. La stessa mutation engine viene modificata a ogni utilizzo.

Virus metamorfico Che si ridefinisce completamente a ogni iterazione, utilizzando più tecniche di trasformazione, incrementando di conseguenza la complessità del rilevamento. I virus metamorfici possono cambiare il proprio comportamento così come l'aspetto.

7.3.6 Macro Virus e di Scripting

Un macro virus si attacca ai documenti e utilizza le funzionalità di programmazione macro dell'applicazione del documento per essere eseguito e propagarsi.

Risultano particolarmente minacciosi per una serie di ragioni:

1. Un macro virus è indipendente dalla piattaforma, infatti essi infettano il contenuto attivo in applicazioni di uso comune come documenti della suite Office o documenti Adobe PDF. Qualsiasi piattaforma hardware e sistema operativo che supporti queste applicazioni può essere infettato.

2. I macro virus infettano documenti, non porzioni eseguibili di codice e si diffondono con facilità in quanto i documenti generalmente vengono condivisi nel corso di un normale utilizzo.

3. Dal momento che i macro virus infettano i documenti dell'utente, piuttosto che i programmi di sistema, i tradizionali controlli di accesso al file system hanno un'utilità limitata nel prevenire la loro diffusione, visto che si suppone che siano gli utenti a modificarli.

4. I macro virus sono molto più facili da creare o modificare rispetto ai tradizionali virus eseguibili.

5. I dettagli delle macro dipendono dall'applicazione che le interpreta, quindi colpiranno sempre i documenti per un'applicazione specifica.

Un altro possibile obiettivo per malware di tipo macro virus è costituito dai documenti PDF di Adobe. Tali documenti possono supportare una serie di componenti integrati, inclusi Javascript e altri tipi di codice di scripting.

7.4 I Worm

Un worm è un programma che cerca attivamente altre macchine da infettare e poi ogni macchina infettata sferra, a sua volta, attacchi ad altre macchine.

7.4.1 Meccanismi di Diffusione

I programmi worm sfruttano le vulnerabilità del software nei programmi client o server per ottenere l'accesso a ogni nuovo sistema. Per replicarsi, un worm si serve di mezzi per accedere ai sistemi remoti. Fra questi vi sono i seguenti, la maggior parte dei quali è ancora oggi in uso:

Posta elettronica o messaggistica istantanea Un worm invia una copia di se stesso ad altri sistemi via e-mail o messaggistica istantanea, cosicché il suo codice venga eseguito quando l'e-mail o l'allegato viene ricevuto o visualizzato.

Condivisione di file Un worm crea una copia di se stesso oppure infetta altri file idonei su un supporto rimovibile come una chiavetta USB; poi entra in esecuzione quando l'unità viene collegata a un altro sistema utilizzando il meccanismo di esecuzione automatica, sfruttando una qualche vulnerabilità del software, o quando un utente apre il file infetto sul sistema target.

Funzionalità di esecuzione remota Un worm esegue una copia di se stesso su un altro sistema utilizzando un'esplicita funzione di esecuzione remota o sfruttando una falla di programma in un servizio di rete per sovvertirne le operazioni.

Funzionalità di trasferimento o accesso remoto ai file Un worm utilizza un servizio di accesso remoto ai file o di trasferimento a un altro sistema per copiarsi da un sistema all'altro, dove gli utenti di quel sistema potrebbero poi eseguirlo.

Funzionalità di login remoto Un worm accede a un sistema remoto come utente e poi utilizza dei comandi per copiare se stesso da un sistema all'altro, dove poi viene eseguito.

Di norma un worm presenta le stesse fasi di un virus informatico: dormiente, di propagazione, di attivazione e di esecuzione.

7.4.2 Ricerca del Target

La prima operazione nella fase di propagazione di un worm di rete consiste nel cercare altri sistemi da infettare, un processo noto come scanning o fingerprinting. Tipicamente, il codice del worm appena installato sulle macchine infette ripete lo stesso processo di scanning, fino a quando non viene creata un'ampia rete distribuita di macchine infette.

Un worm utilizza generalmente le seguenti strategie di scanning degli indirizzi di rete:

Casuale Ogni host compromesso analizza indirizzi casuali nello spazio degli indirizzi IP servendosi di un seme diverso.

Hit-list L'attaccante redige anzitutto una lunga lista di macchine potenzialmente vulnerabili. Questo può essere un processo lento e prolungato per evitare che si scopra che è in corso un attacco. Una volta che la lista è stata stilata, l'attaccante inizia a infettare le macchine presenti nella lista. A ogni macchina infettata viene fornita una porzione della lista da scansionare. Questa strategia si traduce in un processo di scanning piuttosto rapido che potrebbe rendere difficile rilevare che l'infezione è in corso.

Topologica Questo metodo utilizza le informazioni contenute nelle macchine vittime infettate per trovare altri host da scansionare.

Sottorete locale Se un host può essere infettato alle spalle di un firewall, tale host poi cerca target nella propria rete locale.

7.4.3 Propagazione del Worm

La propagazione si sviluppa seguendo tre fasi. Nella fase iniziale il numero di host aumenta esponenzialmente. Dopo un po' gli host infetti sprecano tempo attaccando host già infetti, il che riduce il tasso d'infezione. Durante questa fase intermedia la crescita è approssimativamente lineare, ma il tasso di infezione è elevato. Quando la maggior parte dei computer vulnerabili è stata infettata, l'attacco entra in una fase finale lenta in cui il worm va alla ricerca degli host rimanenti di difficile identificazione.

7.5 Vulnerabilità Lato Client

Le vulnerabilità nel software, in particolare nelle applicazioni client come browser e plugin, rappresentano un obiettivo comune per gli attaccanti. L'approccio più comune per sfruttare queste vulnerabilità è il drive-by-download, una tecnica che consente il download e l'installazione di malware sul sistema dell'utente senza il suo consenso o la sua consapevolezza.

7.5.1 Drive-by-Download

Un attacco drive-by-download si verifica quando un utente visita una pagina web compromessa, che contiene codice malevolo. Questo codice sfrutta vulnerabilità nel browser o nei plugin installati per scaricare e installare malware. A differenza dei worm, che si diffondono attivamente, il malware in un attacco drive-by-download attende semplicemente che l'utente visiti la pagina infetta.

7.5.2 Watering Hole

Questa variante degli attacchi drive-by-download è particolarmente mirata. Gli attaccanti identificano i siti web che le loro vittime designate sono propense a visitare e quindi cercano vulnerabilità in questi siti. L'obiettivo è compromettere tali siti in modo che, quando la vittima li visita, il codice malevolo venga eseguito solo sul suo sistema, riducendo la possibilità che il sito venga scoperto.

7.5.3 Malvertising

Il malvertising è un'altra tecnica utilizzata per diffondere malware senza compromettere direttamente i siti web. Gli attaccanti acquistano spazi pubblicitari che contengono malware e li pubblicano su siti legittimi. Gli annunci malevoli possono essere progettati per attivarsi solo in determinati orari o per determinati gruppi di utenti, rendendo più difficile la loro identificazione e rimozione.

7.5.4 Clickjacking

Il clickjacking è una vulnerabilità sfruttata dagli attaccanti per manipolare le interazioni degli utenti con un'interfaccia web. Questo tipo di attacco mira a ingannare gli utenti, indirizzando i loro clic verso elementi invisibili o mascherati, piuttosto che verso gli oggetti legittimi a cui si stanno rivolgendo.

Gli attaccanti utilizzano tecniche che impiegano più livelli di elementi grafici, come iframe o oggetti HTML, rendendoli invisibili o posizionandoli in modo ingannevole sopra o sotto elementi legittimi. Ciò consente all'attaccante di dirottare i clic dell'utente.

7.6 Trojan

Un trojan è un programma, in apparenza utile, contenente codice nascosto che, se invocato, svolge alcune operazioni indesiderate o dannose.

Possono essere utilizzati per compiere in maniera indiretta azioni che l'attaccante non potrebbe svolgere direttamente. I trojan possono rientrare in uno di questi tre modelli:

1. Proseguire nell'esecuzione del programma originale e svolgere l'attività malevola in parallelo.
2. Proseguire nell'esecuzione del programma originale, ma modificarne alcune funzioni per svolgere attività dannose (ad esempio, un programma di login che raccoglie le password) o per mascherarne altre (ad esempio, una versione trojan horse di un programma che elenca i processi che non mostra determinati processi dannosi).
3. Svolgere una funzione dannosa che sostituisce completamente la funzione del programma originale.

Alcuni trojan evitano la richiesta di intervento da parte dell'utente sfruttando alcune vulnerabilità software che consentono la loro installazione ed esecuzione in automatico. A differenza dei worm, i trojan non si replicano.

7.7 Payload di un Malware

Quando un malware viene attivato su un sistema target, l'attenzione si sposta sul suo payload, cioè le azioni specifiche che il malware eseguirà. Molti malware sono progettati per eseguire attività malevole a beneficio dell'attaccante, come la distruzione dei dati, il furto di informazioni o il danneggiamento di hardware e software. Alcuni payload possono restare latenti fino all'attivazione in base a specifiche condizioni, note come bombe logiche.

7.7.1 Distruzione dei Dati

La distruzione dei dati è uno dei payload più antichi e pericolosi. Un esempio è il virus Chernobyl (1998), che infettava i file eseguibili su Windows 95 e 98. Una volta attivato, cancellava il primo megabyte del disco rigido, danneggiando gravemente il file system. Un altro caso è il worm Klez (2001), che si diffondeva via e-mail e, in certe date, cancellava i file sui sistemi infetti.

7.7.2 Ransomware

Una variante ancora più dannosa è il ransomware, un malware che cripta i dati dell'utente e richiede un pagamento (di solito in criptovaluta) per fornire la chiave di decrittazione. Versioni più recenti di ransomware utilizzano chiavi di crittografia così avanzate che non possono essere facilmente violate, costringendo le vittime a pagare per recuperare i propri file.

WannaCry

Un esempio significativo di ransomware è WannaCry (2017), che ha infettato migliaia di sistemi in tutto il mondo, criptando i file e richiedendo un riscatto in Bitcoin per il

loro recupero. Questo attacco ha colpito gravemente organizzazioni in diversi settori, causando perdite economiche significative e attirando molta attenzione mediatica. WannaCry ha sfruttato una vulnerabilità nei sistemi Windows e ha sottolineato l'importanza di backup regolari e di una strategia di disaster recovery.

7.7.3 Bombe Logiche

Le bombe logiche sono un altro tipo di payload comune nei malware. Sono porzioni di codice programmato per attivarsi quando si verificano determinate condizioni, come una specifica data, l'uso di un determinato software o l'accesso da parte di un particolare utente. Quando queste condizioni vengono soddisfatte, la bomba logica "esplode", causando la cancellazione di dati, l'arresto del sistema o altri danni.

7.8 Agenti di Attacco

7.8.1 Bot

Un bot (o zombie) assume segretamente il controllo di un altro computer collegato a Internet per poi utilizzarlo per lanciare attacchi che sono difficili da ricondurre al creatore del malware stesso.

7.8.2 Botnet

Il bot viene in genere installato su centinaia o migliaia di computer appartenenti a terzi ignari. I sistemi compromessi comprendono PC, server e dispositivi embedded quali router o videocamere di sorveglianza. L'insieme dei bot è spesso in grado di agire in modo coordinato; una rete di questo tipo prende il nome di botnet. Questo tipo di payload compromette l'integrità e la disponibilità del sistema infettato.

7.8.3 Uso dei Bot

I bot vengono generalmente utilizzati per i seguenti attacchi:

Attacchi DDoS Le botnet vengono impiegate per sovraccaricare un server o una rete con un flusso massiccio di traffico, causando l'interruzione del servizio per gli utenti legittimi.

Spamming Grazie a una botnet, gli attaccanti possono inviare enormi quantità di e-mail di spam, utilizzando i bot per aggirare i controlli anti-spam e moltiplicare la portata dell'attacco.

Sniffing del traffico I bot possono essere dotati di sniffer per intercettare dati sensibili (come credenziali) trasmessi in chiaro su una rete. Questo consente agli attaccanti di rubare informazioni critiche.

Keylogging Se i dati sono trasmessi in modo criptato (ad esempio, via HTTPS), lo sniffing potrebbe non essere efficace. Tuttavia, i bot possono installare keylogger per catturare tutto ciò che viene digitato sulla tastiera, come nomi utente e password.

Diffusione di malware Le botnet possono essere utilizzate per diffondere altri tipi di malware, come worm o virus, sfruttando la capacità dei bot di scaricare ed eseguire file dannosi su larga scala.

Installazione di adware Per ottenere guadagni finanziari, le botnet possono generare clic automatizzati su pubblicità online. I bot possono anche reindirizzare le pagine iniziali dei browser delle macchine compromesse per incrementare ulteriormente i clic sugli annunci.

Attacchi a reti IRC Le botnet possono attaccare le reti di chat IRC attraverso attacchi "clone", dove migliaia di bot si connettono simultaneamente alla rete bersaglio, causando disservizi simili a un DDoS.

Manipolazione di sondaggi e giochi online Poiché ogni bot ha un indirizzo IP unico, le botnet possono essere usate per manipolare i risultati di sondaggi online o giochi, con migliaia di voti falsi che sembrano provenire da utenti reali.

7.8.4 Controllo Remoto

La funzione di controllo remoto è l'aspetto chiave che differenzia un bot da un worm. Un worm è auto-propagante e agisce autonomamente, mentre un bot viene gestito tramite un'infrastruttura di command-and-control (CnC), che consente agli attaccanti di controllare il bot in modo diretto o periodico, in base alla connessione del bot alla rete.

Evoluzione del Controllo Remoto

Inizio tramite IRC I primi bot utilizzavano server IRC per comunicare. Tutti i bot si collegavano a un canale IRC e interpretavano i messaggi ricevuti come comandi.

Uso di HTTP e altri protocolli Le botnet moderne evitano l'uso di IRC per minimizzare i rischi di individuazione e utilizzano protocolli come HTTP per comunicazioni più nascoste. Alcune utilizzano anche meccanismi peer-to-peer per distribuire il controllo, evitando punti di vulnerabilità singoli.

Tecniche Avanzate di Protezione dei CnC

Generazione automatica di nomi di dominio Malware più recenti usano algoritmi per generare automaticamente numerosi nomi di dominio. Questo consente agli attaccanti di cambiare rapidamente server di controllo in caso di compromissione, rendendo difficile per le autorità sequestrare tutti i domini.

Fast-flux DNS Questa tecnica cambia rapidamente gli indirizzi IP associati a un dominio, utilizzando server proxy all'interno della botnet stessa. I server CnC possono quindi alternarsi frequentemente, rendendo più difficile la loro individuazione e disattivazione.

Modalità Operative del Modulo di Controllo

- Il modulo di controllo può inviare comandi ai bot per eseguire procedure preconfigurate, come attacchi DDoS o furto di dati.
- Per maggiore flessibilità, il modulo può ordinare ai bot di scaricare ed eseguire nuovi file, trasformandoli in strumenti più generici per una varietà di attacchi.
- Inoltre, il modulo raccoglie i dati sottratti dai bot, che vengono poi sfruttati dall'attaccante.

7.8.5 Contromisure

Un'efficace contromisura contro una botnet è il sequestro o la disattivazione della sua infrastruttura CnC.

7.9 Furto di Credenziali

Il furto di credenziali tramite keylogger e spyware è una tecnica comune utilizzata dagli attaccanti per aggirare le protezioni crittografiche, come HTTPS o POP3S, che proteggono i dati durante la trasmissione.

7.9.1 Keylogger

Un keylogger è un malware che registra le sequenze di tasti digitate sulla tastiera di un computer infetto. Anche se le credenziali vengono inviate su canali cifrati, il keylogger cattura i dati mentre vengono inseriti, prima che vengano criptati. Poiché registra ogni sequenza di tasti, i keylogger moderni utilizzano meccanismi di filtraggio, restituendo solo le informazioni pertinenti, come login, password, o URL sensibili.

Contromisure

Per contrastare i keylogger, alcuni siti bancari e servizi online hanno adottato applet grafiche per l'inserimento delle password. Questi strumenti non si basano sul testo digitato con la tastiera, rendendo inefficaci i keylogger tradizionali.

7.9.2 Spyware

In risposta all'uso di contromisure grafiche, gli attaccanti hanno sviluppato spyware più avanzati, capaci di:

- Monitorare la cronologia e l'attività del browser.
- Reindirizzare le richieste di siti web a pagine false controllate dall'attaccante.
- Modificare dinamicamente i dati scambiati tra il browser e siti web legittimi, rubando o alterando informazioni personali e finanziarie.

Trojan Bancario Zeus

Un esempio noto di spyware sofisticato è il Trojan Zeus, distribuito tramite il toolkit crimeware. Zeus combina funzionalità di keylogger e di manipolazione dei form web, rubando credenziali bancarie e modificando i dati inseriti dagli utenti su siti finanziari. Zeus viene tipicamente diffuso tramite email di spam o con attacchi drive-by-download su siti compromessi, infettando i dispositivi senza che l'utente se ne accorga.

7.9.3 Phishing

Il phishing è una tecnica di attacco che sfrutta l'ingegneria sociale per rubare credenziali e informazioni sensibili attraverso e-mail ingannevoli. Queste e-mail solitamente contengono un link a un sito web fraudolento, progettato per imitare pagine di login spingendo la vittima a inserire le proprie credenziali.

Gli attaccanti spesso usano un pretesto di urgenza, come l'imminente blocco dell'account, per convincere la vittima ad agire rapidamente. Una volta che l'utente fornisce i dettagli richiesti, gli attaccanti possono accedere all'account per sfruttarlo.

Meccanismo del Phishing

1. **Email ingannevole:** L'attaccante invia una e-mail che sembra provenire da una fonte affidabile, contenente un link a un sito fraudolento o un modulo da compilare e restituire.

2. **Sito web ingannevole:** Il sito web è progettato per sembrare autentico, ma è controllato dall'attaccante.

3. **Furto d'identità:** Con abbastanza informazioni raccolte, l'attaccante può assumere l'identità della vittima, accedendo a conti bancari, ottenendo credito o compiendo altre azioni fraudolente.

Spam e Botnet

Le e-mail di phishing vengono solitamente inviate a un vasto numero di destinatari tramite botnet. Anche se molte e-mail non colpiscono i destinatari corretti, una piccola percentuale di persone può essere ingannata, rendendo comunque l'attacco redditizio.

Spear-phishing

Una variante più mirata è il spear-phishing, in cui gli attaccanti selezionano con attenzione le vittime e creano e-mail personalizzate. Questo tipo di attacco è spesso utilizzato per spionaggio industriale o frodi finanziarie, come false autorizzazioni di trasferimenti bancari.

7.10 Stealthing

Con il termine stealthing si intendono le tecniche utilizzate dai malware per nascondere la propria presenza in un sistema infetto, rendendo difficile per il software antivirus rilevarlo e rimuoverlo.

7.10.1 Backdoor

Una backdoor (o trapdoor) è un accesso segreto a un sistema che consente a un utente autorizzato di bypassare le normali procedure di sicurezza.

Tipologie e Utilizzi

Backdoor legittime (o maintenance hooks), progettate dagli sviluppatori per semplificare il debug di software complessi. Possono consentire di ottenere privilegi speciali o saltare processi di autenticazione durante lo sviluppo.

Backdoor malevole sfruttabili per ottenere accesso non autorizzato ai sistemi.

Sicurezza e Prevenzione

Implementare misure di sicurezza contro le backdoor richiede attenzione durante le fasi di sviluppo e aggiornamento del software. Alcuni approcci includono:

- **Revisione del codice:** Analizzare il codice sorgente per identificare eventuali backdoor non autorizzate.
- **Test di penetrazione:** Condurre test di sicurezza per simulare attacchi e identificare vulnerabilità.
- **Monitoraggio delle connessioni di rete:** Tenere traccia delle connessioni in ingresso e uscita per rilevare attività sospette.
- **Utilizzo di strumenti di rilevamento delle anomalie:** Implementare soluzioni di sicurezza che possono identificare comportamenti anomali o non autorizzati nel sistema.

7.10.2 Rootkit

Un rootkit è un insieme di strumenti progettati per ottenere e mantenere un accesso segreto a un sistema informatico, spesso con privilegi di amministratore (o "root"). Questi programmi nascondono la loro presenza e possono modificare il funzionamento del sistema operativo in modo da sfuggire alla rilevazione.

Funzionalità e Obiettivi

Il principale obiettivo di un rootkit è fornire all'attaccante il controllo completo sul sistema, consentendo azioni come:

- Modifica o aggiunta di programmi e file: Un rootkit può installare ulteriori malware o alterare file esistenti.

- Monitoraggio dei processi: Può tenere traccia delle attività del sistema, compreso il monitoraggio delle comunicazioni di rete.
- Accesso da backdoor: Permette di accedere al sistema anche dopo un riavvio o una disconnessione.

Come Nasconde la Propria Presenza

I rootkit utilizzano diverse tecniche per nascondersi, rendendo difficile la loro identificazione:

1. **Sovversione dei meccanismi di monitoraggio:** Modificano i sistemi di registrazione, monitoraggio e reporting di file e processi per non mostrare la loro presenza.
2. **Alterazione dei risultati delle chiamate API:** Possono intercettare le chiamate delle API per mascherare le informazioni su processi e file associati.

Classificazione dei Rootkit

I rootkit possono essere classificati in base a diverse caratteristiche:

Persistente Si attiva ad ogni avvio del sistema, memorizzando il codice in un archivio persistente (come il registro di sistema). Questi sono più facili da rilevare, ma possono essere progettati per rimanere nascosti.

Residente in memoria Non ha codice persistente e quindi non sopravvive a un riavvio. È più difficile da rilevare perché risiede solo nella memoria.

Modalità utente Intercetta le chiamate alle API e altera i risultati, come escludere i file di rootkit dalle liste dei file.

Modalità kernel Intercetta le chiamate alle API di sistema a livello di kernel, nascondendo processi e file a livello più profondo.

Basato su macchina virtuale Installa un monitor di macchina virtuale che esegue il sistema operativo in un ambiente virtuale. Il rootkit può quindi modificare gli stati del sistema senza essere rilevato.

Modalità esterna Risiede al di fuori del sistema operativo, in modalità BIOS o di gestione del sistema, avendo accesso diretto all'hardware.

8 - Attacchi Denial-of-Service

8.1 Introduzione

Un attacco Denial-of-Service (DoS) è un tentativo di compromettere la disponibilità di un servizio ostacolandone o bloccandone completamente la fornitura.

L'attacco tenta di esaurire alcune risorse critiche associate al servizio. Un esempio è inviare ad un server web un numero elevato di richieste tale da impedirgli di rispondere tempestivamente alle richieste valide degli utenti.

8.1.1 Gli Attacchi DDoS

Gli attacchi DDoS (Distributed Denial-of-Service) raggiungono l'efficacia sfruttando come fonti di attacco più sistemi informatici compromessi. Queste reti sono costituite da computer e altre macchine che vengono infettati da malware che ne permette il controllo a distanza da parte di un attaccante. Questi singoli dispositivi sono noti come bot.

8.2 TCP SYN Flood

Un attacco SYN flood è un tipo di attacco Denial-of-Service che mira a rendere un server indisponibile al traffico legittimo consumandone tutte le risorse. Inviando ripetutamente pacchetti di richieste di connessione iniziale SYN, l'attaccante è in grado di sopraffare tutte le porte disponibili di una macchina server, obbligando così la vittima designata a rispondere lentamente, o non rispondere affatto, al traffico legittimo.

8.2.1 Funzionamento del TCP SYN Flood

Funzionamento Base di una Connessione TCP

Gli attacchi SYN flood sfruttano il processo di handshake di una connessione TCP. In condizioni normali, per stabilire una connessione TCP sono necessari tre processi distinti:

1. Il client invia un pacchetto SYN al server per avviare la connessione.
2. Il server risponde con un pacchetto SYN/ACK attraverso il quale conferma esplicitamente la comunicazione.
3. Infine, il client restituisce un pacchetto ACK per confermare a sua volta la ricezione del pacchetto dal server. Dopo aver completato questa sequenza di invio e ricezione di pacchetti, la connessione TCP è aperta e in grado di inviare e ricevere dati.

Provocare un Denial-of-Service

Per provocare un Denial-of-Service, l'attaccante sfrutta il fatto che dopo aver ricevuto un pacchetto SYN iniziale, il server designato risponderà con uno o più pacchetti SYN/ACK e attenderà la fase finale dell'handshake.

Quando un server lascia una connessione aperta ma la macchina dall'altra parte della connessione non lo è, la connessione viene considerata semi-aperta.

8.2.2 Come si Verifica un TCP SYN Flood

1. L'attaccante invia un elevato volume di pacchetti SYN al server vittima, spesso con indirizzi IP spoofati.

2. Il server risponde a ciascuna delle richieste di connessione e lascia una porta aperta pronta per ricevere la risposta.

3. Mentre il server attende il pacchetto ACK finale (che non arriverà mai) l'attaccante continua a inviare altri pacchetti SYN. La ricezione di ogni nuovo pacchetto SYN costringe il server a mantenere temporaneamente una connessione con una nuova porta aperta per un certo periodo di tempo. Così, una volta utilizzate tutte le porte disponibili, il server non sarà più in grado di funzionare normalmente.

8.2.3 Tipologie di Attacco SYN Flood

1. **Attacco diretto:** in cui l'indirizzo IP dell'attaccante non è spoofato. In questo genere di attacco, l'attaccante è altamente vulnerabile alla scoperta e alla mitigazione. Questo metodo viene usato raramente, poiché la mitigazione è abbastanza semplice: basta bloccare l'indirizzo IP di qualsiasi sistema antagonista. Se l'aggressore invece usa una botnet non gli importerà di mascherare l'IP del dispositivo infetto.

2. **Attacco di spoofing:** l'attaccante può anche spoofare l'indirizzo IP in ciascun pacchetto SYN inviato per rendere più difficile la propria identificazione. Pur potendo essere spoofati, tali pacchetti possono essere eventualmente ricondotti alla loro origine. Svolgere questo tipo di indagine è difficile ma non impossibile, soprattutto se i provider di servizi Internet (ISP) sono disposti a collaborare.

3. **Attacco distribuito (DDoS):** se un attacco viene creato utilizzando una botnet, la probabilità di rintracciare l'origine dell'attacco è scarsa. Per aumentare il livello di camuffamento, l'aggressore può disporre che ogni dispositivo distribuito falsifichi anche gli indirizzi IP da cui invia i pacchetti.

8.2.4 Mitigazione degli Attacchi SYN Flood

Cloudflare mitiga in parte questo tipo di attacco posizionandosi tra il server aggredito e il SYN flood. Quando viene effettuata la richiesta SYN iniziale, Cloudflare gestisce il processo di handshake nel cloud, trattenendo la connessione con il server preso di mira fino al completamento dell'handshake TCP. Questa strategia sposta il consumo di risorse sulla rete di Cloudflare.

8.3 ICMP Flood

Un ICMP flood (o ping flood) è un attacco Denial-of-Service in cui l'attaccante tenta di sommergere un dispositivo target con pacchetti ICMP echo-request, rendendo il bersaglio inaccessibile al traffico normale.

8.3.1 Funzionamento dell'ICMP Flood

L'Internet Control Message Protocol (ICMP) è un protocollo utilizzato dai dispositivi di rete per comunicare. Gli strumenti di diagnostica di rete traceroute e ping funzionano entrambi con ICMP. In genere, questi messaggi vengono utilizzati per diagnosticare lo stato della connettività del dispositivo.

8.3.2 Provocare un ICMP Flood

L'attacco in questione mira a sopraffare la capacità del dispositivo bersaglio di rispondere all'elevato numero di richieste e a saturarne la banda.

La forma DDoS di un ICMP Flood può essere suddivisa in 2 fasi ripetute:

1. L'attaccante invia molti pacchetti ICMP al server bersaglio utilizzando più dispositivi.
2. Il server bersaglio invia quindi un pacchetto di risposta ICMP all'indirizzo IP di ciascun dispositivo richiedente come risposta.

8.3.3 Mitigare l'ICMP Flood

Per mitigare un ping flood è più facile disabilitare la funzionalità ICMP del dispositivo vittima. Un amministratore di rete può accedere all'interfaccia amministrativa del dispositivo e disabilitare la sua capacità di inviare e ricevere richieste utilizzando ICMP, eliminando di fatto sia l'elaborazione della richiesta che la risposta.

8.4 UDP Flood

Un UDP flood è un tipo di attacco Denial-of-Service in cui un gran numero di pacchetti User Datagram Protocol viene inviato a un server vittima con l'obiettivo di sovraccaricare la capacità del dispositivo di elaborare e rispondere. Anche il firewall che protegge il server preso di mira può esaurirsi a causa del flooding UDP.

8.4.1 Funzionamento dell'UDP Flood

Un flood UDP funziona principalmente sfruttando le fasi che un server compie quando risponde a un pacchetto UDP inviato a una delle sue porte. Quando un server riceve un pacchetto UDP su una determinata porta, esegue due fasi di risposta:

1. Il server controlla se ci sono programmi in esecuzione che sono attualmente in ascolto di richieste sulla porta specificata.

2. Se nessun programma riceve pacchetti su quella porta, il server risponde con un pacchetto ICMP (ping) per informare il mittente che la destinazione non è raggiungibile.

8.4.2 Parallelismo dell'UDP Flood

Si può pensare a un flood UDP nel contesto di una receptionist d'albergo che smista le chiamate. In primo luogo, l'addetto alla reception riceve una telefonata in cui il chiamante chiede di essere collegato a una stanza specifica. L'addetto alla reception deve quindi scorrere l'elenco di tutte le camere per assicurarsi che l'ospite sia disponibile in quella stanza e che sia disposto a rispondere alla chiamata. Quando l'addetto alla reception si accorge che l'ospite non risponde alle chiamate, deve riprendere il telefono e dire al chiamante che l'ospite non risponderà alla chiamata. Se all'improvviso tutte le linee telefoniche si accendono contemporaneamente con richieste simili, l'addetto alla reception sarà rapidamente sopraffatto.

8.4.3 Mitigare l'UDP Flood

La maggior parte dei sistemi operativi limita la velocità di risposta dei pacchetti ICMP, in parte per interrompere gli attacchi DDoS che richiedono una risposta ICMP. Un inconveniente di questo tipo di mitigazione è che durante un attacco possono essere filtrati anche pacchetti legittimi. Se il flood UDP ha un volume sufficiente a saturare la tabella di stato del firewall del server preso di mira, qualsiasi mitigazione a livello di server sarà insufficiente, poiché il collo di bottiglia si verificherà a monte del dispositivo preso di mira.

8.5 HTTP Flood

Un attacco HTTP flood è un tipo di attacco Denial-of-Service progettato per sommergere un server mirato con richieste HTTP. Una volta che il bersaglio è stato saturato dalle richieste e non è in grado di rispondere al traffico normale, si verificherà un denial-of-service per ulteriori richieste da parte degli utenti effettivi.

8.5.1 Funzionamento dell'HTTP Flood

Gli attacchi HTTP flood sono un tipo di attacco DDoS di livello 7. Il livello 7 è il livello applicativo del modello OSI e si riferisce ai protocolli Internet come l'HTTP, protocollo alla base delle richieste Internet basate su browser. La mitigazione degli attacchi di livello applicativo è particolarmente complessa, poiché il traffico dannoso è difficile da distinguere dal traffico normale.

Per ottenere la massima efficienza, i malintenzionati di solito utilizzano o creano botnet per massimizzare l'impatto del loro attacco.

8.5.2 Tipologie di Attacco HTTP Flood

HTTP GET In questa forma di attacco, più computer o altri dispositivi vengono coordinati per inviare richieste multiple di immagini, file o altre risorse da un server mirato.

HTTP POST In genere quando viene inviato un modulo su un sito Web, il server deve gestire la richiesta in arrivo e inviare i dati ad un database. La gestione dei dati del modulo e l'esecuzione dei comandi necessari per il database sono processi relativamente intensivi rispetto alla potenza necessaria per inviare la richiesta POST. Questo attacco sfrutta la disparità nel consumo di risorse relative, inviando molte richieste POST.

8.5.3 Mitigare un HTTP Flood

La mitigazione degli attacchi di livello 7 è piuttosto complessa. Un metodo consiste nell'implementare una challenge al client per verificare se si tratta o meno di un bot.

8.6 DNS Amplification Attack

8.6.1 Reflection Attack

Il reflection attack è un tipo di attacco Denial-of-Service che sfrutta la tecnica di IP spoofing e il protocollo TCP o UDP. Il server risponde alla richiesta inviando una risposta all'indirizzo IP della vittima. Qualsiasi server che opera su servizi basati su UDP o TCP può essere sfruttato come "riflettore".

8.6.2 Amplification Attack

L'amplification attack genera un alto volume di pacchetti con lo scopo di sovraccaricare il sito web bersaglio senza allertare il servizio intermediario. Questo accade quando un servizio vulnerabile risponde con una risposta di grandi dimensioni a fronte di una richiesta dell'attaccante, spesso chiamata trigger packet. L'attaccante può inviare migliaia di queste richieste a servizi vulnerabili, causando risposte che sono notevolmente più grandi rispetto alla richiesta originale e amplificano significativamente il traffico e la larghezza di banda diretti verso il bersaglio.

8.6.3 DNS Amplification Attack

Il DNS amplification attack sfrutta i server DNS per amplificare la quantità di traffico inviato verso il sistema bersaglio. L'attacco si basa sulla capacità del protocollo DNS di trasformare una piccola richiesta in una risposta molto più grande, con l'obiettivo di sovraccaricare il target con traffico eccessivo.

Funzionamento del DNS Amplification Attack

Un DNS amplification attack può essere suddiviso in quattro fasi:

1. L'attaccante utilizza un endpoint compromesso per inviare pacchetti UDP con indirizzi IP spoofati a un server DNS.

2. Ciascuno dei pacchetti UDP effettua una richiesta a un resolver DNS, spesso passando un argomento come ANY o record DNS complessi (ad esempio, indirizzi IPv6) per ricevere la risposta più grande possibile.

3. Dopo aver ricevuto le richieste, il resolver DNS, che rispondendo sta cercando di essere utile, invia una risposta di grandi dimensioni all'indirizzo IP spoofato.

4. L'indirizzo IP della vittima riceve la risposta e l'infrastruttura di rete circostante viene sovraccaricata da una marea di traffico che provoca una negazione del servizio.

Sebbene poche richieste non siano sufficienti ad abbattere l'infrastruttura di rete, quando questa sequenza viene moltiplicata tra più richieste e resolver DNS, l'amplificazione dei dati ricevuti dal bersaglio può essere sostanziale.

8.6.4 Mitigare il DNS Amplification Attack

La difesa principale consiste nel bloccare lo spoofing degli indirizzi IP a livello di rete, tramite tecniche come Ingress Filtering, che verifica che il traffico in uscita da una rete provenga da indirizzi IP legittimi. Inoltre, limitare l'accesso ai server DNS restringendo l'uso della funzionalità di recursione ai soli utenti interni, può prevenire che i server DNS aperti vengano sfruttati per amplificare attacchi.

9 - Intrusion Detection System

9.1 Introduzione

Gli Intrusion Detection System (IDS) sono strumenti di sicurezza progettati per monitorare e analizzare eventi all'interno di un sistema, con l'obiettivo di identificare in tempo reale o quasi, i tentativi di accesso non autorizzati.

Un'intrusione di sicurezza è definita come qualsiasi evento in cui un intruso riesca, o tenti di riuscire, a ottenere accesso non autorizzato a un sistema o a una risorsa del sistema. Il rilevamento delle intrusioni, invece, si riferisce a un servizio che monitora tali eventi per fornire tempestivamente degli avvisi.

9.2 Struttura di un IDS

Un IDS è tipicamente strutturato in tre componenti principali:

Sensori Che raccolgono dati da varie parti del sistema come pacchetti di rete, file di log o chiamate di sistema. Queste informazioni vengono inoltrate agli analizzatori per un'ulteriore valutazione.

Analizzatori Che hanno il compito di determinare se si è verificata un'intrusione. Possono anche suggerire azioni da intraprendere in seguito all'intrusione.

Interfaccia utente La quale consente all'utente di visualizzare gli output del sistema e di controllare il comportamento dell'IDS.

9.3 Configurazioni di IDS

Gli IDS si classificano in base alla fonte e al tipo di dati analizzati:

Host-based IDS (HIDS) Monitora le caratteristiche di un singolo host e gli eventi che vi accadono.

Network-based IDS (NIDS) Analizza il traffico di rete su segmenti o dispositivi specifici, verificando protocolli di rete, di trasporto e applicativi per identificare attività anomale.

IDS distribuiti o ibridi Combinano informazioni provenienti da più sensori, sia host-based che network-based, in un unico analizzatore centrale, migliorando l'identificazione e la risposta a attività intrusiva.

9.4 Principi di Base di un IDS

Il principio fondamentale del rilevamento delle intrusioni è basato sull'assunto che il comportamento di un intruso differisca, in modo quantificabile, da quello di un utente legittimo. La progettazione di un IDS richiede un bilanciamento tra la capacità di rilevare intrusi e la necessità di minimizzare i falsi allarmi.

Un'interpretazione ampia dei comportamenti sospetti può generare falsi positivi, mentre una definizione troppo rigida può portare a non rilevare attacchi reali. L'obiettivo ideale è quindi quello di avere un alto tasso di rilevamento (rapporto tra attacchi rilevati e attacchi totali) e un basso tasso di falsi allarmi (rapporto tra falsi positivi e uso normale del sistema).

9.5 Requisiti di un IDS

Per essere efficace, un IDS dovrebbe soddisfare determinati requisiti:

- Funzionamento continuo con minima supervisione umana.
- Tolleranza ai guasti, ovvero la capacità di recuperare da crash o riavvii del sistema.
- Resistenza alla sovversione, ossia deve essere in grado di monitorare se stesso per rilevare eventuali modifiche da parte di un attaccante.
- Basso impatto sulle prestazioni del sistema su cui è in esecuzione.
- Configurabilità in base alle politiche di sicurezza del sistema monitorato.
- Adattabilità alle variazioni del comportamento del sistema e degli utenti nel tempo.
- Scalabilità per monitorare un numero elevato di host.
- Degradazione graduale del servizio, in cui l'impatto sul sistema è minimizzato in caso di malfunzionamento di alcuni componenti dell'IDS.
- Riconfigurabilità dinamica senza la necessità di riavviare il sistema.

9.6 Analisi di un IDS

Gli Intrusion Detection System utilizzano diversi approcci per analizzare i dati dei sensori al fine di rilevare intrusioni.

9.6.1 Approcci di Analisi

Rilevazione delle Anomalie

La rilevazione delle anomalie prevede la raccolta di dati sul comportamento degli utenti legittimi in un periodo di tempo specifico. I comportamenti osservati vengono confrontati con il modello costruito per stabilire, con un certo livello di sicurezza, se si tratta di un comportamento lecito o di un'intrusione. Tuttavia, la raccolta e l'analisi dei dati necessari per creare un modello accurato possono essere complesse e generare un alto tasso di falsi allarmi.

Fasi del Processo

1. **Fase di addestramento:** si costruisce un modello del comportamento legittimo degli utenti, analizzando i dati raccolti dal sistema monitorato.
2. **Fase di rilevamento:** Il comportamento attuale viene confrontato con il modello precedentemente creato per classificare l'attività come lecita o anomala.

Tecniche di Classificazione Le principali tecniche di classificazione utilizzate sono:

- **Approcci statistici:** Analizzano il comportamento utilizzando modelli univariati, multivariati o di serie temporali.
- **Approcci basati sulla conoscenza:** Utilizzano un sistema esperto per classificare il comportamento osservato in base a un insieme di regole.
- **Approcci di machine learning:** Utilizzano tecniche di data mining per determinare automaticamente un modello di classificazione dai dati di addestramento.

Rilevazione tramite Firma o Euristica

Le tecniche basate su firma o euristica rilevano intrusioni osservando eventi nel sistema e applicando set di pattern di firma o regole euristiche. Questo approccio, noto anche come misuse detection, può identificare attacchi noti con efficienza, ma è inefficace contro attacchi zero-day.

Approcci Basati su Firma Questi metodi confrontano una vasta collezione di pattern di dati dannosi noti con i dati del sistema. Gli approcci basati su firma sono ampiamente utilizzati in prodotti antivirus e nel monitoraggio del traffico di rete. I vantaggi includono bassi costi in termini di tempo e risorse, ma richiedono un continuo aggiornamento delle firme per tenere traccia delle nuove minacce. Inoltre, non possono rilevare attacchi sconosciuti.

Approcci Basati su Regole Euristiche Le regole euristiche identificano penetrazioni note o attacchi che sfruttano vulnerabilità conosciute. Queste regole, spesso specifiche per macchine e sistemi operativi, vengono sviluppate analizzando strumenti e script d'attacco presenti online. Anche se efficaci, richiedono esperti di sicurezza per la creazione di regole accurate. Un esempio di IDS basato su regole è il sistema SNORT, che dispone di una vasta collezione di regole per il rilevamento di attacchi di rete.

9.7 Host-Based Intrusion Detection System

Tradizionalmente, i sistemi HIDS operavano in modalità autonoma su singoli sistemi. Tuttavia la difesa di una rete distribuita di host richiede un approccio coordinato che preveda la cooperazione tra diversi HIDS. Questo approccio distribuito è particolarmente utile per monitorare reti estese e rilevare minacce che potrebbero sfuggire a un singolo sistema.

9.7.1 Architetture Centralizzate vs Decentralizzate

Un HIDS distribuito può avere un'architettura centralizzata o decentralizzata:

Centralizzata Un unico punto centrale raccoglie e analizza tutti i dati dei sensori. Questo semplifica la correlazione degli eventi, ma introduce un punto di congestione e vulnerabilità.

Decentralizzata Esistono più centri di analisi che devono coordinarsi e scambiarsi informazioni. Questo modello riduce i rischi di un singolo punto di fallimento ma richiede un'efficace sincronizzazione tra i vari nodi.

9.7.2 Architettura di un HIDS Distribuito

L'architettura di un HIDS distribuito può essere suddivisa in tre componenti principali:

1. **Agente host:** Un processo in background che raccoglie dati sugli eventi di sicurezza nel sistema monitorato. Questo modulo invia i dati raccolti al gestore centrale per l'analisi.

2. **Agente LAN Monitor:** Funziona come l'agente host, ma si concentra sull'analisi del traffico di rete LAN, riportando i risultati al gestore centrale.

3. **Gestore Centrale:** Riceve i report dagli agenti host e LAN e li elabora per rilevare eventuali intrusioni. Il gestore centrale dispone di un sistema esperto per trarre inferenze dai dati ricevuti e può interrogare i sistemi per ottenere copie dei dati audit degli host (HAR).

Questa architettura è progettata per essere indipendente dal sistema operativo e dalle implementazioni di auditing, garantendo una flessibilità nella configurazione e gestione.

9.7.3 Processo di Rilevazione degli Eventi Sospetti

Il modulo agente host segue un processo specifico per rilevare attività sospette:

1. **Cattura dei Dati di Audit:** L'agente cattura ogni record di audit prodotto dal sistema di raccolta nativo e applica un filtro che conserva solo i record rilevanti per la sicurezza.

2. **Formattazione Standard:** I record vengono riformattati in un formato standard, il "host audit record" (HAR).

3. **Analisi Basata su Template:** Utilizzando un modulo di logica basato su template, i record vengono analizzati per rilevare attività sospette a diversi livelli:

   - **Eventi Isolati:** L'agente rileva eventi singoli di interesse, come accessi falliti ai file o modifiche nei permessi di accesso.
   - **Sequenze di Eventi:** L'agente riconosce sequenze di eventi che corrispondono a pattern di attacco noti.

- **Comportamento Anomalo:** Confronta il comportamento attuale di un utente con il suo profilo storico, basato su metriche come il numero di programmi eseguiti o file accessibili.

Quando viene rilevata un'attività sospetta, viene inviata un'allerta al gestore centrale, che può esaminare i dati raccolti per correlare eventuali anomalie.

9.7.4 Monitoraggio della LAN

L'agente monitor della LAN contribuisce fornendo informazioni sulla connessione tra host, i servizi utilizzati e il volume di traffico. Monitora eventi significativi, come improvvisi cambiamenti nel carico di rete o utilizzi sospetti di servizi, e segnala tali attività al gestore centrale.

9.8 Network-Based Intrusion Detection System

I Network-based Intrusion Detection System (NIDS) monitorano il traffico di rete per rilevare attività sospette analizzando i pacchetti che attraversano la rete.

9.9 IDS Distribuito

Un IDS distribuito centralizza le informazioni provenienti da fonti HIDS e NIDS, combinando così dati a livello di host con dati di eventi di rete.

9.9.1 Problemi Chiave per i Sistemi IDS

Due sono i problemi principali che affliggono gli IDS:

1. **Riconoscimento di nuove minacce:** I sistemi di sicurezza tradizionali spesso non riescono a identificare nuove minacce o varianti radicalmente modificate di minacce esistenti.

2. **Rapidità di aggiornamento:** È difficile aggiornare rapidamente i sistemi di sicurezza per rispondere ad attacchi che si diffondono in modo fulmineo.

Inoltre, la difesa perimetrale, come i firewall, incontra sfide aggiuntive in un contesto in cui le aziende moderne non hanno confini ben definiti e i dispositivi possono entrare e uscire dalla rete, per esempio tramite connessioni wireless o dispositivi mobili.

9.9.2 Attacchi e Metodi di Intrusione Moderni

Gli attaccanti adottano tattiche che sfruttano le vulnerabilità dei sistemi. I metodi più tradizionali prevedono la diffusione di malware e attacchi DDoS (Distributed Denial of Service) per colpire con forza e rapidità, sfruttando la lentezza delle risposte difensive. Tuttavia, negli ultimi anni, sono emerse tattiche più sofisticate, come attacchi che si diffondono lentamente e che sono progettati per eludere il rilevamento dei sistemi di difesa convenzionali.

9.9.3 Sistemi Cooperativi e Rilevamento Distribuito

Per contrastare gli attacchi moderni, sono stati sviluppati sistemi di rilevamento cooperativi. In questi sistemi, i nodi della rete agiscono come sensori locali e utilizzano protocolli peer-to-peer per scambiarsi informazioni sugli attacchi sospetti. Ad esempio, un sistema può rilevare un comportamento insolito in un nodo locale e, invece di reagire immediatamente con il rischio di un falso positivo, può segnalare l'attività sospetta ai nodi vicini. Se la quantità di messaggi di sospetto raggiunge una soglia predefinita, il sistema assume che un attacco sia in corso e attiva una risposta.

Un esempio concreto di questo approccio è Autonomic Enterprise Security, sviluppato da Intel. Questo sistema distribuito non si basa esclusivamente sulla difesa perimetrale (come i firewall) né su difese host-based individuali, ma considera ogni dispositivo di rete come un potenziale sensore. In questo modo, i sensori distribuiti collaborano per verificare lo stato della rete e determinare se è in corso un attacco.

9.9.4 Vantaggi del Rilevamento Distribuito

L'approccio distribuito presenta vari vantaggi:

1. **Maggiore copertura e risposta più rapida:** IDS multipli che condividono informazioni possono rilevare attacchi diffusi in modo più efficace, specialmente quando si tratta di attacchi che si sviluppano lentamente.

2. **Riduzione del traffico e miglioramento del rapporto segnale/rumore:** Il monitoraggio a livello host consente di analizzare il traffico locale, spesso ridotto rispetto al traffico globale analizzato da dispositivi di rete come i router, aumentando la visibilità dei modelli di attacco.

3. **Utilizzo di dati specifici dell'host:** I rilevatori host-based possono usare un set di dati più ricco (ad esempio, dati delle applicazioni) come input per i classificatori locali.

9.9.5 Implementazione del Sistema IDS Ibrido

Un IDS distribuito o ibrido può essere costruito con prodotti di un singolo fornitore, progettati per condividere e scambiare dati tra loro, o utilizzando software specializzato di Security Information and Event Management (SIEM), che consente di importare e analizzare dati provenienti da varie fonti. Un approccio centralizzato semplifica il coordinamento dei sensori, ma può non essere la soluzione più economica o completa.

9.9.6 Comportamento del Sistema Centrale e Sensori Distribuiti

Il sistema centrale è configurato con un set di politiche di sicurezza predefinite, che vengono adattate sulla base dei dati provenienti dai sensori distribuiti. Le politiche possono includere azioni immediate o parametri da regolare e istruzioni di collaborazione tra i dispositivi. I principali input del sistema centrale sono:

1. **Eventi di Riepilogo:** Eventi raccolti da vari punti della rete (come firewall e IDS) e sintetizzati per il sistema centrale.
2. **Eventi DDI (Distributed Detection and Inference):** Allarmi generati quando il traffico di "gossip" permette ai nodi di concludere che è in corso un attacco.
3. **Eventi PEP (Policy Enforcement Point):** Punti di applicazione delle politiche che risiedono su piattaforme fidate, capaci di correlare informazioni distribuite e decisioni locali per rilevare intrusioni.

9.10 Honeypot

Gli honeypot sono sistemi esca progettati per attirare gli attaccanti e distoglierli dai sistemi critici, raccogliendo al contempo informazioni utili sul loro comportamento.

9.10.1 Caratteristiche degli Honeypot

Gli honeypot sono caratterizzati da dati fittizi che sembrano preziosi ma che un utente legittimo non accederebbe mai, rendendo ogni tentativo di interazione sospetto. La loro funzione principale è di apparire vulnerabili per consentire agli amministratori di rilevare e tracciare gli attacchi senza mettere a rischio i sistemi produttivi.

9.10.2 Obiettivi degli Honeypot

Gli honeypot sono progettati per:

- Deviare gli attacchi dai sistemi critici: attraggono l'attenzione degli aggressori verso risorse non produttive.
- Raccogliere dati sugli attaccanti: registrano le attività malevole per analisi e miglioramento delle difese.
- Permettere una risposta tempestiva: lasciano all'amministratore il tempo necessario per reagire e intervenire.

9.10.3 Classificazione degli Honeypot

Gli honeypot si classificano in due categorie principali in base al livello di interazione con l'attaccante.

Honeypot a Bassa Interazione

Gli honeypot a bassa interazione rappresentano una soluzione di sicurezza che emula servizi o sistemi IT in modo limitato, offrendo un'interazione iniziale realistica senza implementare una versione completa del sistema. Questi strumenti sono progettati per identificare le prime fasi di un attacco con uno sforzo contenuto, sebbene non siano in grado di mantenere coinvolto l'attaccante per lunghi periodi.

Un esempio tipico è la simulazione di servizi di rete come FTP, SSH o web server, dove l'interazione è minima e spesso limitata a risposte predefinite.

Vantaggi e Svantaggi **Vantaggi:**

- Facili da implementare, richiedendo una configurazione minima e risorse contenute
- Offrono un elevato livello di sicurezza grazie alla loro gamma ristretta di interazione
- Necessitano di poca manutenzione e aggiornamenti

Svantaggi:

- Forniscono informazioni meno dettagliate sugli attacchi
- Possono essere più facilmente identificabili come honeypot da parte di attaccanti esperti

Honeypot ad Alta Interazione

Gli honeypot ad alta interazione costituiscono sistemi completi dotati di un sistema operativo reale, servizi e applicazioni pienamente accessibili agli attaccanti. Questi sistemi più sofisticati offrono un'esperienza molto più realistica e sono in grado di trattenere l'attaccante per periodi più lunghi, permettendo di raccogliere informazioni più approfondite.

Un esempio concreto è rappresentato da un honeypot che fornisce un vero sistema operativo, permettendo all'attaccante di eseguire comandi, caricare malware e condurre exploit complessi.

Vantaggi e Svantaggi **Vantaggi:**

- Consentono una ricca raccolta di informazioni registrando ogni azione dell'attaccante
- Offrono un elevato grado di realismo che rende più difficile il loro riconoscimento
- Permettono lo studio approfondito di exploit sofisticati

Svantaggi:

- Presentano un maggiore rischio di essere sfruttati per compromettere altre parti della rete
- Richiedono una configurazione complessa e risorse significative
- Necessitano di un monitoraggio continuo e una gestione della sicurezza impegnativa

9.10.4 Evoluzione degli Honeypot

Inizialmente, gli honeypot consistevano in singoli computer con indirizzi IP progettati per attrarre hacker. Con il tempo, la ricerca si è concentrata sulla creazione di reti di honeypot che simulano intere infrastrutture aziendali con traffico e dati, consentendo un'osservazione più dettagliata e completa degli attacchi.

9.10.5 Posizionamento degli Honeypot nella Rete

La posizione degli honeypot all'interno della rete è un fattore cruciale per il loro funzionamento e la raccolta di dati. Diverse opzioni di distribuzione possono offrire vantaggi e svantaggi a seconda dell'uso desiderato.

Honeypot all'Esterno del Firewall

Un honeypot collocato all'esterno del firewall monitora i tentativi di connessione a indirizzi IP inutilizzati, riducendo i rischi per la rete interna. Sebbene sia efficace nel catturare gli attacchi esterni, ha un'efficacia limitata nell'identificazione di attacchi interni.

Honeypot nella DMZ

Un honeypot nella DMZ monitora l'accesso ai servizi esterni, come web e posta elettronica, ma richiede una sicurezza rigorosa per evitare che le attività generate compromettano gli altri sistemi della DMZ. L'efficacia può essere limitata dai filtri del firewall, che deve garantire un equilibrio tra sicurezza e monitoraggio.

Honeypot Interno

Un honeypot all'interno della rete aziendale è utile per rilevare attacchi interni e potenziali falle nei firewall. Tuttavia, richiede una configurazione attenta del firewall per evitare complicazioni nella gestione del traffico e ridurre il rischio di compromissioni.

10 - Firewall

10.1 Introduzione

I firewall si basano sulla semplice idea che il traffico di rete proveniente da ambienti meno sicuri debba essere autenticato e ispezionato prima di passare a un ambiente più sicuro. Ciò impedisce a utenti, dispositivi e applicazioni non autorizzati di entrare in un ambiente o segmento di rete protetto. Senza firewall, computer e dispositivi nella rete sono soggetti ad attacchi e rendono l'utente vulnerabile.

10.2 Requisiti Minimi

Un firewall deve:

1. Garantire che tutto il traffico (sia in entrata che in uscita) passi attraverso esso, che agisce come unico punto di accesso alla rete locale. Questo obiettivo è raggiunto bloccando fisicamente ogni altro accesso alla rete.

2. Permettere solo il traffico autorizzato, conforme alle policy di sicurezza definite dall'organizzazione.

3. Essere immune a tentativi di penetrazione.

10.3 Politiche d'Accesso

Le politiche d'accesso sono basate su una serie di caratteristiche di filtraggio, come l'accesso in base a indirizzi IP, valori di protocollo, direzione del traffico (in entrata o uscita), o altre proprietà a livello di rete e trasporto. Un'altra modalità di filtraggio è basata sulle attività di rete, come limitare le richieste a specifiche ore o identificare tentativi di scansione.

10.4 Caratteristiche di un Firewall

I firewall possono monitorare il traffico di rete a diversi livelli, dai pacchetti di rete a basso livello fino ai dettagli dei protocolli applicativi. Il livello di monitoraggio dipende dalla policy di accesso implementata.

10.5 Filtri del Firewall

Un firewall può agire come filtro positivo, consentendo solo i pacchetti che rispettano determinati criteri, o come filtro negativo, bloccando quelli che li soddisfano. Le policy di accesso si basano sulle informazioni contenute nei pacchetti, come gli header di protocollo o i modelli generati da una sequenza di pacchetti.

10.5.1 Firewall a Filtraggio di Pacchetto

Il firewall a filtraggio di pacchetto applica una serie di regole a ciascun pacchetto IP in entrata e in uscita e decide se inoltrarlo o scartarlo. Le regole di filtraggio si basano su diverse informazioni contenute nel pacchetto, come l'indirizzo IP di origine, l'indirizzo IP di destinazione, i numeri di porta di livello trasporto (come TCP o UDP), il campo di protocollo IP e l'interfaccia di rete attraverso la quale il pacchetto è passato.

Esistono due politiche predefinite:

Default discard Più conservativa e generalmente preferita da aziende, in cui tutto ciò che non è espressamente permesso è bloccato.

Default forward Più aperta ma meno sicura, in cui tutto ciò che non è espressamente proibito è permesso.

10.5.2 Tipi di Attacchi al Firewall a Filtraggio di Pacchetto

I packet filtering firewall possono essere soggetti a diversi tipi di attacchi, ognuno dei quali richiede contromisure specifiche per essere prevenuto o mitigato.

IP Spoofing

Un tipo di attacco è l'IP spoofing, in cui l'attaccante invia pacchetti dall'esterno utilizzando un indirizzo IP spoofato, fingendo che il pacchetto provenga da un host interno fidato. La contromisura principale è scartare qualsiasi pacchetto con un indirizzo di origine interno se questo arriva su un'interfaccia esterna.

Source Routing

Un altro attacco è quello del source routing, in cui l'attaccante specifica il percorso che un pacchetto deve seguire attraverso la rete, sperando di aggirare i controlli di sicurezza che non analizzano le informazioni di routing. La contromisura più efficace è scartare tutti i pacchetti che utilizzano l'opzione di source routing.

Tiny Fragment Attack

L'attacco a piccoli frammenti (tiny fragment attack) separa le informazioni dell'header TCP in frammenti diversi. L'attacco mira a bypassare le regole di filtraggio che dipendono dalle informazioni dell'header TCP, poiché molti filtri analizzano solo il primo frammento di un pacchetto. Per contrastare questo attacco, il firewall dovrebbe imporre una regola secondo cui il primo frammento deve contenere una quantità minima di informazioni dell'header di trasporto. Inoltre, se il primo frammento viene scartato, il firewall deve ricordare il pacchetto e scartare tutti i frammenti successivi.

10.5.3 Firewall a Filtraggio di Pacchetto con Stato

Il firewall a filtraggio di pacchetto con stato offre un livello di sicurezza superiore, poiché tengono conto del contesto delle connessioni e non si limitano a esaminare i pacchetti in modo isolato.

Le applicazioni basate su TCP, come SMTP, seguono il modello client/server. Per stabilire la comunicazione, si crea una connessione TCP tra il client e il server. Il server SMTP utilizza una porta TCP ben definita (porta 25), mentre il client SMTP utilizza una porta temporanea, solitamente compresa tra 1024 e 65535.

Il problema con un firewall a filtraggio di pacchetto è che, per consentire il traffico TCP in entrata, deve permettere il traffico su tutte queste porte ad alto numero (oltre 1024). Questo succede perché un firewall tradizionale non è in grado di distinguere quali connessioni siano legittime e quali siano tentativi di attacco.

Il firewall a filtraggio di pacchetto con stato gestisce questa vulnerabilità mantenendo un registro delle connessioni TCP in uscita. Questo registro contiene informazioni su tutte le connessioni attive, incluse le porte utilizzate dal client e dal server. Il firewall consente il traffico in entrata su porte ad alto numero solo se il pacchetto in arrivo corrisponde a una connessione già esistente nel registro.

10.6 Tipi di Firewall

10.6.1 Gateway a Livello Applicativo

Un gateway a livello applicativo (o proxy applicativo) funge da intermediario per il traffico di rete a livello applicativo. In pratica, un utente si connette al gateway utilizzando un'applicazione TCP/IP come Telnet o FTP, e il gateway chiede all'utente di specificare l'host remoto che desidera contattare. Una volta forniti un nome utente valido e le informazioni di autenticazione, il gateway contatta l'applicazione sull'host remoto e si occupa di inoltrare i segmenti TCP contenenti i dati dell'applicazione tra l'utente e l'host remoto.

I gateway a livello applicativo sono generalmente più sicuri rispetto ai filtri di pacchetti, poiché operano a un livello più alto e si concentrano solo su un numero limitato di applicazioni consentite, evitando di dover gestire tutte le possibili combinazioni di traffico TCP e IP. Un ulteriore vantaggio è la possibilità di registrare e controllare facilmente tutto il traffico in entrata a livello applicativo, migliorando la capacità di auditing e monitoraggio.

Tuttavia, uno svantaggio significativo di questo tipo di gateway è l'overhead di elaborazione aggiuntivo su ogni connessione. Poiché il gateway agisce come punto di "giunzione" tra due connessioni distinte (una verso l'utente e l'altra verso l'host remoto), deve esaminare e inoltrare tutto il traffico in entrambe le direzioni, aumentando il carico computazionale.

10.6.2 Proxy a Livello di Circuito

Un proxy a livello di circuito (o circuit-level gateway) è un tipo di firewall che gestisce le connessioni TCP tra due host senza permettere una connessione TCP diretta end-to-end. Simile a un gateway a livello applicativo, il circuit-level gateway crea due connessioni TCP separate: una con l'host interno e una con l'host esterno. Una volta stabilite queste due connessioni, il gateway funge da intermediario, inoltrando i segmenti TCP tra le due connessioni senza esaminare il contenuto dei pacchetti.

La principale funzione di sicurezza di un circuit-level gateway è determinare quali connessioni saranno consentite, invece di analizzare ogni singolo pacchetto. Questo tipo di gateway è particolarmente utile in ambienti in cui l'amministratore

di sistema si fida degli utenti interni. Questa configurazione permette di ottenere un bilanciamento tra sicurezza e prestazioni, poiché limita l'analisi approfondita ai dati in entrata, dove il rischio è maggiore, e ottimizza le connessioni in uscita, riducendo il carico computazionale.

10.7 Classificazione per Locazione

Una ulteriore classificazione può essere operata sulla base della locazione del firewall, quindi si individuano:

Bastion host Un sistema estremamente protetto, che gestisce il minor numero di servizi possibili (tipicamente, solo i firewall).

Host-based firewall Firewall installati direttamente su un host al fine di evitare attacchi diretti, sono configurati con regole progettate appositamente per l'host considerato.

Personal firewall In questo caso ci si riferisce a software installati su PC o workstation, ideati tipicamente per soluzioni non professionali.

10.7.1 Bastion Host

Un bastion host effettua la funzione di interfaccia tra la rete interna e quella esterna e per questo è spesso soggetto di attacchi dall'esterno ma del resto la sua più classica configurazione d'uso è quella di primo ed unico punto di contatto tra privato e pubblico dominio.

I progettisti di reti posizionano il bastion host nella prima linea di difesa. Costituisce un punto nevralgico per tutte le comunicazioni fra la rete e Internet: nessun computer della rete può accedere a Internet senza passare attraverso il bastion host e viceversa. Se si concentra ogni accesso alla rete in un unico computer, può essere molto facile gestire la sicurezza della rete.

Un bastion (computer bastione) è un computer specializzato nell'isolare una rete locale da una connessione pubblica, creando uno scudo che permette di proteggere la rete locale.

Il posizionamento del bastion host (con finalità firewall) può prevedere diverse configurazioni, tra cui:

- Un singolo firewall tra la rete interna e quella esterna
- Creazione di una DMZ

pfSense come Bastion Host

Un esempio di bastion host e pfSense, sistema operativo basato su FreeBSD, spesso utilizzato come firewall e router. Può essere configurato come bastion host grazie alla sua robustezza e capacità di gestire vari servizi di rete, come VPN, proxy e filtraggio del traffico.

10.7.2 Host-Based Firewall

Esempi di host-based firewall:

iptables (Linux) È il firewall integrato nel kernel Linux, che consente di configurare regole specifiche per filtrare pacchetti in entrata e in uscita su singoli host. È utilizzato comunemente nei server per gestire la sicurezza a livello di host.

Windows Defender Firewall Integrato nei sistemi operativi Windows, consente di impostare regole per filtrare il traffico di rete su un singolo computer, permettendo di configurare restrizioni per applicazioni e servizi.

pf (OpenBSD, FreeBSD, macOS) Un firewall host-based su Unix-like OS, che consente di configurare policy specifiche per il filtraggio del traffico a livello di singolo sistema.

10.7.3 Personal Firewall

Esempi di personal firewall:

Norton Personal Firewall Parte della suite di sicurezza Norton, questo firewall personale protegge i computer domestici bloccando le connessioni non autorizzate e monitorando le attività sospette.

Little Snitch (macOS) Un firewall personale per macOS che avvisa l'utente ogni volta che un'applicazione tenta di connettersi a Internet, permettendo un controllo dettagliato sulle connessioni in uscita.

10.8 Architetture di Sicurezza di Rete

Un amministratore di sicurezza deve decidere la posizione e il numero di firewall necessari.

10.8.1 La Rete DMZ

Un'implementazione comune è quella di reti DMZ (Demilitarized Zone), dove una configurazione tipica prevede un firewall esterno collocato al margine di una rete locale o aziendale, appena all'interno del router di frontiera che collega la rete a Internet.

Un secondo firewall interno protegge la maggior parte della rete aziendale. Tra questi due firewall si trova una zona DMZ, un segmento di rete dedicato a dispositivi accessibili dall'esterno che richiedono protezione, come un sito web aziendale, un server di posta elettronica o un server DNS.

Il Ruolo dei Firewall

Il Firewall Esterno Il firewall esterno controlla e limita l'accesso ai sistemi nella DMZ, garantendo al contempo un primo livello di protezione per il resto della rete aziendale.

Il Firewall Interno I firewall interni svolgono tre funzioni principali:

1. Offrono un filtraggio più rigoroso rispetto al firewall esterno, proteggendo server e workstation aziendali da attacchi esterni.
2. Forniscono protezione bidirezionale riguardo alla DMZ:
   - Proteggono la rete interna da attacchi provenienti dai sistemi DMZ compromessi (a causa di worm, rootkit, bot o altro malware).
   - Difendono i sistemi DMZ da attacchi provenienti dalla rete interna.
3. È possibile utilizzare più firewall interni per proteggere diverse parti della rete interna l'una dall'altra, separando server interni dalle workstation o suddividendo diverse aree operative della rete per aumentare la sicurezza.

Questa configurazione garantisce una protezione stratificata, con il firewall esterno che gestisce il traffico proveniente dall'esterno e il firewall interno che offre una protezione più specifica e approfondita.

10.8.2 La Rete VPN

Una VPN (Virtual Private Network) consiste in un gruppo di computer collegati tramite una rete non sicura, che utilizzano protocolli speciali e crittografia per garantire la sicurezza. In ogni sito aziendale, server e database sono collegati tramite una o più LAN. Per interconnettere questi siti, si può utilizzare Internet o un'altra rete pubblica. Il problema principale che il gestore deve affrontare è la sicurezza.

L'uso di una rete pubblica espone il traffico a rischi di intercettazione e può offrire un punto di accesso per utenti non autorizzati.

Garantire la Sicurezza

La VPN, per risolvere questo problema, utilizza la crittografia e l'autenticazione negli strati inferiori del protocollo per garantire una connessione sicura. La crittografia può essere eseguita tramite software di firewall o, in alcuni casi, dai router stessi. Il protocollo più comune per questo scopo è a livello IP, ed è conosciuto come IPSec.

Implementazione di una VPN

Un metodo logico per implementare IPSec è all'interno di un firewall. Tuttavia, se IPSec viene implementato in un dispositivo separato, posto dietro il firewall, tutto il traffico VPN che attraversa il firewall in entrambe le direzioni sarà criptato. In questo scenario, il firewall non può eseguire le sue funzioni di filtraggio o altre funzioni di sicurezza.

10.8.3 Firewall Distribuiti

Una configurazione di firewall distribuiti coinvolge dispositivi firewall autonomi e firewall host-based che lavorano insieme sotto un controllo amministrativo centrale.

I firewall autonomi forniscono una protezione globale, inclusi firewall interni ed esterni. In un sistema di firewall distribuiti, potrebbe essere utile stabilire sia una

DMZ interna che una esterna. I server web che richiedono meno protezione, poiché contengono informazioni meno sensibili, possono essere collocati in una DMZ esterna, fuori dal firewall esterno. La protezione necessaria per questi server è garantita dai firewall basati su host.

Un aspetto fondamentale è il monitoraggio della sicurezza, che include l'aggregazione e l'analisi dei log, statistiche del firewall e il monitoraggio remoto dettagliato dei singoli host, se necessario.

10.8.4 Topologia delle Reti con Firewall

Firewall residente su host Include software firewall personali e firewall su server. Possono essere utilizzati da soli o come parte di una strategia di sicurezza approfondita.

Router di screening Un router singolo tra la rete interna ed esterna con filtraggio di pacchetti stateless o completo. Questo è tipico per piccole imprese o reti domestiche (SOHO).

Bastion singolo in-line Un dispositivo firewall tra un router interno ed esterno. Può implementare filtri stateful o proxy applicativi.

Bastion singolo a T Simile al bastion host in linea, ma con una terza interfaccia di rete per una DMZ dove si trovano server visibili esternamente.

Doppio bastion in-line Il DMZ è posizionato tra due firewall bastion. Questo tipo di configurazione è utilizzato da grandi aziende e enti governativi.

Doppio bastion host a T Simile alla configurazione precedente, ma la DMZ è su una interfaccia di rete separata del firewall bastion. Comune in grandi aziende e enti governativi.

Configurazione di firewall distribuiti Include firewall autonomi e basati su host, coordinati centralmente. Utilizzata in grandi organizzazioni e enti governativi.

10.9 Intrusion Prevention System

10.9.1 Introduzione

L'Intrusion Prevention System (IPS), anche noto come Intrusion Detection and Prevention System (IDPS), è un'estensione di un Intrusion Detection System (IDS) che include la capacità di bloccare o prevenire attività malevole rilevate. Mentre un IDS è progettato principalmente per rilevare intrusioni, l'IPS va oltre, intervenendo attivamente per fermarle. Può essere implementato a livello host-based, network-based o in una configurazione distribuita o ibrida.

10.9.2 Rilevamento degli Attacchi di un IPS

Può utilizzare tecniche di rilevamento anomalo per individuare comportamenti insoliti, non associati ad utenti legittimi, o tecniche basate su firme o euristiche per

identificare comportamenti malevoli già noti. Se un IDS rileva un'attività malevola, un IPS può rispondere, sfruttando algoritmi avanzati, tramite la modifica o il blocco di pacchetti di rete o chiamate di sistema. Immaginiamo quindi l'IPS come un'evoluzione del firewall.

10.9.3 Host-Based IPS (HIPS)

Un Host-Based Intrusion Prevention System (HIPS) è un tipo di IPS specifico per singoli host.

Rilevamento degli Attacchi

- Nel rilevamento per firma, il sistema si concentra sul contenuto del traffico di rete delle applicazioni o sulle sequenze di chiamate di sistema, cercando schemi noti come malevoli.
- Nel rilevamento di anomalie, invece, il sistema cerca comportamenti che indicano la presenza di malware.

Comportamenti Malevoli

Alcuni esempi di comportamenti malevoli che un HIPS può rilevare includono:

- Modifiche alle risorse di sistema: Operazioni da parte di rootkit, trojan o backdoor che alterano il sistema.
- Escalation dei privilegi: Attacchi che cercano di concedere privilegi di root a utenti normali.
- Esecuzione di attacchi buffer overflow: Tipologia di attacco che sfrutta vulnerabilità legate alla memoria.
- Accesso alla lista dei contatti e-mail: Molti worm si diffondono inviando copie di se stessi tramite la lista dei contatti della vittima.
- Directory traversal: Vulnerabilità che permettono a un attaccante di accedere a file esterni alla normale portata di un server.

Le Sandbox

Una tecnica avanzata utilizzata da un HIPS è l'uso di sandbox, utile per applet Java o linguaggi di scripting. La sandbox isola il codice sospetto, esegue il codice e ne monitora il comportamento. Se il codice viola le politiche predefinite o corrisponde a firme di comportamenti malevoli, l'esecuzione viene interrotta e il codice viene impedito di operare nell'ambiente normale del sistema.

10.9.4 Network-Based IPS (NIPS)

Il Network-Based Intrusion Prevention System (NIPS) è una versione inline del Network Intrusion Detection System (NIDS), ma con l'autorità di modificare o scartare pacchetti e interrompere connessioni TCP.

Funzionamento

Il NIPS può intervenire attivamente bloccando il traffico malevolo in tempo reale, tramite tecniche di rilevamento per firma/euristiche che di rilevamento di anomalie, simili a quelle impiegate da un NIDS, ma con funzionalità aggiuntive di prevenzione.

Una delle caratteristiche distintive del NIPS, rispetto ai firewall tradizionali, è la protezione del flusso di dati. Questa funzione richiede la ricostruzione del payload dell'applicazione in una sequenza di pacchetti, così da poter applicare filtri a ogni nuovo pacchetto ricevuto. Se un flusso viene identificato come malevolo, i pacchetti successivi appartenenti a quel flusso vengono immediatamente scartati.

Rilevamento degli Attacchi

Il NIPS utilizza diverse tecniche per identificare pacchetti malevoli, tra cui:

- **Pattern matching:** Scansiona i pacchetti in arrivo alla ricerca di sequenze di byte specifiche, note come firme.
- **Stateful matching:** Analizza le firme degli attacchi nel contesto di un flusso di traffico piuttosto che esaminare i singoli pacchetti isolati.
- **Protocol anomaly:** Identifica deviazioni dagli standard definiti nei documenti RFC, che specificano il comportamento atteso dei protocolli.
- **Traffic anomaly:** Analizza il traffico per identificare comportamenti anomali che possono indicare la presenza di un attacco.
- **Statistical anomaly:** Utilizza modelli statistici per rilevare deviazioni nei modelli di traffico standard, suggerendo potenziali minacce.

10.9.5 Sistema Immunitario Digitale

Il Sistema Immunitario Digitale rappresenta una difesa globale contro comportamenti malevoli causati da malware. Il sistema funziona raccogliendo dati da una vasta gamma di sensori IPS che inviano le informazioni a un sistema centrale di analisi. Questo sistema correlativo elabora i dati, identificando nuove minacce e comportamenti malevoli. Le informazioni raccolte vengono poi utilizzate per aggiornare firme e pattern comportamentali su tutti i sistemi coordinati, migliorando così la capacità di risposta collettiva agli attacchi.

Funzionamento del Sistema Immunitario Digitale

Quando un nuovo malware viene introdotto in un'organizzazione, il sistema immunitario digitale lo cattura, lo analizza, ne crea una firma di rilevamento e protezione, e infine lo rimuove. Le informazioni vengono poi distribuite ai client connessi, in modo che il malware possa essere identificato prima che possa diffondersi ulteriormente. Questo processo garantisce una risposta rapida e una protezione aggiornata e continua contro nuove varianti di malware.

11 - Buffer Overflow

11.1 Principi del Buffer Overflow

Un buffer overflow, noto anche come buffer overrun o buffer overwrite, è una condizione che si verifica quando un'area di memoria viene riempita con più dati di quelli che può contenere, sovrascrivendo così altre informazioni in memoria.

Questo fenomeno può essere sfruttato da attaccanti per mandare in crash un sistema o inserire codice malevolo, ottenendo così il controllo del sistema.

11.1.1 Cause e Conseguenze

Il buffer overflow è spesso il risultato di errori di programmazione, quando un processo tenta di memorizzare dati oltre i limiti di un buffer di dimensioni fisse. Questo può sovrascrivere celle di memoria adiacenti contenenti variabili del programma, parametri o dati di controllo del flusso, come indirizzi di ritorno e puntatori a stack frame precedenti. Il buffer può trovarsi in diverse aree di memoria, come lo stack, l'heap o la sezione dati del processo.

Le principali conseguenze di un buffer overflow includono:

Corruzione dei dati I dati utilizzati dal programma possono essere modificati involontariamente.

Alterazione del flusso di esecuzione L'overflow può deviare il flusso di controllo verso aree non previste.

Violazioni di accesso alla memoria L'accesso a celle di memoria non autorizzate può interrompere il programma.

Esecuzione di codice arbitrario In caso di attacco, l'overflow permette di eseguire codice scelto dall'attaccante con i privilegi del processo compromesso.

11.1.2 Requisiti per Sfruttare un Buffer Overflow

Per sfruttare un buffer overflow, un attaccante deve:

1. **Identificare una vulnerabilità:** il programma deve presentare una vulnerabilità di buffer overflow che può essere sfruttata attraverso dati esterni sotto il controllo dell'attaccante.

2. **Comprendere la struttura della memoria:** è essenziale capire come il buffer è memorizzato nella memoria del processo, consentendo all'attaccante di sovrascrivere le celle di memoria adiacenti e potenzialmente alterare il flusso del programma.

Identificazione delle Vulnerabilità

La ricerca di programmi vulnerabili può essere effettuata tramite l'analisi del codice sorgente, il monitoraggio dell'esecuzione del programma con input sovradimensionati o strumenti come il fuzzing. Quest'ultimo è un metodo per testare la sicurezza di un programma inviandogli grandi quantità di input casuali per individuare potenziali falle di sicurezza.

11.2 Shellcode

In molti attacchi di buffer overflow, il controllo del programma viene trasferito a codice appositamente fornito dall'attaccante, noto come shellcode, che ha come obiettivo l'esecuzione di una shell a riga di comando. Su sistemi UNIX, questo avviene attraverso una chiamata alla funzione di sistema `execve("/bin/sh")`, mentre su Windows si utilizza generalmente `system("command.exe")` o `cmd.exe`.

11.2.1 Caratteristiche dello Shellcode

Lo shellcode è costituito da codice macchina, ossia istruzioni binarie specifiche per l'architettura del processore e per il sistema operativo attaccato. Questa dipendenza dal sistema target rende gli attacchi di buffer overflow tipicamente mirati a un componente software specifico su una piattaforma definita. La scrittura di shellcode richiede conoscenza del linguaggio assembly e del funzionamento interno del sistema attaccato, anche se strumenti come il Metasploit Project hanno semplificato la generazione di shellcode e lo sviluppo di exploit, rendendolo accessibile a un numero più ampio di utenti.

11.2.2 Sviluppo dello Shellcode

Per comprendere la struttura base di uno shellcode, consideriamo un esempio classico su un sistema Linux Intel, in cui l'obiettivo è eseguire la shell. Lo shellcode deve configurare gli argomenti per la chiamata `execve`, specificando i parametri necessari e invocando la funzione. Poiché `execve` è una funzione di libreria, lo shellcode inserisce esplicitamente le istruzioni per configurare i registri e invocare l'interrupt software necessario per la chiamata al kernel.

11.2.3 Restrizioni

Nello sviluppo di shellcode, esistono alcune restrizioni tecniche:

Indipendenza dalla posizione Lo shellcode non deve contenere riferimenti a indirizzi assoluti, poiché l'attaccante non può prevedere l'esatta posizione del buffer nello stack del programma attaccato.

Assenza di valori NULL Il codice non può contenere valori NULL all'interno, eccetto alla fine, a causa delle routine di gestione delle stringhe in C che terminano le stringhe con un carattere NULL. Includere valori NULL interromperebbe la copia dello shellcode nel buffer.

11.3 Difese Contro i Buffer Overflow

Le difese contro i buffer overflow possono essere implementate sia a tempo di compilazione sia a livello di runtime.

11.3.1 Difese a Tempo di Compilazione

Le difese a tempo di compilazione mirano a rendere i programmi più robusti contro i buffer overflow direttamente durante il processo di compilazione.

Scelta del Linguaggio di Programmazione

Usare linguaggi di alto livello con controllo automatico sui limiti dei buffer riduce la vulnerabilità ai buffer overflow. Tuttavia, la sicurezza di questi linguaggi ha un costo in termini di prestazioni. Nonostante i vantaggi, l'accesso diretto a istruzioni e risorse hardware è limitato, per cui linguaggi come il C continuano a essere utilizzati, specialmente per codice che richiede controllo a basso livello.

Tecniche Sicure di Scrittura del Codice

Nei linguaggi non sicuri come il C, i programmatori devono seguire pratiche di programmazione sicura, come il controllo dello spazio disponibile prima di scrivere su un buffer. È essenziale una scrittura robusta del codice che prevenga il buffer overflow controllando sempre le dimensioni dei dati e dei buffer.

Estensioni del Linguaggio e Librerie Sicure

Per il C sono stati sviluppati compilatori che inseriscono automaticamente controlli sui limiti dei buffer. Le librerie sicure, come Libsafe, proteggono lo stack evitando che le operazioni di copia superino lo spazio disponibile, prevenendo così le modifiche agli indirizzi di ritorno e impedendo alcune forme di attacco.

Meccanismi di Protezione dello Stack

Questi metodi aggiungono controlli sullo stack per rilevare eventuali tentativi di sovrascrittura.

Stackguard Un'estensione del compilatore GCC, Stackguard inserisce un valore sentinella sotto l'indirizzo di ritorno nello stack. Se questo valore viene modificato, si interrompe il programma, evitando che l'attaccante possa alterare il flusso di esecuzione. Il canarino è generato in modo casuale, rendendo più difficile per l'attaccante prevederne il valore.

Stackshield e RAD (Return Address Defender) Questi strumenti, anch'essi estensioni di GCC, copiano l'indirizzo di ritorno in una regione sicura della memoria all'ingresso della funzione e lo confrontano con il valore nello stack all'uscita della funzione. Se l'indirizzo di ritorno è stato alterato, il programma viene interrotto. Questa tecnica non modifica la struttura dello stack frame e quindi è compatibile con i debugger tradizionali.

11.3.2 Difese a Tempo di Esecuzione

Le difese a tempo di esecuzione proteggono i sistemi contro i buffer overflow senza necessità di ricompilare i programmi esistenti, rendendole particolarmente utili per sistemi operativi aggiornati o per mitigare vulnerabilità già presenti nei programmi.

Protezione dello Spazio degli Indirizzi Eseguibili

Una difesa efficace consiste nel rendere non eseguibili lo stack, supportando questa protezione tramite l'unità di gestione della memoria del processore (MMU), che etichetta le pagine come non eseguibili. Questa tecnica è inclusa nei sistemi operativi moderni e previene l'esecuzione di codice sullo stack, anche se alcuni programmi (ad esempio i compilatori just-in-time) possono richiedere eccezioni per funzionare correttamente.

Randomizzazione dello Spazio degli Indirizzi

Modificando in modo casuale l'indirizzo dello stack per ogni esecuzione del processo, si riduce la probabilità di successo di un attacco di stack overflow, poiché l'attaccante non può prevedere esattamente l'indirizzo di destinazione.

La randomizzazione può essere estesa anche al caricamento delle librerie standard, modificando casualmente l'ordine e gli indirizzi di memoria delle funzioni di libreria, rendendo impraticabili attacchi basati sul riutilizzo di codice di libreria.

Pagine di Guardia

Un'ulteriore protezione consiste nell'inserimento di pagine di guardia (pagine vuote e non accessibili) tra le aree di memoria critiche, come gli stack frame o le allocazioni sull'heap. Queste pagine, segnalate come illegali dalla MMU, interrompono il processo se si tenta di accedervi, bloccando potenziali attacchi che cercano di sovrascrivere regioni di memoria adiacenti. Sebbene queste pagine migliorino la sicurezza, introducono un sovraccarico in termini di tempo di esecuzione a causa delle mappature aggiuntive richieste.

11.4 Altre Forme di Attacchi Overflow

Gli attacchi di buffer overflow non si limitano allo stack, ma includono varianti come il return-to-system-call, gli overflow di heap e quelli nell'area dati globale. Questi attacchi sfruttano vulnerabilità specifiche di diverse aree di memoria e richiedono difese mirate.

11.4.1 Sostituzione dello Stack Frame

In un classico stack buffer overflow, l'attaccante sovrascrive l'area delle variabili locali dello stack, puntatore al frame e indirizzo di ritorno. Una variante di questo attacco modifica solo il puntatore al frame, indirizzandolo a uno stack frame fittizio all'interno del buffer sovrascritto, il quale include un indirizzo di ritorno che punta allo shellcode. Questo attacco è utile in caso di overflow parziale, ma richiede precisione nell'individuare l'indirizzo esatto del buffer.

Difese

Le difese includono protezioni dello stack (come Stackguard), stack non eseguibili e randomizzazione dello stack in memoria, che riducono la probabilità di individuare indirizzi esatti.

11.4.2 Ritorno alla Chiamata di Sistema

Con la diffusione degli stack non eseguibili, gli attaccanti hanno iniziato a modificare l'indirizzo di ritorno per trasferire il controllo a codice già presente nel sistema, tipicamente funzioni di libreria come `system()`. L'attaccante sovrascrive il buffer, sostituisce l'indirizzo di ritorno con una funzione di sistema e inserisce i parametri necessari allo stack, sfruttando il codice della libreria per eseguire i propri comandi.

Una variante prevede di concatenare chiamate a due funzioni di libreria: ad esempio, `strcpy()` per copiare lo shellcode in una parte eseguibile della memoria e poi un'altra funzione per eseguirlo, aggirando le protezioni dello stack.

Difese

Le difese includono meccanismi di protezione dello stack, randomizzazione dello stack e delle librerie di sistema.

11.4.3 Heap Overflow

Gli attaccanti possono sfruttare gli overflow nei buffer situati nell'heap, l'area di memoria allocata dinamicamente per strutture dati come liste e record. Sebbene l'heap non contenga indirizzi di ritorno, un attaccante può sovrascrivere un puntatore a funzione e farlo puntare al proprio shellcode.

Un esempio comune prevede la sovrascrittura di un puntatore a funzione in una struttura dati vulnerabile per indirizzarlo al codice malevolo nell'heap.

Difese

Per difendersi dagli heap overflow, si possono rendere non eseguibili le pagine dell'heap, randomizzare l'allocazione della memoria e aggiungere controlli di integrità nei gestori di allocazione della memoria.

11.4.4 Overflow nell'Area Dati Globale

Anche i buffer nell'area dati globale possono essere vulnerabili a overflow, consentendo di modificare puntatori a funzione o strutture dati adiacenti. Questo tipo di attacco sfrutta la natura statica delle variabili globali, che hanno indirizzi fissi all'interno del codice.

Difese

Proteggere l'area dei dati globali rendendola non eseguibile, posizionare i puntatori a funzione sotto altri dati per ridurre il rischio di sovrascrittura, e utilizzare pagine di guardia per prevenire accessi non autorizzati.

12 - Sicurezza del Software

12.1 Problemi di Sicurezza del Software

La stragrande maggioranza delle vulnerabilità di sicurezza informatica deriva da pratiche di programmazione inadeguate. Enti di ricerca e liste autorevoli evidenziano come la maggior parte degli attacchi sfrutti un sottoinsieme limitato e ricorrente di errori architetturali tra cui: **Injection** (es. SQL Injection, OS Command Injection), **Cross-Site Scripting (XSS)**, **Buffer Overflow**, **Gestione insicura delle risorse** e della memoria, **Difese logiche inadeguate** (es. autenticazione mancante, credenziali *hard-coded*, crittografia debole).

12.2 Sicurezza vs. Qualità del Software

- **Qualità e Affidabilità (*Software Reliability*):** Si occupa della prevenzione e gestione dei fallimenti accidentali, tipicamente causati da input imprevisti o *bug* stocastici. L'obiettivo è massimizzare la stabilità del sistema durante il suo utilizzo nominale.
- **Sicurezza (*Software Security*):** Si concentra sui fallimenti indotti deliberatamente da un attaccante (attore ostile), il quale seleziona input anomali e specifici al fine di innescare e sfruttare vulnerabilità nel codice. Tali input sono spesso statisticamente improbabili in un contesto di utilizzo standard.

12.3 Programmazione Difensiva

Per mitigare le minacce alla sicurezza, l'ingegneria del software adotta il paradigma della **programmazione difensiva**. Esso consiste nel progettare e implementare il software affinché mantenga la propria integrità e operatività anche sotto attacco.

Il sistema deve essere in grado di rilevare stati anomali e, nel peggiore dei casi, attuare una *terminazione controllata* (fail gracefully), evitando di esporre dati sensibili o concedere privilegi non autorizzati. Il postulato fondamentale di questo approccio è: *non dare mai nulla per scontato*. Ogni assunzione deve essere verificata a tempo di esecuzione e ogni possibile stato di errore deve essere gestito esplicitamente.

12.4 Gestione Sicura dell'Input

La corretta gestione dell'input rappresenta la prima linea di difesa. Per "input" si intende qualsiasi flusso di dati proveniente dall'esterno del perimetro di fiducia del programma (tastiera, rete, file system, variabili d'ambiente). Le due criticità principali relative all'input sono la sua **dimensione** e la sua **interpretazione**.

12.4.1 Dimensione dell'Input e Buffer Overflow

Se un'applicazione non verifica rigorosamente che la dimensione dei dati in ingresso sia compatibile con la capacità del buffer di memoria preposto a contenerli, si verifica un **Buffer Overflow**.

Questa vulnerabilità permette a un attaccante di sovrascrivere aree di memoria adiacenti, alterando il flusso di esecuzione del programma e arrivando, nei casi più gravi, all'esecuzione di codice arbitrario (*Arbitrary Code Execution*). La mitigazione primaria consiste nell'utilizzo esclusivo di funzioni di manipolazione della memoria e delle stringhe che richiedano esplicitamente la dimensione massima del buffer (es. `strncpy` in luogo di `strcpy` in C).

12.4.2 Interpretazione dell'Input e Attacchi di Injection

Un attacco di *Injection* si verifica quando input non sanitizzato viene interpretato dal sistema come istruzione o comando, alterando la logica di esecuzione.

- **Command Injection:** L'input dell'utente viene concatenato a una *system call*. Inserendo metacaratteri della shell (es. `;` o `|`), l'attaccante può eseguire comandi arbitrari sul sistema operativo ospite.
- **SQL Injection (SQLi):** L'input viene impiegato per la costruzione dinamica di una query al database. Sfruttando metacaratteri SQL (es. `'` o `-`), è possibile manipolare la sintassi della query per aggirare l'autenticazione, esfiltrare dati o corrompere il database.
- **Code Injection:** L'input fornito viene valutato ed eseguito direttamente come codice sorgente dall'interprete (es. vulnerabilità di *Remote File Inclusion* in PHP).

12.4.3 Tecniche di Validazione dell'Input

Per neutralizzare tali minacce, è imperativo sottoporre ogni input a rigorosi processi di validazione:

- **Whitelisting (Raccomandato):** Consiste nell'accettare esclusivamente input che corrisponda a un formato, tipo o pattern (*Regular Expression*) esplicitamente definito come sicuro. Tutto il traffico non conforme viene scartato per impostazione predefinita.
- **Blacklisting (Sconsigliato):** Tentativo di bloccare stringhe o pattern noti per essere malevoli. È un approccio intrinsecamente debole, poiché l'attaccante troverà costantemente nuove codifiche per eludere il filtro.
- **Canonicalization:** Prima di qualsiasi validazione, l'input deve essere convertito nella sua forma canonica (la rappresentazione standard e minimale). Questo impedisce che tecniche di evasione basate su codifiche alternative (es. *URL encoding*, codifiche UTF-8 non standard) superino i controlli di sicurezza.

12.5 Scrittura di Codice Sicuro e Gestione della Memoria

Oltre alla validazione perimetrale dell'input, la sicurezza intrinseca dipende dall'implementazione logica e dalla gestione delle risorse in memoria.

12.5.1 Corretta Implementazione dell'Algoritmo

Un difetto nella logica algoritmica può esporre il sistema a criticità severe, indipendentemente dalla pulizia dell'input. Esempi storici includono:

- **Debolezza crittografica (Il caso Netscape):** L'utilizzo di un generatore di numeri pseudocasuali (PRNG) con *seeding* prevedibile compromise la sicurezza delle prime sessioni SSL.
- **TCP Session Hijacking:** L'impiego di *Initial Sequence Numbers* (ISN) prevedibili nello stack TCP/IP consentiva ad attaccanti di iniettare pacchetti in sessioni legittime.
- **Backdoor e Debug Residuo:** Il *Morris Worm* sfruttò la mancata rimozione di un comando di `DEBUG` nel demone `sendmail` nei sistemi in produzione.

12.5.2 Tipizzazione, Memory Leak e Race Condition

- **Tipizzazione Forte (*Strong Typing*):** Linguaggi come Java o Rust impongono vincoli rigorosi sui tipi di dato, mitigando intere classi di vulnerabilità tipiche dei linguaggi a tipizzazione debole (come C/C++), dove conversioni implicite e aritmetica dei puntatori espongono al rischio di corruzione della memoria.
- **Memory Leak:** La mancata deallocazione dinamica della memoria sull'*heap* causa l'esaurimento progressivo delle risorse, condizione sfruttabile per attacchi di *Denial-of-Service* (DoS).
- **Race Condition:** Si manifesta quando processi operanti in concorrenza accedono a risorse condivise senza un'adeguata sincronizzazione (es. assenza di *mutex* o operazioni atomiche). Questo può condurre alla corruzione dello stato interno o a *deadlock*.

12.6 Interazione con il Sistema Operativo e Privilegi

L'esecuzione di un programma è mediata dal Sistema Operativo, il quale introduce ulteriori vettori di attacco.

12.6.1 Il Principio del Minimo Privilegio (*Least Privilege*)

Un processo deve essere eseguito possedendo unicamente i privilegi strettamente necessari all'assolvimento della propria funzione, e per il minor tempo possibile. Per implementare questo principio si ricorre a:

- **Privilege Dropping:** Demoni che necessitano di privilegi di *root* solo per il binding iniziale (es. su porte < 1024) devono de-escalare i propri permessi subito dopo l'avvio.
- **Sandboxing e Chroot:** Esecuzione di processi non fidati in ambienti virtualmente isolati dal file system principale e dalle chiamate di sistema critiche.

12.7 Gestione Sicura dell'Output e Cross-Site Scripting (XSS)

Nel contesto web, il **Cross-Site Scripting (XSS)** si verifica quando un'applicazione riflette o memorizza input malevolo fornito da un utente (tipicamente *payload* in JavaScript) e lo ripropone al browser di altre vittime. Poiché il browser si fida dell'origine della pagina, eseguirà lo script, consentendo all'attaccante di esfiltrare *cookie* di sessione o manipolare il DOM (*Document Object Model*).

Difese per l'Output:

- **Sanitizzazione e Output Encoding:** Trasformare i caratteri speciali in entità inoffensive (es. `<` in `<`) prima della renderizzazione.
- **Dichiarazione del Character Encoding:** Specificare esplicitamente la codifica (es. UTF-8) nell'header HTTP per impedire interpretazioni anomale da parte del parser del browser.

12.8 Analisi delle Vulnerabilità e Sicurezza delle Dipendenze

Lo sviluppo software moderno si affida pesantemente a librerie e moduli open-source di terze parti. Una vulnerabilità in una singola libreria può compromettere l'intero sistema (Supply Chain Attack). Per mitigare questo rischio si utilizzano strumenti di **Software Composition Analysis (SCA)**. Nel panorama Python, ad esempio, esistono strumenti come `pip-audit`. Tali tool analizzano l'albero delle dipendenze del progetto confrontandolo con i database ufficiali delle vulnerabilità (come il CVE - *Common Vulnerabilities and Exposures*) segnalando in tempo reale pacchetti obsoleti o compromessi, obbligando così lo sviluppatore a mantenere un ecosistema di librerie aggiornato e sicuro.

13 - Sicurezza nei Sistemi Operativi e Virtualizzazione

13.1 Introduzione alla Sicurezza dei Sistemi Operativi

I sistemi client e server costituiscono le fondamenta dell'infrastruttura IT di qualsiasi organizzazione, ospitando dati sensibili e applicazioni *mission-critical*. È imperativo riconoscere che i Sistemi Operativi (OS) presentano vulnerabilità intrinseche, costantemente sondate da agenti malevoli (es. *worm* o *botnet*). Un sistema può essere compromesso già durante la fase di installazione (*deployment*), prima ancora che vengano applicate le patch di sicurezza o implementate le policy di *hardening*.

Per mitigare tale rischio, il deployment di un sistema deve seguire un processo strutturato, concepito per integrare la sicurezza *by design* e mantenerla per l'intero ciclo di vita operativo.

13.2 Il Processo di Messa in Sicurezza (NIST SP 800-123)

La pubblicazione del **NIST SP 800-123** definisce un framework rigoroso per il deployment sicuro dei sistemi, articolato nelle seguenti fasi:

1. **Valutazione dei Rischi e Pianificazione:** Analisi preventiva del contesto operativo e dei requisiti.
2. **Hardening dell'OS base:** Messa in sicurezza del sistema operativo prima dell'esposizione in rete.
3. **Sicurezza delle Applicazioni:** Configurazione sicura dei servizi applicativi chiave.
4. **Protezione del Contenuto:** Salvaguardia dei dati critici tramite crittografia e controlli di accesso.
5. **Sicurezza di Rete:** Implementazione di firewall e sistemi di rilevamento.
6. **Manutenzione Continua:** Definizione di processi per il patching, il logging e il backup nel tempo.

L'obiettivo della pianificazione è massimizzare la sicurezza minimizzando i costi operativi. Tentare di applicare controlli di sicurezza in *retrofit* su un sistema già in produzione risulta sempre più oneroso e architetturalmente fragile rispetto a un'implementazione nativa.

13.3 Hardening del Sistema Operativo

La messa in sicurezza dell'OS rappresenta il fondamento fiduciario (*Root of Trust*) per tutte le applicazioni sovrastanti. Le configurazioni di *default* dei produttori privilegiano storicamente l'usabilità e la retrocompatibilità a scapito della sicurezza (approccio *insecure by default*).

L'hardening procede attraverso i seguenti principi:

- **Installazione Sicura:** Il deployment iniziale deve avvenire in una rete isolata (VLAN di *staging*) per prevenire infezioni pre-patching.
- **Principio di Minimalità:** Installare esclusivamente i pacchetti, i moduli e i driver strettamente necessari. La rimozione (o la mancata installazione) di un servizio riduce drasticamente la *superficie di attacco*.
- **Sicurezza del Boot:** Proteggere il processo di avvio (es. password UEFI/BIOS, *Secure Boot*) per prevenire il caricamento di *rootkit* o *hypervisor* malevoli.

13.3.1 Manutenzione, Logging e Backup

La postura di sicurezza deve essere mantenuta dinamicamente. Il **Logging** è un controllo reattivo fondamentale per la *Digital Forensics* e l'*Incident Response*. I log devono essere raccolti, ruotati e analizzati (preferibilmente in modo automatizzato tramite SIEM o IDS). Parallelamente, le policy di **Backup e Archiviazione** garantiscono la *Disaster Recovery*, proteggendo l'organizzazione da guasti fisici o attacchi distruttivi (come i *Ransomware*), a patto che i backup siano mantenuti offline o logicamente segregati.

13.4 Sicurezza in Ambiente Linux/Unix

I sistemi *Unix-like* basano la loro sicurezza su un modello di permessi granulare (Controllo degli Accessi Discrezionale - DAC).

- **Utenti, Gruppi e ACL:** I permessi classici (lettura, scrittura, esecuzione per *owner, group, others*) possono essere estesi tramite le *Access Control Lists* (ACL) per scenari complessi. L'autenticazione è tipicamente demandata al framework **PAM** (*Pluggable Authentication Modules*).
- **Programmi setuid/setgid:** I file eseguibili con il bit `setuid` impostato vengono eseguiti con i privilegi del proprietario del file (spesso *root*), indipendentemente da chi li lancia. Sono vettori primari per attacchi di *privilege escalation* (exploit locali) e il loro numero deve essere minimizzato.
- **Isolamento tramite Chroot Jail:** Un processo di rete vulnerabile (es. un server FTP) può essere confinato in una *chroot jail*, riassegnando la radice del file system (`/`) a una specifica sottodirectory. In caso di compromissione, l'attaccante rimarrà intrappolato nella *jail*, senza visibilità sul resto dell'OS.

13.5 Sicurezza in Ambiente Windows

I sistemi Microsoft Windows utilizzano un'architettura di sicurezza basata sui **Security Identifier (SID)** e su una gestione centralizzata (tramite *Active Directory* nei domini aziendali).

- **Mandatory Integrity Control (MIC):** Introdotto con Windows Vista, il MIC implementa una forma del modello di sicurezza di *Biba*. Assegna livelli di integrità (Basso, Medio, Alto, Sistema) ai processi e agli oggetti, impedendo a codice non attendibile (es. un processo del browser a integrità bassa) di modificare file di sistema (a integrità alta).
- **User Account Control (UAC):** Principio del minimo privilegio applicato all'interfaccia utente. Gli amministratori operano con un token di accesso standard, richiedendo un'elevazione esplicita per eseguire task amministrativi.
- **Registro di Sistema e Crittografia:** L'hardening in Windows passa spesso dalla configurazione di chiavi di registro (tramite *Group Policy Objects* - GPO). È inoltre fondamentale abilitare la *Data at Rest Encryption* (es. BitLocker per il disco intero, EFS per i singoli file).

13.6 Sicurezza della Virtualizzazione

La virtualizzazione astrae le risorse hardware fisiche fornendole a molteplici Macchine Virtuali (VM). L'**Hypervisor** (o *Virtual Machine Monitor* - VMM) è il software responsabile del partizionamento logico, dello *scheduling* e dell'isolamento delle VM.

13.6.1 Tipologie di Hypervisor

1. **Tipo 1 (Nativo / Bare-Metal):** Installato direttamente sull'hardware (es. VMware ESXi, Proxmox). Offre prestazioni superiori e una superficie di attacco minore, non dipendendo da un OS ospite sottostante.
2. **Tipo 2 (Hosted):** Eseguito come applicazione all'interno di un OS preesistente (es. VirtualBox). Più flessibile per l'uso *client*, ma eredita tutte le vulnerabilità dell'OS *host*.

13.6.2 Minacce e Isolamento

L'introduzione dell'hypervisor aggiunge un nuovo strato di criticità:

- **VM Escape:** Una vulnerabilità critica in cui un attaccante, partendo da una VM compromessa, riesce a eludere l'isolamento ed eseguire codice direttamente sull'hypervisor o nelle altre VM.
- **Traffico di Rete Virtuale:** Le VM comunicano spesso tramite switch virtuali interni all'hypervisor, rendendo il traffico invisibile ai tradizionali IPS hardware. È necessario implementare **Firewall Virtuali** (operanti nell'hypervisor o come *appliance* VM dedicate) e separare rigidamente il traffico di gestione (*management*), di infrastruttura (es. migrazione VM) e applicativo.

13.7 Metasploit Framework: Penetration Testing e Assessment

Il **Metasploit Framework** è l'infrastruttura open-source *de facto* per lo sviluppo e l'esecuzione di codice *exploit*. È utilizzato dai professionisti della sicurezza (*White Hat*) per validare l'efficacia delle misure di *hardening* e condurre *Penetration Test*.

13.7.1 Architettura e Moduli

Metasploit vanta un'architettura modulare che permette di disaccoppiare la fase di ingaggio (*exploit*) dal payload esecutivo:

Modulo	Descrizione e Funzione
Exploits	Codice progettato per sfruttare una specifica vulnerabilità architetturale o logica nel sistema bersaglio (es. buffer overflow, RCE).
Payloads	Codice eseguito sul sistema bersaglio ad avvenuta compromissione. Il più celebre è il **Meterpreter**, una *reverse shell* avanzata e *fileless* (residente interamente in RAM) che consente manipolazione processi, *keylogging* e *pivoting*.
Auxiliary	Moduli di supporto per la fase di ricognizione (*reconnaissance*): scanner di porte, *fuzzer*, *sniffer* e attacchi DoS.
Post	Strumenti di *post-exploitation* per il consolidamento dell'accesso, l'esfiltrazione degli *hash* delle password e i movimenti laterali nella rete.
Encoders/Nops	Meccanismi di offuscamento per alterare la firma crittografica del payload, eludendo i controlli degli IDS/IPS (antivirus), e sequenze per l'allineamento in memoria (*No-Operation sleds*).

Tabella 13.1: Moduli principali dell'architettura del Metasploit Framework.

13.7.2 Considerazioni Etiche e Legali

Metasploit è uno strumento a duplice uso (*dual-use*). Nel contesto accademico e professionale, il suo utilizzo è strettamente normato. L'esecuzione di moduli di exploit verso infrastrutture di cui non si possiede la proprietà, o per le quali non si è ottenuta un'autorizzazione scritta ed esplicita (*Scope of Work / Rules of Engagement*), costituisce un **reato informatico** severamente punito dalle normative internazionali.

14 - Cloud Computing e Internet of Things (IoT)

14.1 Cloud Computing: Definizioni e Architettura

Il paradigma IT moderno evidenzia una marcata transizione verso la migrazione delle operazioni infrastrutturali, computazionali e di archiviazione verso piattaforme interconnesse note come *Cloud Computing*. Tale adozione su larga scala introduce sfide architetturali e problematiche di sicurezza inedite, in particolare per quanto concerne la *Data Security* e l'isolamento dei *tenant*.

14.1.1 Definizione e Caratteristiche Essenziali

Secondo il **NIST SP 800-145**, il Cloud computing è "un modello che abilita l'accesso di rete pervasivo, conveniente e *on-demand* a un pool condiviso di risorse di calcolo configurabili che possono essere rapidamente fornite e rilasciate con un minimo sforzo di gestione". Il modello poggia su cinque caratteristiche essenziali:

1. **Accesso alla Rete Esteso (*Broad network access*):** Risorse fruibili tramite meccanismi standard da piattaforme client eterogenee.
2. **Elasticità Rapida (*Rapid elasticity*):** Scalabilità dinamica (*scale-up* e *scale-down*) delle risorse in base al carico.
3. **Servizio Misurato (*Measured service*):** Monitoraggio, controllo e fatturazione trasparente dell'utilizzo delle risorse (modello *pay-per-use*).
4. **Self-Service On-Demand:** Provisioning automatico delle capacità computazionali senza interazione umana diretta con il fornitore.
5. **Pooling delle Risorse (*Resource pooling*):** Modello *multi-tenant* in cui risorse fisiche e virtuali sono allocate e riallocate dinamicamente in base alla domanda.

14.1.2 Modelli di Servizio Cloud

Il NIST definisce tre modelli di servizio principali, che determinano la granularità del controllo e la ripartizione delle responsabilità tra Cliente (*Cloud Service Consumer* - CSC) e Fornitore (*Cloud Service Provider* - CSP).

- **Software as a Service (SaaS):** Il cliente fruisce di un'applicazione completa erogata dal CSP (es. Microsoft 365, Salesforce). Il cliente demanda l'intera gestione dell'infrastruttura e dell'applicativo.
- **Platform as a Service (PaaS):** Il CSP fornisce l'infrastruttura, il SO e l'ambiente di runtime. Il cliente ha il controllo esclusivo sullo sviluppo e sul deployment delle proprie applicazioni (es. Heroku, AWS Elastic Beanstalk).

Stack IT	On-Premises	IaaS	PaaS	SaaS
Applicazioni	Cliente	Cliente	Cliente	CSP
Dati	Cliente	Cliente	Cliente	CSP
Runtime	Cliente	Cliente	CSP	CSP
Middleware	Cliente	Cliente	CSP	CSP
Sistema Operativo	Cliente	Cliente	CSP	CSP
Virtualizzazione	Cliente	CSP	CSP	CSP
Server / Storage / Rete	Cliente	CSP	CSP	CSP

Tabella 14.1: Matrice delle responsabilità nei diversi modelli di servizio Cloud.

- **Infrastructure as a Service (IaaS):** Il CSP fornisce unicamente le risorse computazionali virtualizzate base (CPU, RAM, Storage, Rete). Il cliente ha il controllo completo dallo strato del Sistema Operativo in su (es. Amazon EC2, Google Compute Engine).

14.1.3 Modelli di Deployment

L'infrastruttura Cloud può essere implementata secondo quattro topologie principali:

Modello	Descrizione	Vantaggi e Svantaggi
Cloud Pubblico	Risorse hardware/software di proprietà di terzi e offerte via Internet a molteplici tenant.	*Pro:* Bassi costi operativi, alta scalabilità. *Contro:* Minor controllo e confini normativi complessi.
Cloud Privato	Infrastruttura dedicata a uso esclusivo di una singola organizzazione (on-premise o ospitata).	*Pro:* Massima sicurezza e governance sui dati. *Contro:* Alti costi di capitale (CAPEX) e gestione.
Cloud Comunitario	Condivisione dell'infrastruttura tra organizzazioni indipendenti con requisiti/policy affini (es. settore PA o sanitario).	*Pro:* Costi condivisi, compliance di settore. *Contro:* Scalabilità limitata rispetto al pubblico.
Cloud Ibrido	Architettura composita che orchestra due o più infrastrutture.	*Pro:* Flessibilità operativa e ottimizzazione dei costi. *Contro:* Elevata complessità di integrazione e gestione.

Tabella 14.2: Confronto tra i modelli di deployment Cloud.

L'Architettura di Riferimento (NIST SP 500-292) introduce, oltre a CSP e CSC, figure intermediarie come il **Cloud Broker** (che aggrega, arbitra o intermedia servizi

per conto del cliente) e il **Cloud Auditor** (parte terza che valuta compliance, sicurezza e SLA).

14.2 Sicurezza nel Cloud Computing

14.2.1 Multi-Tenancy e Isolamento

L'allocazione delle risorse tramite il modello multi-tenant espone i clienti al rischio di *cross-tenant attacks* o *side-channel attacks* sulla cache condivisa dell'hardware.

14.2.2 Sicurezza dei Dati (*Data at Rest* e *Data in Transit*)

La perdita di controllo fisico sui server richiede un affidamento critico alla crittografia. La *Best Practice* impone che i dati vengano crittografati dal cliente *prima* di essere caricati nel Cloud (crittografia *client-side*), mantenendo la gestione delle chiavi (*Key Management*) separata dal CSP. In ambienti database, si distinguono due architetture:

- **Modello Multi-Istanza:** Un DBMS isolato in una VM dedicata per singolo tenant (massimo isolamento logico e controllo RBAC).
- **Modello Multi-Tenant:** Singola istanza DBMS condivisa. I record sono segregati tramite un "identificatore di tenant". Minor overhead ma maggiore rischio di fuga di dati (*data leakage*).

14.2.3 Security as a Service (SecaaS)

La *Cloud Security Alliance* (CSA) ha categorizzato la fornitura di servizi di sicurezza in outsourcing (SecaaS). Tra le categorie principali vi sono: l'**IAM** (*Identity and Access Management*), la **DLP** (*Data Loss Prevention*) e i servizi di monitoraggio integrato (**SIEM**).

14.2.4 Il Modulo di Sicurezza OpenStack Keystone

Nei deployment IaaS open-source basati su *OpenStack*, l'autenticazione, la gestione delle *policy* e il registro dei servizi sono demandati al componente **Keystone**. Esso emette *token* di sessione e funge da *Identity Provider* centralizzato, garantendo il Controllo degli Accessi Basato sui Ruoli (RBAC) in un ambiente multi-tenant.

14.3 Internet of Things (IoT) e Livelli di Rete

L'Internet of Things (IoT) rappresenta l'interconnessione globale di dispositivi *deeply embedded*, sensori e attuatori, in grado di comunicare autonomamente. Costituisce la quarta generazione dell'Internet, seguendo l'IT tradizionale, le Reti Operative (OT) e il *Mobile/Personal Computing*.

Un dispositivo IoT (*Smart Object*) è composto tipicamente da: **Sensori** (trasduttori da grandezze fisiche a segnali digitali), **Attuatori** (trasduttori per l'interazione

con l'ambiente), un **Microcontrollore** (MCU) per l'elaborazione locale, e un **Ricetrasmettitore** per la comunicazione wireless. Spesso la tecnologia **RFID** funge da abilitatore per l'identificazione passiva.

14.3.1 Architettura di Rete IoT

La complessità topologica dei sistemi IoT richiede un'architettura suddivisa in quattro livelli:

1. **Edge:** La rete terminale dei dispositivi e dei gateway. Operano con latenze dell'ordine dei millisecondi su protocolli eterogenei (es. ZigBee, Bluetooth).
2. **Fog (Edge Computing):** Strato intermedio di pre-elaborazione. Il Fog Computing decentralizza il Cloud, portando capacità di elaborazione, stoccaggio e distillazione del dato (*data reduction*) fisicamente vicino ai sensori, garantendo latenze bassissime.
3. **Core:** La dorsale IP/MPLS che interconnette nodi geograficamente dispersi con elevata ridondanza e capacità.
4. **Cloud:** Data Center massivi per l'analisi predittiva e l'archiviazione a lungo termine di moli di dati strutturati e non strutturati (Big Data).

14.4 Sicurezza nell'Ambiente IoT

14.4.1 Requisiti e Framework Operativi

Il **Cisco IoT Security Framework** organizza la sicurezza in:

- **RBAC:** Controllo accessi basato sui ruoli, scalabile per milioni di endpoint.
- **Anti-Tampering:** Rilevamento di manomissioni fisiche per sensori geograficamente esposti.
- **Data & IP Protection:** Cifratura persistente del dato sia a riposo che in transito lungo tutto lo stack di rete (dal livello di link fino al livello applicativo).

14.4.2 Sicurezza Open-Source: Il modulo MiniSec

In contesti estremamente vincolati come le *Wireless Sensor Networks* (WSN) governate da **TinyOS**, vengono impiegati protocolli ottimizzati come **MiniSec**. MiniSec assicura riservatezza, autenticazione del mittente e protezione dai *Replay Attack* pur minimizzando il consumo energetico. Utilizza:

- Il cifrario a blocchi **Skipjack** (progettato da NSA per architetture con basso *memory footprint*).
- La modalità di cifratura autenticata **OCB (*Offset Codebook*)**, che genera *ciphertext* e tag di autenticazione in un singolo passaggio.

MiniSec opera in due modalità: **MiniSec-U** (Unicast) impiega contatori sincronizzati, mentre **MiniSec-B** (Broadcast) mitiga i *Replay Attack* dividendo il tempo in "epoche" e sfruttando la struttura dati probabilistica dei *Bloom Filter* all'interno della stessa epoca.

15 - Sicurezza IT e Prevenzione dei Rischi

15.1 Definizione e Ciclo di Gestione

La Gestione della Sicurezza IT è il processo formale atto a sviluppare e mantenere livelli di sicurezza proporzionati per gli *asset* informativi di un'organizzazione. L'obiettivo è preservare le proprietà fondamentali dell'informazione: **Riservatezza** (*Confidentiality*), **Integrità** (*Integrity*), **Disponibilità** (*Availability*) – il noto paradigma CIA – unito ad Autenticità, Affidabilità e Responsabilità (*Accountability*).

15.1.1 Il Modello Plan-Do-Check-Act (PDCA)

Il processo non è statico, bensì iterativo e continuo. Standard come l'ISO 27005 modellano questo processo sul ciclo di Deming, noto come **PDCA**.

- **Plan (Pianificare):** Stabilire policy, definire il perimetro, eseguire il *Risk Assessment* e sviluppare un Piano di Trattamento del Rischio.
- **Do (Eseguire):** Implementare i controlli tecnici e organizzativi definiti nel piano.
- **Check (Verificare):** Monitorare continuativamente l'efficacia dei controlli, misurare la *compliance* e gestire gli incidenti.
- **Act (Agire):** Migliorare il sistema di gestione sulla base degli audit, degli incidenti o dei mutamenti del panorama delle minacce.

15.2 Contesto e Policy di Sicurezza

L'innesco del processo risiede nell'allineamento della sicurezza IT con gli obiettivi di *business* dell'organizzazione. La governance della sicurezza si declina in tre livelli gerarchici:

1. **Obiettivi di Sicurezza:** I risultati attesi in termini di protezione (es. "Garantire il 99.9% di uptime per il database clienti").
2. **Strategie:** Le metodologie architetturali e operative per raggiungere gli obiettivi.
3. **Policy di Sicurezza:** Il documento programmatico (approvato dal *Senior Management*) che definisce l'ambito, i requisiti legali, le responsabilità, la classificazione delle informazioni e l'approccio alla gestione del rischio.

La supervisione dell'intero processo è tipicamente delegata a una figura dirigenziale dedicata, come il **CISO** (*Chief Information Security Officer*) o l'IT Security Officer.

15.3 Il Processo di Risk Assessment

La Valutazione del Rischio (*Risk Assessment*) è il motore logico dell'ISMS. Senza di essa, l'allocazione delle risorse difensive risulterebbe casuale: si rischierebbe di sotto-proteggere asset critici o di sprecare budget per minacce inesistenti.

15.3.1 Approcci alla Valutazione del Rischio

Approccio	Descrizione e Trade-off
Baseline	Applicazione di configurazioni standard e *best practice* (es. CIS Benchmarks) a tutti i sistemi. *Pro:* Economico e veloce. *Contro:* Ignora il profilo di rischio specifico; può risultare insufficiente o inutilmente restrittivo.
Informale	Valutazione pragmatica non strutturata basata sull'esperienza (*expert judgment*). *Pro:* Agile. *Contro:* Soggettivo, incostante e difficile da giustificare in fase di audit.
Dettagliato	Analisi formale, quantitativa o qualitativa rigorosa, di ogni asset, vulnerabilità e minaccia. *Pro:* Massima *assurance*. *Contro:* Estremamente oneroso in termini di tempo e costi; rischio di obsolescenza rapida.
Combinato	(Raccomandato). Applicazione *Baseline* per tutti i sistemi, seguita da un'analisi informale rapida per individuare i sistemi critici, i quali verranno poi sottoposti ad Analisi Dettagliata. Ottimizza il rapporto costi/benefici.

Tabella 15.1: Confronto degli approcci alla Valutazione del Rischio.

15.4 Analisi Dettagliata del Rischio

15.4.1 1. Contesto e Identificazione degli Asset

Si definisce l'**Appetito per il Rischio** (*Risk Appetite*) del *management* (il livello di rischio che l'azienda è disposta ad accettare) e i vincoli normativi (es. GDPR, HIPAA). Successivamente, si esegue l'inventario e la classificazione degli **Asset** (hardware, software, dati, personale), pesandoli in base all'impatto che la loro compromissione avrebbe sul business.

15.4.2 2. Identificazione di Minacce e Vulnerabilità

- **Minaccia (*Threat*):** Un agente o evento potenziale (naturale, umano accidentale, umano deliberato) capace di causare danno.
- **Vulnerabilità (*Vulnerability*):** Una debolezza architetturale, software o procedurale intrinseca all'asset.

Una minaccia *sfrutta* una vulnerabilità per causare un impatto.

15.4.3 3. Analisi e Calcolo del Rischio

Il rischio è funzione della probabilità che l'evento si verifichi e dell'entità del danno:

$$\text{Rischio} = \text{Probabilità}(\text{Minaccia} \times \text{Vulnerabilità}) \times \text{Impatto}$$

Si procede valutando l'efficacia dei **Controlli Esistenti**, per poi stimare due vettori qualitativi:

1. **Probabilità (*Likelihood*):** Stimata dall'analista (es. da *1 - Raro* a *5 - Quasi Certo*).
2. **Impatto (*Consequence*):** Stimato dai proprietari degli asset (*Asset Owner*) e dal management (es. da *1 - Insignificante* a *6 - Doomsday/Catastrofico*).

La combinazione di queste due metriche genera una Matrice del Rischio che restituisce il Livello di Rischio finale (Basso, Medio, Alto, Estremo). Tutti i risultati confluiscono in un documento formale chiamato **Registro dei Rischi** (*Risk Register*).

15.5 Valutazione e Trattamento del Rischio

Una volta quantificato, il rischio viene confrontato con l'*Appetito per il Rischio*. Se il rischio residuo eccede la soglia di tolleranza, il *management* deve attuare un piano di trattamento scegliendo tra cinque opzioni strategiche:

1. **Riduzione della Probabilità (*Mitigation*):** Implementazione di controlli preventivi (es. firewall, *patching*) per rendere l'attacco più difficile.
2. **Riduzione dell'Impatto:** Implementazione di controlli correttivi o di resilienza (es. backup archiviati *offline*, piani di *Disaster Recovery*).
3. **Trasferimento del Rischio (*Transfer*):** Spostamento dell'onere finanziario su terzi (es. stipula di polizze assicurative *cyber* o *outsourcing* a provider Cloud qualificati).
4. **Evitamento del Rischio (*Avoidance*):** Dismissione del sistema, del processo di business o della tecnologia che genera il rischio, qualora i costi di mitigazione superino i benefici.
5. **Accettazione del Rischio (*Acceptance*):** Riconoscimento formale del rischio da parte del *Senior Management* senza ulteriori azioni di mitigazione (tipicamente applicato quando il rischio è già al di sotto dell'appetito aziendale o quando il costo dei controlli supera il valore dell'asset protetto).

16 - Controlli di Sicurezza

16.1 Classificazione dei Controlli di Sicurezza

Un **Controllo di Sicurezza** (o contromisura) è un'azione, un dispositivo, una procedura o una direttiva implementata per mitigare un rischio, riducendo la probabilità che si verifichi una violazione o minimizzandone l'impatto.

1. **Controlli di Gestione (*Management Controls*):** Di natura direttiva e strategica. Riguardano l'emanazione di policy, la pianificazione, il *Risk Assessment* e l'autorizzazione dei sistemi.

2. **Controlli Operativi (*Operational Controls*):** Eseguiti prevalentemente dal personale. Includono la formazione (*Awareness*), la sicurezza fisica, la gestione del personale e la risposta agli incidenti.

3. **Controlli Tecnici (*Technical Controls*):** Implementati a livello hardware e software dal sistema informatico. Includono il Controllo degli Accessi, la crittografia, l'autenticazione e i meccanismi di *audit/logging*.

Dal punto di vista funzionale, i controlli agiscono come: **Preventivi** (bloccano la minaccia alla fonte), **Rilevativi** (segnalano un'intrusione in corso, es. IDS) o **Correttivi/Di Ripristino** (ripristinano lo stato sicuro post-incidente, es. backup).

16.2 Selezione e Analisi Costi-Benefici

La selezione dei controlli avviene a valle del *Risk Assessment*. L'organizzazione adotta generalmente un approccio ibrido: implementa una **Baseline** di controlli standard per tutti i sistemi e aggiunge controlli **Basati sul Rischio** per gli asset critici.

La selezione finale è una decisione di *management* governata da un'**Analisi Costi-Benefici**. Un controllo è ritenuto fattibile solo se il costo (in termini finanziari e di impatto operativo) è inferiore al valore della riduzione del rischio (danno atteso) che esso garantisce. I controlli selezionati vengono formalizzati nell'**IT Security Plan**, un documento che assegna risorse, responsabilità e scadenze per l'implementazione.

16.3 Monitoraggio e Gestione dei Cambiamenti

Le fasi di verifica e mantenimento sono cruciali per prevenire l'obsolescenza difensiva:

- **Security Compliance e Audit:** Verifiche periodiche per accertare che i controlli siano utilizzati correttamente e che le configurazioni non abbiano subito derive (*configuration drift*).

- **Change Management:** Valutazione rigorosa dell'impatto sulla sicurezza di qualsiasi modifica architetturale o aggiornamento software prima della sua messa in produzione.

17 - Sicurezza Fisica e delle Infrastrutture

17.1 Panoramica sulla Sicurezza Fisica

Il ruolo della Sicurezza Fisica per i sistemi informativi è duplice:

1. Prevenire danni all'infrastruttura fisica che supporta l'information system.
2. Prevenire l'uso improprio dell'infrastruttura fisica che porta al danneggiamento o all'uso improprio delle informazioni protette.

L'infrastruttura da proteggere comprende:

- **Hardware del sistema informativo:** Apparecchiature di elaborazione e archiviazione, rete, supporti di archiviazione.
- **Struttura Fisica:** Edifici e strutture che ospitano i componenti.
- **Strutture di Supporto:** Energia elettrica, servizi di comunicazione e controlli ambientali (riscaldamento, umidità, ecc.).
- **Personale:** Individui coinvolti nel controllo, manutenzione e utilizzo dei sistemi.

L'uso improprio può essere accidentale o doloso (es. furto, vandalismo, accesso non autorizzato).

17.2 Minacce alla Sicurezza Fisica

Le minacce alla sicurezza fisica possono essere organizzate in tre categorie principali: Ambientali (comprese le catastrofi naturali), Tecniche e causate dall'Uomo.

17.2.1 Catastrofi Naturali e Minacce Ambientali

Le catastrofi naturali sono una fonte primaria di minacce ambientali, che possono causare danni estesi a data center, strutture e personale. Alcuni esempi includono:

Tornado: Avvertimento poco specifico per il sito, durata breve ma intensa. Provoca danni strutturali, perdita di apparecchiature esterne e interruzione temporanea di utility (misurato dalla Scala Fujita).

Uragano (Tropical Cyclone): Significativo preavviso (ore o giorni). Causa danni strutturali gravi e interruzione su vasta scala delle infrastrutture pubbliche (necessari generatori e rifornimenti di emergenza).

Terremoto: Nessun avvertimento, durata breve (con aftershock). Può causare danno catastrofico o distruzione totale, ribaltamento di hardware non fissato e danni alle strade e ai servizi.

Inondazione: Diversi giorni di preavviso, periodo esteso. Provoca danni gravi, effetti duraturi e contaminazione da residui.

Minacce Ambientali

Condizioni nell'ambiente che possono danneggiare o interrompere il servizio dei sistemi:

- **Temperatura e Umidità Inadeguate:**
  - *Temperatura Alta:* Danni ai componenti per surriscaldamento (danno permanente a 79°C per l'attrezzatura, 66°C per gli hard disk).
  - *Temperatura Bassa:* Shock termico all'accensione, che può danneggiare i circuiti.
  - *Umidità Alta:* Corrosione, cortocircuiti per condensa, effetto galvanico (fusione dei connettori).
  - *Umidità Bassa:* Deformazione dei materiali, accumulo di elettricità statica (scariche anche di poche centinaia di volt possono danneggiare i circuiti).
  - *Intervallo Ideale:* Umidità relativa mantenuta tra il 40% e il 60%.
- **Incendi e Fumo:** La minaccia maggiore è il fuoco. Il danno non deriva solo dalle fiamme, ma anche dal calore (es. l'alluminio fonde a 625°C), dai fumi tossici, dall'acqua usata per la soppressione e dal fumo abrasivo/conduttivo.
- **Danni da Acqua:** Cortocircuiti causati da perdite, rotture di tubi, o i sistemi sprinkler stessi. L'acqua da inondazione è la più dannosa a causa del residuo fangoso e contaminante che lascia.
- **Rischi Chimici, Radiologici e Biologici:** Pericolo primario per il personale. Possono danneggiare l'elettronica (agenti chimici e radiazioni) e possono essere introdotti accidentalmente o intenzionalmente (es. tramite il sistema di ventilazione).
- **Polvere (Dust):** Abrasiva e leggermente conduttiva; vulnerabilità per i componenti con parti in movimento (dischi, ventole). Può bloccare la ventilazione e ridurre il raffreddamento.
- **Infestazione:** Organismi viventi (muffe, insetti, roditori) che danneggiano attrezzature e carta, e possono prosperare in condizioni di alta umidità.

17.2.2 Minacce Tecniche

Minacce legate all'alimentazione elettrica e alle emissioni elettromagnetiche.

Alimentazione Elettrica (Electrical Power): Problemi raggruppabili in:

- *Sottotensione (Undervoltage):* Dips temporanei, Brownout (sottotensione prolungata) o Power Outages (blackout). La maggior parte dei computer tollera una riduzione del 20% prima di uno shutdown. Generalmente non provoca danni, ma interrompe il servizio.
- *Sovratensione (Overvoltage):* Surge (picchi) causati da anomalie, difetti di cablaggio o fulmini. Un picco sufficiente può distruggere i componenti basati sul silicio (processori, memorie).

- *Rumore (Noise):* Segnali spuri che interferiscono con l'alimentazione e possono causare errori logici nei dispositivi elettronici.

Interferenza Elettromagnetica (EMI): Il rumore elettrico generato da motori, ventole, altre apparecchiature o stazioni radio/antenne a microonde. Può essere trasmesso via cavo o via aria e causare problemi intermittenti. Anche dispositivi a bassa intensità, come i telefoni cellulari, possono interferire.

17.2.3 Minacce Causate dall'Uomo

Sono le più difficili da affrontare perché meno prevedibili e specificamente progettate per superare le misure di prevenzione.

- **Accesso Fisico Non Autorizzato:** L'accesso di persone non autorizzate a server, data center o aree ristrette. Può portare ad altre minacce.
- **Furto:** Furto di attrezzature o supporti, furto di dati (copia) e intercettazioni (Eavesdropping e Wiretapping). Può essere commesso da un outsider o da un insider.
- **Vandalismo:** Distruzione intenzionale di attrezzature e dati.
- **Uso Improprio (Misuse):** Uso improprio delle risorse da parte di personale autorizzato o uso delle risorse da parte di persone non autorizzate.

17.3 Misure di Prevenzione e Mitigazione della Sicurezza Fisica

Le misure di prevenzione e mitigazione mirano a prevenire o scoraggiare gli attacchi fisici, in linea con gli standard come ISO 27002 e NIST SP 800-53. Un vantaggio generale è l'uso del Cloud Computing, che riduce la necessità di asset di sistemi informativi in loco, diminuendo l'esposizione alle minacce fisiche on-site.

17.3.1 Minacce Ambientali

Temperatura e Umidità Inadeguate

La prevenzione richiede:

- Apparecchiature di Controllo Ambientale (HVAC) con capacità adeguate.
- Sensori appropriati per rilevare il superamento delle soglie.
- Manutenzione dell'Alimentazione Elettrica (vitale per il funzionamento dell'HVAC).

Incendi e Fumo

La gestione degli incendi combina allarmi, prevenzione e mitigazione:

- **Scelta del Sito:** Minimizzare i pericoli di incendio, acqua e fumo provenienti dalle aree adiacenti (pareti tagliafuoco con classificazione di almeno un'ora).
- **Condotti Aeraulici:** Progettati per non propagare il fuoco.
- **Buona Gestione (Good Housekeeping):** Non conservare materiali infiammabili o registrazioni non essenziali nell'area IS; installazione ordinata delle apparecchiature.
- **Sistemi di Spegnimento:** Estintori a disposizione e testati regolarmente. Estintori automatici installati in modo da minimizzare i danni alle apparecchiature.
- **Rivelatori e Controlli:** Rivelatori di fumo installati sotto i pavimenti rialzati e sopra i soffitti sospesi. Interruttore di spegnimento generale (Power-off switch) ben marcato e accessibile.
- **Procedure e Ispezioni:** Pubblicare procedure di emergenza, formare il personale, imporre il divieto di fumo nelle sale computer e condurre ispezioni regolari.
- **Conservazione dei Dati:** Archiviazione di registrazioni importanti in armadi ignifughi e archiviazione fuori sede delle copie di backup.

Danni da Acqua e Altre Minacce

- **Acqua:** Conoscenza del layout esatto delle tubazioni, posizione delle valvole di chiusura documentata e sensori d'acqua collocati sul pavimento che interrompono automaticamente l'alimentazione in caso di allagamento.
- **Chimiche, Biologiche, Radiologiche:** Approcci specifici di progettazione, sensori specializzati e procedure di mitigazione.
- **Polvere e Infestazioni:** Corretta manutenzione dei filtri, procedure regolari di controllo dei parassiti e mantenimento di un ambiente pulito.

17.3.2 Minacce Tecniche

Alimentazione Elettrica: Per brevi interruzioni/sottotensioni si utilizzano Gruppi di Continuità (UPS) per apparecchiature critiche (forniscono batteria, protezione da sovratensione e filtro antirumore). Per blackout prolungati è necessario il collegamento a una Fonte di Alimentazione di Emergenza, come un generatore.

Interferenza Elettromagnetica (EMI): Gestita attraverso una combinazione di filtri e schermatura (screening), a seconda della fonte e della natura dell'interferenza.

17.3.3 Minacce Causate dall'Uomo

L'approccio generale a queste minacce è il Controllo dell'Accesso Fisico.

Restrizione dell'Accesso

Si possono usare diverse strategie, spesso combinate, per isolare le risorse:

- **Restrizione a livello di Edificio:** Negare l'accesso agli estranei (non risolve il problema degli insider non autorizzati).
- **Isolamento e Ancoraggio:** Collocare le risorse in armadietti chiusi a chiave e fissare le apparecchiature a oggetti immobili.
- **Dispositivi di Tracciamento (Tracking Device):** Portali di rilevamento (allertano se una risorsa viene spostata) e monitoraggio continuo della posizione di oggetti portatili.

Misure di Controllo Perimetrale

Le tecniche includono aree controllate o pattugliate, barriere fisiche, porte con serrature o misure di screening, e sistemi di sorveglianza e rilevamento (sensori, telecamere, allarmi). L'introduzione del Wi-Fi estende la sicurezza fisica oltre i confini dell'edificio (es. parcheggi).

17.4 Ripristino da Violazioni della Sicurezza Fisica

L'elemento più essenziale per il ripristino è la **Ridondanza**.

17.4.1 Ripristino dei Dati e dei Servizi

- **Backup Off-Site:** Tutti i dati importanti devono essere disponibili fuori sede, aggiornati quasi in tempo reale (spesso tramite backup crittografati su banda larga).
- **Hot Site:** Per le situazioni più critiche, un sito pronto a subentrare nelle operazioni istantaneamente, con una copia dei dati operativi quasi in tempo reale.

17.4.2 Ripristino dell'Attrezzatura e del Sito

Il ripristino dipende dalla natura del danno e dal residuo (acqua, fumo, fuoco). Danni da acqua, fumo e fuoco possono lasciare materiali pericolosi che devono essere rimossi meticolosamente, richiedendo spesso l'intervento di specialisti esterni in disaster recovery.

18 - Algoritmi di Crittografia

18.1 Principi degli Algoritmi Crittografici

Un aspetto fondamentale nella progettazione di un algoritmo crittografico è la sua semplicità di analisi: se può essere descritto in modo chiaro e conciso, risulta più facile individuarne eventuali vulnerabilità e valutarne l'affidabilità. Questo approccio aperto segue il **Principio di Kerckhoffs**: la sicurezza di un crittosistema non deve basarsi sulla segretezza dell'algoritmo, ma esclusivamente su quella della chiave.

Nei cifrari a blocchi simmetrici basati su una struttura di Feistel, il processo di decifratura è essenzialmente identico a quello di cifratura. La regola è la seguente: si usa il testo cifrato come input per l'algoritmo, ma si utilizzano le sottochiavi (subkeys) K_i in ordine inverso. A livello operativo, l'elaborazione avviene dividendo il blocco di dati in due metà (sinistra e destra): a ogni round, la metà destra viene processata da una funzione non lineare F (insieme alla sottochiave) e il risultato viene unito in XOR con la metà sinistra, procedendo poi con lo scambio delle due metà. Questa caratteristica è estremamente vantaggiosa perché garantisce la perfetta reversibilità del processo senza dover implementare due algoritmi distinti, a prescindere da quanto sia complessa la funzione matematica F.

18.2 Data Encryption Standard (DES)

Gli algoritmi di cifratura simmetrica più comuni sono i cifrari a blocchi (block ciphers), i quali elaborano l'input in chiaro (plaintext) in blocchi di dimensione fissa per produrre un blocco di testo cifrato (ciphertext) di uguale dimensione.

Il DES, adottato nel 1977 dal NIST e basato sul Data Encryption Algorithm (DEA), presenta le seguenti caratteristiche tecniche:

- **Dimensione del Blocco:** 64 bit. L'algoritmo prevede inoltre l'applicazione di una Permutazione Iniziale (IP) al blocco prima dei round di elaborazione, e di una Permutazione Finale inversa (IP^{-1}) al termine di questi.

- **Dimensione della Chiave:** 56 bit effettivi. Sebbene la chiave fornita in input sia nominalmente di 64 bit, ogni ottavo bit viene scartato dall'algoritmo crittografico poiché utilizzato esclusivamente come bit di controllo di parità per rilevare eventuali errori.

- **Struttura:** Variante della rete di Feistel, composta da 16 round di elaborazione. La struttura è appositamente progettata per massimizzare l'**effetto valanga** (*avalanche effect*): una proprietà crittografica fondamentale per cui la variazione di un singolo bit nel plaintext o nella chiave innesca un cambiamento in circa metà dei bit del ciphertext, complicando enormemente il lavoro della crittoanalisi.

- **Sottochiavi:** Dalla chiave originale a 56 bit vengono generate 16 sottochiavi a 48 bit, una per ogni round.

Il processo di decifratura utilizza il testo cifrato come input e applica le sottochiavi in ordine inverso (da K_{16} a K_1).

18.3 Triple DES (3DES)

Il Triple DES (3DES) è stato standardizzato per superare i limiti di sicurezza del DES, la cui chiave a 56 bit era diventata vulnerabile ad attacchi a forza bruta. Utilizza tre chiavi e tre esecuzioni dell'algoritmo DES, secondo una sequenza EDE (Encrypt-Decrypt-Encrypt):

$$C = E(K_3, D(K_2, E(K_1, P)))$$

dove:

- C è il testo cifrato.
- P è il plaintext.
- $E(K, X)$ è la cifratura DES di X con la chiave K.
- $D(K, Y)$ è la decifratura DES di Y con la chiave K.

La decifratura avviene invertendo la sequenza e l'ordine delle chiavi:

$$P = D(K_1, E(K_2, D(K_3, C)))$$

L'uso della decifratura nella fase centrale offre il vantaggio di permettere la compatibilità con il DES singolo se tutte e tre le chiavi sono uguali ($K_1 = K_2 = K_3$).

Lunghezza della Chiave:

- Con tre chiavi distinte, la lunghezza effettiva è di 168 bit (3×56).
- Con la variante a due chiavi ($K_1 = K_3$), la lunghezza effettiva scende a 112 bit.

18.4 Advanced Encryption Standard (AES)

L'AES è stato pubblicato dal NIST nel 2001 (FIPS 197) per sostituire definitivamente DES e 3DES.

18.4.1 Panoramica dell'Algoritmo

L'AES opera su blocchi di 128 bit e supporta lunghezze di chiave di 128, 192 o 256 bit. A differenza del DES, non utilizza una struttura di Feistel: elabora l'intero blocco in parallelo durante ogni round.

L'input viene copiato in una matrice 4×4 di byte chiamata *State*, modificata in ogni fase. La struttura della cifratura prevede:

1. Una fase iniziale di *Add Round Key*.
2. Un numero di round completi (es. 9 round per chiavi a 128 bit).
3. Un round finale privo della fase *Mix Columns*.

La sicurezza dell'AES deriva dall'alternanza tra la fase di introduzione della chiave e le fasi che forniscono confusione e diffusione dei bit.

18.4.2 Dettagli delle Trasformazioni AES

Substitute Bytes (SubBytes): Sostituzione byte per byte non lineare tramite una S-box 16×16 predefinita, progettata per resistere ad attacchi crittanalitici.

Shift Rows (ShiftRows): Permutazione sulle righe della matrice *State*: la prima riga resta inalterata, la seconda trasla a sinistra di 1 byte, la terza di 2 e la quarta di 3. Garantisce un'ottima diffusione.

Mix Columns (MixColumns): Opera sulle colonne della matrice, combinando i quattro byte di ogni colonna. Insieme a *ShiftRows*, assicura che dopo pochi round ogni bit di output dipenda da ogni bit di input.

Add Round Key (AddRoundKey): Operazione di XOR bit a bit tra i 128 bit della matrice *State* e i 128 bit della sottochiave di round. Essendo lo XOR inverso di se stesso, la trasformazione inversa è identica.

18.5 Cifrari a Flusso (Stream Ciphers) e RC4

A differenza dei cifrari a blocchi, i cifrari a flusso elaborano gli elementi di input in modo continuo (spesso un byte alla volta).

18.5.1 Struttura e Funzionamento

Un tipico cifrario a flusso si basa su un generatore di bit pseudocasuali:

1. Una chiave inizializza il generatore.
2. Il generatore produce un flusso pseudocasuale imprevedibile chiamato *keystream*.
3. Il *keystream* viene combinato in XOR con il plaintext per produrre il ciphertext (e viceversa per la decifratura).

Vantaggi e Svantaggi: Sono estremamente veloci e leggeri in termini di codice. Il limite principale è che **una chiave non può mai essere riutilizzata**: cifrare due plaintext diversi con la stessa chiave rende banale la crittoanalisi (lo XOR tra i due testi cifrati equivale allo XOR dei plaintext originali).

18.5.2 L'Algoritmo RC4

Progettato nel 1987, l'RC4 utilizza una chiave variabile (da 1 a 256 byte) per inizializzare un vettore di stato S di 256 byte, contenente una permutazione dei numeri da 0 a 255.

- **Inizializzazione:** Il vettore S viene riempito e permutato in base a un vettore temporaneo T derivato dalla chiave.
- **Generazione del Keystream:** La chiave viene scartata. L'algoritmo cicla permutando continuamente gli elementi di S per generare un byte del keystream k a ogni iterazione, applicando poi lo XOR con il plaintext/ciphertext.

18.6 Modalità Operative dei Cifrari a Blocchi

Per elaborare messaggi più lunghi del singolo blocco (es. 64 o 128 bit), il NIST definisce diverse modalità operative:

Electronic Codebook (ECB): Ogni blocco è cifrato indipendentemente. **Svantaggio:** Blocchi di plaintext uguali producono ciphertext uguali, esponendo pattern.

Cipher Block Chaining (CBC): L'input dell'algoritmo è lo XOR tra il plaintext corrente e il ciphertext precedente. Usa un Vettore di Inizializzazione (IV) per il primo blocco. Nasconde i pattern ripetuti.

Cipher Feedback (CFB): Trasforma un cifrario a blocchi in un cifrario a flusso. L'input dell'algoritmo è il ciphertext precedente, e il suo output subisce uno XOR con il plaintext corrente.

Counter (CTR): Utilizza un contatore incrementale che viene cifrato e unito in XOR al plaintext. Offre **parallelismo**, efficienza e accesso casuale (possibilità di decifrare un singolo blocco isolato).

18.7 Distribuzione delle Chiavi

Per la crittografia simmetrica, la distribuzione sicura delle chiavi è fondamentale. I metodi includono la consegna fisica (diretta o tramite terzi) o la trasmissione cifrata. Per le reti su larga scala, si utilizza comunemente un **Key Distribution Center (KDC)**:

1. **Session Key:** Chiave temporanea usata per cifrare i dati utente.
2. **Permanent Key (Master Key):** Chiave a lungo termine condivisa solo tra il singolo host e il KDC.

Quando l'Host A vuole comunicare con B, richiede al KDC una *Session Key*. Il KDC la genera e la invia ad A (cifrata con la Master Key di A) e a B (cifrata con la Master Key di B). A questo punto, A e B condividono una chiave sicura per la sessione.

19 - Crittografia a Chiave Pubblica e Autenticazione del Messaggio

19.1 Funzioni di Hash Sicure

La funzione di hash unidirezionale, o funzione di hash sicura, è un elemento crittografico fondamentale non solo per l'autenticazione dei messaggi, ma anche per le firme digitali. Essa consente di ottenere un'impronta digitale (digest) di lunghezza fissa a partire da un input di lunghezza arbitraria, garantendo efficienza computazionale e compattezza del risultato. Le funzioni di hash moderne sono progettate per soddisfare proprietà di sicurezza ben precise, come la resistenza alla preimmagine, alla seconda preimmagine e alle collisioni. Tali proprietà rendono computazionalmente impraticabile risalire al messaggio originale o trovare due messaggi distinti che producano lo stesso valore di hash. Per questo motivo, esse rappresentano un componente essenziale nei protocolli di sicurezza e nei sistemi di autenticazione.

19.1.1 Funzioni di Hash Semplici

Tutte le funzioni di hash operano secondo principi generali comuni. L'input (messaggio, file, ecc.) viene visto come una sequenza di blocchi di n bit. L'input viene elaborato un blocco alla volta in modo iterativo per produrre un valore di hash di n bit.

Una delle funzioni di hash più semplici è l'operazione di XOR (exclusive-OR) bit a bit di ogni blocco. Può essere espressa come:

$$C_i = b_{i1} \oplus b_{i2} \oplus \cdots \oplus b_{im}$$

dove:

- C_i è l'i-esimo bit del codice hash, con $1 \leq i \leq n$.
- m è il numero di blocchi da n bit nell'input.
- b_{ij} è l'i-esimo bit nel j-esimo blocco.
- $\oplus$ è l'operazione di XOR.

Questa operazione (controllo di ridondanza longitudinale) calcola una semplice parità per ogni posizione di bit. È ragionevolmente efficace per dati casuali, ma inefficace con dati formattati in modo prevedibile.

Una semplice miglioria è la procedura RXOR, che consiste nell'eseguire uno spostamento circolare (rotazione) di 1 bit sul valore di hash dopo l'elaborazione di ogni blocco. Sebbene fornisca una buona misura di integrità, è **inutile per la sicurezza**: è banale produrre un nuovo messaggio che generi lo stesso codice hash aggiungendo un blocco finale calcolato ad hoc. Anche se usata con la cifratura in modalità CBC, un attaccante potrebbe riordinare i blocchi senza alterare l'hash finale.

19.1.2 La Funzione di Hash Sicura SHA

Negli ultimi anni, la famiglia di funzioni di hash più utilizzata è stata la Secure Hash Algorithm (SHA).

SHA-1: Pubblicata nel 1995, produce un hash di 160 bit. Oggi è deprecata a causa di vulnerabilità note.

SHA-2: Pubblicata nel 2002, è una famiglia con diverse lunghezze di output (SHA-224, SHA-256, SHA-384, SHA-512). Sono attualmente considerate sicure.

SHA-3: Pubblicata nel 2015 dal NIST, ha una struttura interna completamente diversa da SHA-2, offrendo un'alternativa sicura nel caso venissero scoperte nuove vulnerabilità.

19.1.3 Elaborazione di SHA-512

L'algoritmo SHA-512 prende in input un messaggio di lunghezza inferiore a 2^{128} bit e produce un digest di 512 bit, elaborando blocchi da 1024 bit.

Il processo si articola in 5 fasi:

1. **Aggiunta di bit di padding:** Il messaggio viene riempito affinché la sua lunghezza sia congruente a 896 modulo 1024 (un bit '1' seguito dai necessari '0').

2. **Aggiunta della lunghezza:** Un blocco di 128 bit, indicante la lunghezza originale, viene accodato. Il messaggio è ora un multiplo esatto di 1024 bit.

3. **Inizializzazione del buffer di hash:** Un buffer di 512 bit (otto registri a 64 bit: a, b, c, d, e, f, g, h) viene inizializzato con costanti derivate dalle radici quadrate dei primi 8 numeri primi.

4. **Elaborazione in blocchi:** Il cuore è una funzione di compressione a 80 round. Per ogni blocco, la funzione prende il buffer del passo precedente (H_{i-1}) e produce il nuovo (H_i). Ogni round usa una parola a 64 bit W_t e una costante K_t. L'output dell'80-esimo round viene sommato (modulo 2^{64}) al buffer di input H_{i-1}.

5. **Output:** Dopo l'elaborazione di tutti i blocchi, l'output dell'ultimo stadio è il message digest finale di 512 bit.

L'effetto valanga è garantito dall'estrema complessità delle operazioni: ogni bit dell'output dipende strettamente da ogni bit dell'input.

19.2 HMAC

Per usare una funzione di hash per l'autenticazione dei messaggi serve una chiave segreta. L'approccio standardizzato è l'HMAC (Hash-based Message Authentication Code). Si preferiscono i MAC basati su hash perché le funzioni di hash sono più veloci dei cifrari e il loro codice è ampiamente disponibile.

19.2.1 Obiettivi di Progettazione

- Utilizzare funzioni di hash esistenti senza modifiche.
- Permettere una facile sostituibilità della funzione di hash.
- Preservare le prestazioni originali.
- Gestire le chiavi in modo semplice.
- Avere un'analisi crittografica solida e ben compresa.

19.2.2 Algoritmo HMAC

HMAC combina una chiave segreta con il messaggio tramite due passaggi della funzione di hash:

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

dove:

- H è la funzione di hash (es. SHA-256).
- M è il messaggio.
- K è la chiave segreta.
- K^+ è la chiave K riempita con zeri fino alla dimensione del blocco della funzione H.
- ipad e opad sono costanti esadecimali predefinite.
- $\parallel$ rappresenta la concatenazione.

19.2.3 Sicurezza di HMAC

La sicurezza di HMAC è formalmente dimostrata ed è strettamente legata alla robustezza della funzione di hash H. Compromettere HMAC richiede, in pratica, trovare collisioni o prevedere l'output della funzione di compressione senza conoscere la chiave segreta K, operazione computazionalmente impraticabile. In assenza della chiave, anche un attacco del compleanno offline non risulta efficace.

19.3 Cifratura Autenticata

La cifratura autenticata garantisce contemporaneamente confidenzialità e integrità/autenticità dei dati. Un esempio efficiente di tale tecnica è la modalità **Offset Codebook (OCB)**.

19.3.1 Funzionamento di OCB

OCB trasforma un cifrario a blocchi (come AES) in uno schema AE. Il testo cifrato $C[i]$ del blocco di messaggio $M[i]$ è calcolato come:

$$C[i] = E_K(M[i] \oplus Z[i]) \oplus Z[i]$$

dove E_K è la cifratura con chiave K, e $Z[i]$ è un offset unico generato da un *nonce*. L'uso degli offset garantisce che blocchi plaintext identici generino ciphertext diversi. OCB produce anche un tag di autenticazione cifrando un checksum di tutti i blocchi plaintext.

19.4 L'Algoritmo di Cifratura a Chiave Pubblica RSA

Sviluppato nel 1977 da Rivest, Shamir e Adleman, l'RSA è il cifrario a chiave pubblica più diffuso. È un cifrario a blocchi dove plaintext e ciphertext sono interi compresi tra 0 e $n-1$.

19.4.1 Descrizione dell'Algoritmo

Le operazioni di cifratura (C) e decifratura (M) sono:

$$C = M^e \pmod{n}$$
$$M = C^d \pmod{n} = (M^e)^d \pmod{n} = M^{ed} \pmod{n}$$

Le chiavi sono definite come:

- **Chiave Pubblica:** $PU = \{e, n\}$
- **Chiave Privata:** $PR = \{d, n\}$

La decifratura funziona se e e d sono inversi moltiplicativi modulo $\phi(n)$, dove $\phi(n)$ è la funzione toziente di Eulero. Per due numeri primi p e q, $\phi(n) = (p-1)(q-1)$. La relazione matematica è:

$$ed \equiv 1 \pmod{\phi(n)}$$

19.4.2 Generazione delle Chiavi

1. Scegliere due numeri primi distinti, p e q.
2. Calcolare $n = p \times q$.
3. Calcolare $\phi(n) = (p-1)(q-1)$.
4. Scegliere e tale che $1 < e < \phi(n)$ e $\gcd(\phi(n), e) = 1$.
5. Calcolare d tale che $d \equiv e^{-1} \pmod{\phi(n)}$.

19.4.3 Esempio di Funzionamento

Se $p = 17$ e $q = 11$:

- $n = 187$ e $\phi(n) = 160$.
- Scelto $e = 7$ (coprimo con 160), si calcola $d = 23$ (poiché $23 \times 7 = 161 \equiv 1 \pmod{160}$).

Per cifrare $M = 88$: $C = 88^7 \pmod{187} = 11$. Per decifrare $C = 11$: $M = 11^{23} \pmod{187} = 88$.

19.4.4 La Sicurezza di RSA

Gli attacchi contro RSA si dividono in:

- **Forza Bruta:** Provare tutte le chiavi private. Mitigato usando chiavi molto grandi.
- **Attacchi Matematici (Fattorizzazione):** Basati sulla difficoltà di fattorizzare n in p e q, o di calcolare direttamente $\phi(n)$ o d. Attualmente si raccomandano chiavi tra i 1024 e i 2048 bit.
- **Timing Attacks:** Dedurre la chiave misurando il tempo di esponenziazione modulare. Contromisure includono tempi di calcolo costanti, ritardi casuali o il *blinding* (mascherare il ciphertext prima dell'esponenziazione).
- **Chosen Ciphertext Attacks:** Sfruttano proprietà specifiche dell'algoritmo.

19.5 Diffie-Hellman e Altri Algoritmi Asimmetrici

Il protocollo Diffie-Hellman permette di scambiare in modo sicuro una chiave segreta per la crittografia simmetrica. Si basa sulla difficoltà di calcolare i **logaritmi discreti** (trovare l'esponente i tale che $b \equiv a^i \pmod{p}$).

19.5.1 L'Algoritmo Diffie-Hellman

Siano q un numero primo e α una sua radice primitiva (pubblici):

1. L'utente A sceglie un segreto $X_A < q$ e calcola $Y_A = \alpha^{X_A} \pmod{q}$.
2. L'utente B sceglie un segreto $X_B < q$ e calcola $Y_B = \alpha^{X_B} \pmod{q}$.
3. A e B si scambiano pubblicamente Y_A e Y_B.
4. A calcola la chiave condivisa: $K = (Y_B)^{X_A} \pmod{q}$.
5. B calcola la chiave condivisa: $K = (Y_A)^{X_B} \pmod{q}$.

Entrambi ottengono lo stesso K poiché:

$$(Y_B)^{X_A} = (\alpha^{X_B})^{X_A} = \alpha^{X_B X_A} = (\alpha^{X_A})^{X_B} = (Y_A)^{X_B} \pmod{q}$$

20 - Sicurezza su Internet: Protocolli e Standard

20.1 Secure E-mail e S/MIME

S/MIME (Secure/Multipurpose Internet Mail Extension) è un'estensione di sicurezza allo standard di formattazione e-mail MIME. È stato progettato per fornire servizi crittografici ai messaggi di posta elettronica, tra cui autenticazione, integrità e riservatezza. Si basa sull'uso della crittografia a chiave pubblica e su certificati digitali X.509 per la gestione delle chiavi e l'identificazione dei mittenti.

20.1.1 MIME

MIME è un'estensione dello standard RFC 822 (che definiva il formato dei messaggi di testo su Internet in semplice ASCII). MIME introduce nuovi campi di intestazione per descrivere il corpo del messaggio, inclusi il formato e la codifica, definendo una serie di formati di contenuto per supportare e-mail multimediali (es. testo, immagini, audio, video). In particolare, permette di suddividere il messaggio in più parti (multipart), ciascuna con un proprio tipo di contenuto, rendendo possibile l'invio di allegati e dati non testuali in modo standardizzato e interoperabile tra diversi client di posta.

20.1.2 S/MIME e Funzionalità

La funzionalità S/MIME è integrata nella maggior parte dei software di posta elettronica moderni. Aggiunge nuovi tipi di contenuto MIME per fornire la capacità di firmare e/o cifrare i messaggi, supportando quattro nuove funzioni:

Dati Busta (Enveloped data): Consiste in un contenuto cifrato di qualsiasi tipo e nelle chiavi di cifratura del contenuto, a loro volta cifrate per uno o più destinatari. In questo modo solo i destinatari previsti, in possesso della chiave privata corretta, possono accedere al contenuto del messaggio.

Dati Firmati (Signed data): Una firma digitale creata calcolando il message digest del contenuto e cifrandolo con la chiave privata del mittente. Il contenuto e la firma vengono codificati in base64. Un messaggio di questo tipo può essere visualizzato solo da un destinatario con capacità S/MIME e garantisce autenticità e integrità.

Dati Firmati in Chiaro (Clear-signed data): Simile ai dati firmati, ma solo la firma digitale viene codificata in base64. Permette ai destinatari senza capacità S/MIME di leggere il contenuto del messaggio (pur non potendo verificare la firma), favorendo compatibilità con client meno avanzati.

Dati Firmati e Busta (Signed and enveloped data): Entità annidate: dati cifrati possono essere firmati, e dati firmati possono essere cifrati. Questa combinazione consente di ottenere simultaneamente riservatezza, autenticità e integrità in un unico messaggio strutturato.

20.1.3 Funzionamento Generale

Firma (Dati Firmati/Firmati in Chiaro):

1. Viene calcolato un message hash del contenuto (tipicamente SHA-256).
2. L'hash viene cifrato con la chiave privata del mittente (es. RSA o DSA) per creare la firma digitale.
3. La firma viene allegata al messaggio e codificata in base64 (che mappa i dati binari in caratteri ASCII stampabili) per proteggerla da alterazioni in transito.

Cifratura (Dati Busta):

1. S/MIME genera una chiave di sessione segreta pseudocasuale per il messaggio.
2. Il messaggio viene cifrato con la chiave di sessione usando un algoritmo simmetrico (es. AES).
3. La chiave di sessione viene cifrata con la chiave pubblica del destinatario (usando RSA) e trasmessa insieme al messaggio cifrato.
4. Il destinatario userà la propria chiave privata per decifrare la chiave di sessione e, di conseguenza, il messaggio.

Per gestire le chiavi pubbliche, S/MIME si affida a certificati a chiave pubblica conformi allo standard X.509v3.

20.2 DomainKeys Identified Mail (DKIM)

DKIM è una specifica per la firma crittografica dei messaggi e-mail che permette a un dominio mittente di assumersi la responsabilità di un messaggio. I destinatari verificano la firma interrogando il DNS del dominio mittente.

20.2.1 Architettura della Posta su Internet

- **MUA (Message User Agent):** Il client di posta dell'utente (es. Outlook).
- **MHS (Message Handling Service):** Composto da:
  - *MSA (Mail Submission Agent):* Accetta il messaggio dal MUA.
  - *MTA (Message Transfer Agent):* Inoltra il messaggio nella rete.
  - *MDA (Mail Delivery Agent):* Consegna il messaggio al Message Store.
- **MS (Message Store):** Archivio messaggi in attesa di recupero (via POP/IMAP).
- **ADMD (Administrative Management Domain):** Provider di posta (ISP, azienda).
- **DNS:** Servizio che mappa i nomi host in indirizzi IP.

20.2.2 Strategia di DKIM

DKIM è trasparente per l'utente finale:

1. Il messaggio viene firmato con la chiave privata del dominio amministrativo (ADMD), coprendo contenuto e intestazioni. Non usa la chiave dell'utente.
2. L'MDA del destinatario recupera la chiave pubblica del dominio mittente tramite DNS.
3. Se la verifica ha successo, conferma che il messaggio proviene dal dominio dichiarato e non è alterato.

A differenza di S/MIME (basato sull'utente), DKIM è uno strumento a livello di server fondamentale per combattere phishing e spoofing.

20.3 Secure Sockets Layer (SSL) e Transport Layer Security (TLS)

TLS, successore di SSL, fornisce un servizio di sicurezza end-to-end affidabile basandosi sul protocollo TCP.

20.3.1 Architettura TLS

TLS è composto da due livelli:

1. **Record Protocol:** Fornisce confidenzialità e integrità ai protocolli superiori.
2. **Protocolli di Livello Superiore:** *Handshake Protocol* (negoziazione parametri), *Change Cipher Spec Protocol* (attivazione parametri) e *Alert Protocol*.

Concetti chiave:

- *Connessione TLS:* Relazione transitoria peer-to-peer per il trasporto.
- *Sessione TLS:* Associazione tra client e server creata dall'Handshake. Definisce i parametri crittografici condivisibili tra più connessioni, evitando costose rinegoziazioni.

20.3.2 Protocolli TLS

Record Protocol: Frammenta i dati, li comprime (opzionale), calcola un MAC (per l'integrità), li cifra (per la confidenzialità) e aggiunge un'intestazione.

Change Cipher Spec Protocol: Singolo messaggio per segnalare il passaggio all'uso della suite di cifratura negoziata.

Alert Protocol: Invia avvisi (errori fatali o notifiche).

Handshake Protocol: Autentica le parti, negozia algoritmi (cifratura e MAC) e stabilisce le chiavi. Si divide in 4 fasi (scambio Hello, invio certificati/chiavi dal server, invio certificati/chiavi dal client, e messaggi Finished).

Heartbeat Protocol: Monitora la disponibilità della controparte e previene la chiusura della connessione da parte dei firewall.

20.3.3 Attacchi a SSL/TLS: Heartbleed

Nel 2014 fu scoperta *Heartbleed*, una vulnerabilità critica in OpenSSL legata all'Heartbeat Protocol. Non era un difetto del protocollo TLS, ma un grave bug di programmazione: una richiesta malformata induceva il server a restituire porzioni della propria memoria contenenti chiavi private e password.

20.4 HTTPS

HTTPS (HTTP over SSL/TLS) combina HTTP e TLS per comunicazioni sicure tra browser e server web (porta 443). HTTPS cifra:

- L'URL del documento richiesto.
- Il contenuto del documento e dei moduli compilati.
- I cookie e le intestazioni HTTP.

La chiusura di HTTPS richiede che TLS scambi correttamente gli avvisi *close_ notify*.

20.5 Sicurezza di IPv4 e IPv6 (IPsec)

IPsec è una suite che fornisce sicurezza a livello di rete (livello IP), garantendo autenticazione e cifratura trasparenti per tutto il traffico (VPN, accessi remoti, extranet). Se implementato su firewall/router, protegge tutto il traffico perimetrale.

20.5.1 Associazioni di Sicurezza (SA)

Una **SA (Security Association)** è una relazione unidirezionale tra mittente e destinatario che definisce i servizi di sicurezza da applicare. È identificata da: Indice dei Parametri di Sicurezza (SPI), IP di destinazione e identificatore protocollo (ESP).

20.5.2 Encapsulating Security Payload (ESP)

ESP è il protocollo IPsec che fornisce confidenzialità, integrità e autenticazione (sostituendo l'obsoleto AH).

ESP supporta due modalità:

Modalità Transport: Protegge solo il payload del pacchetto IP (es. segmento TCP), ma non l'intestazione IP. Usata per comunicazioni end-to-end tra host.

Modalità Tunnel: Protegge l'intero pacchetto IP originale, incapsulandolo in un nuovo pacchetto IP con una nuova intestazione esterna. Usata per le VPN (da gateway a gateway), nascondendo il traffico interno ai router intermedi.

21 - Autenticazione su Internet

21.1 Kerberos

Kerberos è uno standard Internet e *de facto* per l'autenticazione remota, sviluppato originariamente al MIT. È un servizio di autenticazione basato su una terza parte fidata (*trusted third party*). Client e server si affidano a Kerberos per mediare la loro autenticazione reciproca, richiedendo all'utente di provare la propria identità per ogni servizio invocato e, opzionalmente, ai server di provare la loro identità ai client.

21.1.1 Il Protocollo Kerberos

Il protocollo Kerberos coinvolge client, server applicativi e un server Kerberos. È progettato per contrastare varie minacce (in particolare l'impersonificazione) centralizzando l'autenticazione. L'Authentication Server (AS) conosce le password di tutti i client: non si limita a validare l'identità, ma fornisce al client un *ticket* per richiedere servizi in modo sicuro tramite crittografia (originariamente DES, ora AES).

Funzionamento del Protocollo

Il flusso di autenticazione si articola in passaggi ben definiti:

1. **Richiesta di un Ticket-Granting Ticket (TGT):** L'utente si collega a una workstation e il client invia una richiesta all'AS contenente l'ID dell'utente. L'AS verifica l'ID, crea un TGT e una chiave di sessione temporanea. Entrambi vengono cifrati con una chiave derivata dalla password dell'utente e inviati al client. *La password non viene mai trasmessa sulla rete.*

2. **Ottenimento del TGT da parte del Client:** Il client richiede la password all'utente per decifrare il messaggio dell'AS, recuperando TGT e chiave di sessione. Il TGT è una credenziale riutilizzabile (cifrata con una chiave segreta condivisa solo tra AS e il Ticket-Granting Server - TGS) contenente l'ID utente, un timestamp, una scadenza e una copia della chiave di sessione.

3. **Richiesta di un Service-Granting Ticket (SGT):** Per accedere a un server applicativo (es. Server V), il client invia al TGS una richiesta contenente il TGT e un *Authenticator* (struttura dati monouso e a vita breve, contenente ID e timestamp, cifrata con la chiave di sessione). L'Authenticator previene gli attacchi di *replay*.

4. **Concessione del Service-Granting Ticket:** Il TGS decifra il TGT e verifica l'Authenticator. Se valido, crea un SGT specifico per il Server V e lo invia al client insieme a una nuova chiave di sessione, il tutto cifrato con la chiave di sessione precedente. Il SGT è cifrato con una chiave segreta condivisa solo tra TGS e Server V.

5. **Richiesta del Servizio:** Il client invia al Server V il SGT e un nuovo Authenticator. Il Server V decifra il ticket, verifica l'Authenticator e concede l'accesso. Se è richiesta l'autenticazione mutua, il server risponde con un messaggio cifrato per dimostrare la propria autenticità.

21.1.2 Realm Kerberos e Kerberi Multipli

Un ambiente Kerberos completo (server, client, server applicativi) è definito **realm**, gestito da un dominio amministrativo. Per l'autenticazione *inter-realm*, i server Kerberos di due realm diversi devono condividere una chiave segreta e fidarsi reciprocamente, permettendo al client di ottenere un ticket per un TGS remoto.

21.1.3 Versioni di Kerberos

- **Versione 4:** La prima ampiamente utilizzata, basata esclusivamente su DES.
- **Versione 5:** Introdotta nel 1993 (e inclusa in Microsoft Active Directory). Introduce il supporto per algoritmi crittografici multipli (come AES), l'*Authentication Forwarding* (delega delle credenziali a un server per conto del client) e una maggiore scalabilità per l'autenticazione inter-realm.

21.2 X.509

Lo standard X.509 è il formato più accettato per i certificati a chiave pubblica, utilizzati in IPsec, TLS, S/MIME. Un certificato lega una chiave pubblica all'identità del suo proprietario ed è firmato digitalmente da una terza parte fidata, la **Certificate Authority (CA)**.

21.2.1 Formato del Certificato

Un certificato X.509 include elementi chiave come:

- Nome del Soggetto (proprietario della chiave).
- Informazioni sulla chiave pubblica del soggetto.
- Periodo di validità.
- Nome dell'Emittente (la CA).
- Firma digitale della CA.

La versione 3 include estensioni flessibili, come i *Basic Constraints*, che specificano se il certificato appartiene a una CA (abilitata a firmare altri certificati) o a un utente finale.

21.2.2 Revoca dei Certificati

Un certificato può essere revocato prima della scadenza (es. compromissione della chiave privata):

Certificate Revocation List (CRL): Liste firmate dalla CA contenenti i numeri di serie dei certificati revocati. La loro gestione è spesso onerosa e poco pratica.

Online Certificate Status Protocol (OCSP): Alternativa più leggera che permette a un'applicazione di interrogare direttamente e dinamicamente la CA sullo stato di un certificato specifico.

21.3 Public-Key Infrastructure (PKI)

Una PKI è l'insieme di hardware, software, persone, policy e procedure necessarie per creare, gestire, distribuire e revocare certificati digitali.

21.3.1 Modello di Trust e Problemi

La verifica di un certificato richiede di fidarsi della CA emittente, creando una "catena di fiducia" fino a una *Root CA*. I sistemi attuali includono un *trust store* con molte Root CA considerate ugualmente affidabili. Questo modello presenta criticità:

- **Decisioni dell'utente:** Gli utenti spesso non hanno le competenze per valutare avvisi di sicurezza sui certificati.
- **Fiducia incondizionata:** Si presume che tutte le CA nel trust store siano ugualmente sicure (l'incidente DigiNotar del 2011 ha dimostrato le falle di questo presupposto).
- **Trust store frammentati:** Browser e OS diversi usano trust store differenti, portando a visioni di sicurezza non uniformi.

21.3.2 Public Key Infrastructure X.509 (PKIX)

Il gruppo di lavoro PKIX dell'IETF ha definito un modello formale per implementare una PKI su Internet, identificando gli elementi chiave (End entity, CA, Registration Authority) e le funzioni di gestione (registrazione, certificazione, revoca).